

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новгородский государственный университет имени Ярослава Мудрого»
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

РАЗРАБОТКА БАЗЫ ДАННЫХ И ПРОГРАММНЫХ ПРОДУКТОВ ДЛЯ СИСТЕМЫ АВТОМАТИЗАЦИИ РАБОТЫ СКЛАДА

Курсовой проект по учебной дисциплине «Базы данных»
по специальности 09.03.01 Информатика и вычислительная техника
ПТИ.КП 3093.004.9ПЗ

Руководитель

_____ / Л.Н. Цымбалюк

«__» _____ 2025 г.

Студент группы 3093

_____ / Р.А. Михайлов

«__» _____ 2025 г.

Аннотация

Пояснительная записка содержит:

Объем работы:

страниц	75
рисунков	64
таблиц	15
приложений	5
библиографических источников	10

Программный продукт был разработан в процессе выполнения курсового проекта студентом Новгородского Государственного Университета имени Ярослава Мудрого Михайловым Р. А.

Данный документ содержит описание информационной системы “Склад” Разработка базы данных для автоматизации учёта товаров на складе.

Разработанный программный продукт удовлетворяет все требованиям поставленной задачи.

Содержание

Введение	5
1 Постановка задачи	7
2 Проектирование базы данных	9
2.1 Построение ER-модели базы данных	9
2.2 Построение реляционной модели базы данных	11
3 Разработка объектов базы данных	18
3.1 Создание таблиц	18
3.2 Запросы, представления	25
3.3 Хранимые функции	48
3.4 Триггеры, события	50
4 Проектирование и создание приложения	51
4.1 Описание интерфейса	53
4.2 Описание разработки интерфейса	55
4.3 Администрирование системы	56
4.4 Отчёты к базе данных	60
4.5 Тестирование работы приложения	61
4.6 Состав и содержание документации	62
5. Методы защиты базы данных	68
5.1 Защита паролем	68
5.2 Управление доступом	69
5.3 Поддержание целостности данных	71
5.4 Аутентификация	71
Заключение	73
Список литературы	75
Приложение А	76

Приложение Б	81
Приложение В	95
Приложение Г	98
Приложение Д.....	124

Введение

В современных условиях активного развития торговли и логистики особое значение приобретает автоматизация процессов складского учёта. Для предприятий розничной и оптовой торговли склад является важнейшим звеном, обеспечивающим сохранность и учёт товарно-материальных ценностей. Эффективная организация складской деятельности позволяет минимизировать издержки, оптимизировать товарные запасы и повысить оперативность работы сотрудников.

Как показывают исследования в области автоматизации учёта, ручное ведение документации нередко приводит к ошибкам, потере данных и снижению скорости обработки информации. Кроме того, отсутствие единой информационной базы затрудняет контроль за движением товаров и формирование отчётности. Всё это указывает на необходимость внедрения современных информационных систем, обеспечивающих точный учёт поступления, реализации и списания товаров.

Таким образом, актуальность темы заключается в необходимости создания автоматизированной системы, позволяющей вести учёт товарных остатков на складе, формировать отчёты о движении и переоценке товаров, а также контролировать корректность вводимых данных.

Немаловажным аспектом разработки является определение потенциальных пользователей системы. В данном случае это администраторы и операторы склада, которые будут осуществлять ввод данных о приходе и выбытии товаров, а также формировать различные отчётные документы.

Цель: проектирование и создание базы данных для автоматизации учёта складских операций.

Указанная цель определила следующие задачи:

- спроектировать модель базы данных;
- разработать объекты базы данных в СУБД PostgreSQL;

- реализовать прикладное приложение для работы с базой данных;
- обеспечить авторизацию пользователей и разграничение прав доступа;
- реализовать функции формирования отчётов и справочных документов;
- изучить и разработать методы защиты данных и обеспечения целостности информации

Объект исследования – склад предприятия.

Предмет исследования – информационная система автоматизации учёта движения товаров на складе.

1 Постановка задачи

Задачей данного курсового проектирования является проектирование и разработка информационной системы для товарного склада.

Целью системы является автоматизация процессов учёта сотрудников, поступлений, хранения, реализации и списания товаров, а также формирование отчётных и справочных документов по результатам работы склада.

В процессе функционирования системы поступает и обрабатывается следующая информация:

- наименование товара;
- категория (группа) товара;
- количество и цена товара при поступлении;
- дата поступления товара на склад;
- информация о поставщике;
- дата реализации и количество реализованного товара;
- цена реализации;
- информация о сотруднике;
- данные о пользователях системы (логин, пароль, уровень доступа);
- данные о переоценках и актах списания товара;
- ссылка на изображение и описание товара (для карточки товара).

База данных должна обеспечивать возможность:

- учёта поступления и реализации товаров за указанный период;
- формирования отчёта о реализации товара за период;
- формирования отчёта о поступлении товара на склад;
- формирования инвентаризационной ведомости остатков товаров;
- регистрации переоценок и актов списания с возможностью документального оформления;
- отображения карточки товара с изображением и описанием;

- контроля корректности вводимых данных (например, проверка наличия товара при продаже, сроков годности, уникальности кода товара);
- реализации системы авторизации пользователей с разграничением прав доступа (администратор, менеджер склада).

Для проектирования схемы базы данных используется программа DB Designer. Для реализации базы данных выбрана СУБД PostgreSQL, обеспечивающая надёжное хранение, целостность данных и поддержку транзакций. Для разработки прикладного интерфейса используется инструментальное средство Go/Gin и JavaScript/React.JS.

В системе выделяются две основные категории пользователей:

- Администратор системы — имеет полный доступ к данным, может добавлять и редактировать информацию, управлять пользователями, формировать отчёты;
- Оператор склада (пользователь с ограниченным доступом) — может вносить и просматривать информацию о поступлении и реализации товаров, формировать стандартные отчёты.

Создание базы данных включает следующие этапы:

- описание предметной области и построение модели предметной области;
- проектирование информационной модели базы данных;
- построение реляционной модели базы данных (ER-диаграммы);
- разработка базы данных в СУБД PostgreSQL;
- разработка приложения для работы с базой данных;
- описание методов защиты данных и разграничения доступа;
- разработка справочного руководства пользователя.

2 Проектирование базы данных

2.1 Построение ER-модели базы данных

База данных создаётся для системы автоматизации работы склада, позволяющей пользователям просматривать информацию о находящихся на складе товарах, документах, а также дающей возможность приёмки, выбытия и переоценки товаров. БД должна содержать данные о товарах, поставках, производителях, сотрудниках и пользователях. В соответствии с предметной областью система строится с учётом следующих особенностей:

- разная продукция может быть произведена одним производителем;
- множество документов может быть оформлено одним сотрудником;
- у одного сотрудника может быть не более одной учётной записи в система;
- одна категория документов может быть у множества документов;
- продукция может быть принята во множестве документов;
- в одном документе может быть много строк;
- у каждого документа только один тип документа;
- один продукт может поступать на склад множество раз.

Выделим базовые сущности этой предметной области:

- производители (ID (PK), название, ID адреса, инн, фамилия, имя, отчество);
- продукция (ID (PK), название, описание, ID категории продукции ID производитель);
- категория продукции (ID (PK), название);
- партии (ID (PK), дата производства, срок годности, цена);
- документы (ID (PK), дата, ID категории, ID сотрудника);

- содержания документов (ID (PK), ID партия, ID документ, ID продукция, количество);
- категории документов (ID (PK), название, описание);
- сотрудники (ID (PK), фамилия, имя, отчество, ID должности, инн, телефон, адрес, дата рождения, пол);
- пользователи (ID (PK), ID сотрудника, ID роли, логин, пароль);
- должности (ID (PK), название, описание, оклад);
- роли (ID (PK), название, описание);
- адреса (ID (PK), субъект, район, город, улица, дом);
- пол (ID (PK), обозначение);

ER-модель можно увидеть на рисунке А.7.

Произведём анализ сущностей:

Сущности находятся в первой нормальной форме, так как удовлетворяют следующему требованию: Сущность не должна иметь многозначных атрибутов.

Сущность находится во второй и третьей нормальной форме, так как она удовлетворяет условиям второй нормальной формы, и ни одно из не ключевых полей таблицы не идентифицируется с помощью другого не ключевого поля.

2.2 Построение реляционной модели базы данных

2.2.1 Структурная часть

В информационной системе базы данных курсового проекта используются таблицы 1-13.

Таблица 1 – Address

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
subject	Строковый	100	-	Нет
region	Строковый	100	-	Нет
city	Строковый	100	-	Нет
street	Строковый	100	-	Нет
building	Числово	3	1-999	Нет

Таблица 2 – Batch

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
cost	Вещетсвенный	-	>= 0	Нет
production_date	Дата	-	1000-01-01— 9999-12-31	Нет
expiration_date	Дата	-	1000-01-01— 9999-12-31	Нет
id product	Числвой	-	> 0	Нет
createdt_at	Дата	-	1000-01-01— 9999-12-31	Нет

Таблица 3 – Document

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
date	Дата	-	1000-01-01— 9999-12-31	Нет
id_employee	Числовой	-	> 0	Нет
id_document_category	Числовой	-	> 0	Нет

Таблица 4 – Document_category

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Числовой	-	> 0	Да
name	Строковый	50	-	Нет
description	Текст	-	-	Нет

Таблица 5 – Document_content

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
id_codument	Числовой	-	> 0	Нет
id_batch	Числовой	-	> 0	Нет
id_product	Числовой	-	> 0	Нет
quantity	Числовой	-	> 0	Нет

Таблица 6 – Employee

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
surname	Строковый	50	-	Нет
firstname	Строковый	50	-	Нет
patronymic	Строковый	50	-	Нет
id_gender	Числовой	1	1-2	Нет
inn	Строковый	12	-	Нет
phone_number	Строковый	16	-	Нет
id_address	Числовой	-	> 0	Нет
birth_date	Дата	-	1000-01-01— 9999-12-31	Нет
id_position	Числовой	-		Нет

Таблица 7 – Gender

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
sing	Строковый	1	“М” или “Ж”	Нет

Таблица 8 – Position

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
name	Строковый	50	-	Нет
description	Текст	-	-	Нет

Таблица 9 – Producer

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
name	Строковый	100	-	Нет
id_address	Числовой	-	> 0	Нет
inn	Строковый	10	-	Нет
surname	Строковый	50	-	Нет
firstname	Строковый	50	-	Нет
patronymic	Строковый	50	-	Нет

Таблица 10 – Product

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
name	Строковый	100	-	Нет
id_product_category	Числовой	-	> 0	Нет
id_product	Числовой	-	> 0	Нет

Таблица 11 – Product_category

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
name	Строковый	50	-	Нет

Таблица 12 – Role

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
name	Строковый	50	-	Нет
description	Текст	-	-	Нет

Таблица 13 – Sys_user

Название атрибута	Тип данных	Длина в знаках	Диапазон принимаемых значений	Ключевое поле
id	Счётчик	-	> 0	Да
login	Строковый	50	-	Нет
password_hash	Строковый	60	-	Нет
id_role	Числовой	-	> 0	Нет
id_employee	Числовой	-	> 0	Нет

2.2.2 Целостная часть

Схема реляционной модели базы данных представлена на рисунке А.8 и включает следующие объекты:

- address – Родительская таблица по отношению к таблицам producer и employee.
- batch – Родительская таблица по отношению к таблице document_content. Дочерняя таблица по отношению к таблице product. Внешний ключ id_product ссылается на поле product.id.
- document – Родительская таблица по отношению к таблице document_content. Дочерняя таблица по отношению к таблицам document_category и employee. Внешний ключ id_document_category ссылается на поле document_category.id. Внешний ключ id_employee ссылается на поле employee.id.
- document_category – Родительская таблица по отношению к таблице document.
- document_content – Дочерняя таблица по отношению к таблицам product и batch. Внешний ключ id_product ссылается на поле product.id. Внешний ключ id_batch ссылается на поле batch.id.
- employee – Родительская таблица по отношению к таблицам document и sys_user. Дочерняя таблица по отношению к таблицам position, address и gender. Внешний ключ id_position ссылается на поле position.id. Внешний ключ id_gender ссылается на поле gender.id. Внешний ключ id_address ссылается на поле address.id.
- gender – Родительская таблица по отношению к таблице employee.
- position – Родительская таблица по отношению к таблице employee.

- `producer` – Родительская таблица по отношению к таблице `product`. Дочерняя таблица по отношению к таблице `address`. Внешний ключ `id_address` ссылается на поле `address.id`.
- `product` – Родительская таблица по отношению к таблицам `document_content` и `batch`. Дочерняя таблица по отношению к таблицам `producer` и `product_category`. Внешний ключ `id_producer` ссылается на поле `producer.id`. Внешний ключ `id_batch` ссылается на поле `batch.id`.
- `product_category` – Родительская таблица по отношению к таблице `product`.
- `role` – Родительская таблица по отношению к таблице `sys_user`.
- `sys_user` – Дочерняя таблица по отношению к таблицам `employee` и `role`. Внешний ключ `id_employee` ссылается на поле `employee.id`. Внешний ключ `id_role` ссылается на поле `role.id`.

2.2.3 Манипуляционная часть

Возможные запросы к базе данных информационной системы представлены в таблице 14.

Таблица 14 – Возможные запросы к базе данных информационной системы

Запрос	Описание
Список партий, полученных от определённого поставщика	По данному запросу происходит объединение таблиц Партия, Продукт и Производитель по связям id_product и id_producer. Фильтрация по имени производителя «ООО "СтильДрев"» позволяет вывести только те партии товаров, которые были получены от указанного поставщика.
Список продуктов, которых нет на складе	Запрос вычисляет текущие остатки товаров через анализ таблицы document_content (учитывая приход, расход и игнорируя переоценку). Затем выбираются продукты, у которых суммарный остаток отсутствует. В результате выводится перечень товаров из таблицы Product, которых нет на складе.
Количество оставшихся на складе товаров каждого типа	Запрос суммирует приход и расход по каждому продукту, обрабатывая данные из document_content и учитывая тип документа (приход или расход). Затем присоединяется таблица Product, чтобы вывести название товара. Результатом будет список товаров и количество единиц каждого, оставшихся на складе.
Список сотрудников, которые могут пользоваться системой	По данному запросу происходит соединение таблиц Employee, Sys_user и Position по ключу сотрудника. Выводятся сотрудники, имеющие учетную запись в системе, а также их должности — то есть все работники, которые имеют доступ к системе.
Список партий, принятых за определённый период времени	Запрос выбирает партии из таблицы Batch, id которых встречается в документах категории «приход» (id_document_category = 1). Затем данные фильтруются по дате документа в указанном интервале. Результат — список партий, которые были приняты на склад за заданный период.

3 Разработка объектов базы данных

3.1 Создание таблиц

Для создания таблиц использовался sql-скрипт, представленный в листинге Б.1, на котором отражаются созданные первичные и внешние ключи, а также необходимые ограничительные условия.

Связи между таблицами устанавливаются с помощью инструкций SQL, описанных выше. Вид отношений между таблицами – один-ко-многим или отношением многие-к-одному. Схема данных представлена на рисунке А.9.

Источниками данных могут служить данные, сгенерированные нейросетью. Таблицы заполнены тестовыми данными.

Ниже представлены таблицы с тестовыми данными, необходимыми для дальнейшей работы.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			☒	☒	nextval('a
subject	character varying	100		☒	☐	
region	character varying	100		☒	☐	
city	character varying	100		☒	☐	
street	character varying	100		☒	☐	
building	integer			☒	☐	

	id [PK] integer	subject character varying (100)	region character varying (100)	city character varying (100)	street character varying (100)	building integer
1	1	Новгородская область	Новгородский	Великий Новгород	Большая Санкт-Петербургская	25
2	2	Новгородская область	Новгородский	Великий Новгород	Людогоща	14
3	3	Новгородская область	Новгородский	Великий Новгород	Федоровский ручей	2
4	4	Новгородская область	Новгородский	Великий Новгород	Богдана Хмельницкого	42
5	5	Новгородская область	Новгородский	Великий Новгород	Великая	8

Рисунок 1 – Таблица Адрес, тестовые данные таблицы

Таблица address: первичный ключ – id. Все поля таблицы NOT NULL, длина subject, region, city и street не может быть больше 100 символов. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer v			☒	☒	nextval('t
cost	real v			☒	☐	
production_date	date v			☒	☐	
expiration_date	date v			☐	☐	
id_product	integer v			☒	☐	
created_at	timestamp without v			☒	☐	CURREN'

	id [PK] integer	cost real	production_date date	expiration_date date	id_product integer	created_at timestamp without time zone
1	2	48800	2025-02-01	2027-02-01	2	2025-10-15 08:39:40.31846
2	3	18900	2025-02-20	2029-02-20	3	2025-10-15 08:39:40.31846
3	5	89990	2025-03-05	2028-03-05	2	2025-10-15 08:39:40.31846
4	4	24300	2024-01-20	2034-02-20	1	2025-10-15 08:39:40.31846
5	8	2	2025-10-30	2026-01-30	1	2025-11-30 14:27:02.406452

Рисунок 2 – Таблица Партия, тестовые данные таблицы

Таблица batch: первичный ключ – id. Все поля, кроме expiration_date NOT NULL. По умолчанию ключевое поле увеличивается на 1, created_at текущая временная метка.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer v			☒	☒	nextval('c
date	date v			☒	☐	
id_employee	integer v			☒	☐	
id_document_ca	integer v			☒	☐	

	id [PK] integer	date date	id_employee integer	id_document_category integer
1	1	2024-03-10	2	1
2	2	2024-03-18	2	2
3	3	2024-03-25	2	3
4	4	2025-11-22	1	1
5	5	2025-11-22	1	1

Рисунок 3 – Таблица Документ, тестовые данные таблицы

Таблица document: первичный ключ – id. Все поля NOT NULL. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer v			☒	☒	nextval('c
name	character varying v	50		☒	☐	
description	text v			☒	☐	

	id [PK] integer	name character varying (50)	description text
1	1	Поступление	Фиксация поступления разных партии товаров различных категорий
2	2	Списание	Фиксация выбытия определённого количества товаров различных категорий из разных партий
3	3	Переоценка	Фиксация изменение стоимости 1 у. е. продукции определённой категории в определённой партии

Рисунок 4 – Таблица Категория документа, тестовые данные таблицы

Таблица document_category: первичный ключ – id. Все поля NOT NULL, поле name, обозначающее название категории, не может быть больше 50 символов. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer v			☒	☒	nextval('c
id_document	integer v			☒	☐	
id_batch	integer v			☒	☐	
quantity	integer v			☒	☐	

	id [PK] integer	id_document integer	id_batch integer	quantity integer
1	1	1	1	5
2	2	1	2	3
3	3	1	3	10
4	4	1	4	15
5	5	1	5	2

Рисунок 5 – Таблица Содержание документа, тестовые данные таблицы

Таблица document_content: первичный ключ – id. Все поля NOT NULL. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer v			☒	☒	nextval('i
surname	character varying v	50		☒	☐	
firstname	character varying v	50		☒	☐	
patronymic	character varying v	50		☐	☐	
id_gender	integer v			☒	☐	
inn	character varying v	12		☒	☐	
phone_number	character varying v	16		☒	☐	
id_address	integer v			☒	☐	
birth_date	date v			☒	☐	
id_position	integer v			☒	☐	

	id [PK] integer	surname character var	firstname character v	patronymic character varying	id_gend integer	inn character varying (	phone_number character varying (1	id_addre integer	birth_date date	id_posit integer
1	2	Соколова	Анна	Сергеевна	2	525209876543	+7 911 987-65-43	2	1992-08-23	2
2	3	Павлов	Иван	Олегович	1	525205678901	+7 911 456-78-90	3	1988-11-08	3
3	4	Орлов	Денис	Романович	1	525203456789	+7 911 234-56-78	4	1995-02-17	3
4	5	Никитин	Петр	Алексеевич	1	525201987654	+7 911 765-43-21	5	1983-07-30	3
5	1	Волков	Артем	Дмитриевич	1	525201234567	+7 911 123-45-67	1	1985-05-12	1

Рисунок 6 – Таблица Сотрудник, тестовые данные таблицы

Таблица employee: первичный ключ – id. Все поля, кроме patronymic, обозначающее отчество, NOT NULL, поля surname, firstname, patronymic, обозначающие фамилию, имя и отчество соответственно, не может быть больше 50 символов, поле inn, обозначающее ИНН, не может быть больше 12 символов, поле phone_number, обозначающее номер телефон, не может быть больше 16 символов. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			☒	☒	nextval('t
name	character varying	50		☒	☐	
description	text			☒	☐	

	id [PK] integer	name character varying (50)	description text
1	1	Начальник склада	Общее управление, планирование и отчётность
2	2	Кладовщик	Приёмка и отгрузка, учёт и сохранность
3	3	Грузчик	Комплектация заказов, помощь при приёмке
4	4	Не указано	Не указано
5	9	Системный администратор	Администрирование информационной системой, полный доступ к ИС

Рисунок 7 – Таблица Должность, тестовые данные таблицы

Таблица position: первичный ключ – id. Все поля NOT NULL, поле name, обозначающее название должности, не может быть больше 50 символов. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			☒	☒	nextval('t
name	character varying	100		☒	☐	
id_address	integer			☒	☐	
inn	character	10		☒	☐	
surname	character varying	50		☒	☐	
firstname	character varying	50		☒	☐	
patronymic	character varying	50		☐	☐	

	id [PK] integer	name character varying (100)	id_address integer	inn character (10)	surname character varying (50)	firstname character varying (50)	patronymic character varying (50)
1	1	ООО "СтильДрев"	6	2547890123	Громов	Алексей	Викторович
2	2	АО "БытМашПром"	7	6634517890	Захарова	Ольга	Николаевич
3	3	ООО "МикроТех"	8	8901234567	Белочкин	Дмитрий	Игоревич
4	4	ИП "Михайлов"	1	1234323858	Михайлов	Роман	Александрович
5	5	АО "АвтоВлад"	6	5467823934	Судный	Максим	Рэмович

Рисунок 8 – Таблица Производитель, тестовые данные таблицы

Таблица producer: первичный ключ – id. Все поля NOT NULL, кроме patronymic, обозначающее отчество, поле name, обозначающее название производителя, не может быть больше 100 символов, поле inn, обозначающее

ИНН, не может быть больше 10 символов, поля surname, firstname, patronymic, обозначающие фамилию, имя и отчество соответственно, не могут быть больше 50 символов. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			☒	☒	nextval('t
name	character varying	100		☒	☐	
id_product_cate	integer			☒	☐	
id_producer	integer			☒	☐	
image_url	text			☒	☐	'placeholder

	id [PK] integer	name character varying (100)	id_produ integer	id_producer integer	image_url text
1	1	Кухонный гарнитур "Уют"	1	1	/static/products/1765011046095675919_a4c7c78c78a454e231f7718718ae6195.jpg
2	2	Стиральная машина "SM-5000"	2	2	/static/products/1765011070099983639_images (3).jpeg
3	4	Офисное кресло "Director"	1	1	/static/products/1765011089390225675_images (2).jpeg
4	5	Холодильник "Frost+ 300"	2	2	/static/products/1765011109815931171_images.jpeg
5	7	Кухонный гарнитур "Модерн"	1	19	/static/products/1765354277653639797_7-.webp

Рисунок 9 – Таблица Товар, тестовые данные таблицы

Таблица product: первичный ключ – id. Все поля NOT NULL, поле name, обозначающее название товара, не может быть больше 100 символов. По умолчанию ключевое поле увеличивается на 1, поле image_url, обозначающее ссылку на изображение, присваивается значение “/static/products/placeholder.png”, ссылка на заполнитель.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			☒	☒	nextval('t
name	character varying	50		☒	☐	

	id [PK] integer	name character varying (50)
1	1	Мебель
2	2	Электроника
3	3	Бытовая техника
4	8	Посуда

Рисунок 10 – Таблица Категория товара, тестовые данные таблицы

Таблица `product_category`: первичный ключ – `id`. Все поля NOT NULL, поле `name`, обозначающее название товара, не может быть больше 50 символов. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			☒	☒	nextval('r
name	character varying	50		☒	☐	
description	text			☒	☐	
sys_role	character varying	50		☒	☐	
	id [PK] integer	name character varying (50)	description text	sys_role character varying (50)		
1	1	Модератор	Модерирование базы данных, выполнение CRUD операций	moderator		
2	2	Менеджер	Пользование БД, использование готовых запросов	manager		
3	4	Администратор	Администрирование база данные, полный доступ ко всем объектам	admin		

Рисунок 10 – Таблица Роль, тестовые данные таблицы

Таблица `role`: первичный ключ – `id`. Все поля NOT NULL, поля `name` и `role`, обозначающие название роли и роль соответственно, не могут быть больше 50 символов. По умолчанию ключевое поле увеличивается на 1.

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			☒	☒	nextval('s
login	character varying	50		☒	☐	
password_hash	character	60		☒	☐	
id_role	integer			☒	☐	
id_employee	integer			☒	☐	
	id [PK] integer	login character varying (50)	password_hash character (60)	id_role integer id_employee integer		
1	5	moderator_login	\$2a\$10\$Pw6ZaIdf.CT.IKiRy8RYWouV5SB14tmuCmwBYCwQ7KnOaCgJC...	1	5	
2	2	anna_sokolova	\$2a\$10\$ONDNHqgfrEHpioEyaVSDxOG3icMHs3nEsqP1TpzC9jYgB142ezb...	2	2	
3	7	roman	\$2a\$10\$4MWSzOFhEH9X4P25w7YgCeJkH5FoB8lx4S69iRPsnFunJIPOX...	4	14	
4	1	artem_volkov	\$2a\$10\$z9I2uGqAHHc5paoMn9T5yOXLEEDnWMj/Pmwegsv/vdagYct7R...	1	1	
5	9	test	\$2a\$10\$J0iUC7j6gA0qCYuxQRUEOeN62hUua/rz6OlrmQuGfGRPbB0rZ8o02	2	17	

Рисунок 11 – Таблица Системная роль, тестовые данные таблицы

Таблица role: первичный ключ – id. Все поля NOT NULL, поля login и password_hash, обозначающие название логин и хэш пароля соответственно, не могут быть больше 50 и 60 символов соответственно. По умолчанию ключевое поле увеличивается на 1.

3.2 Запросы, представления

Запросы – это формализованное сообщение, содержащее условие (простое или сложное) на поиск данных и указание о том, что необходимо сделать с найденными данными.

Запрос 1

Выборка названий продуктов и их производителей.

```
SELECT
    pt.id,
    pt.name,
    pr.name
FROM product AS pt
JOIN producer AS pr
    ON pt.id_producer = pr.id
ORDER BY pt.name, pr.name
```

Рисунок 12 – Выборка названий продуктов и их производителей

Выводится id товара, название товара и название производителя, при помощи соединения таблиц product и producer по полям связи id_producer и id, сортировка по названию товара и названию производителя.

Результат выполнения запроса:

	id integer	name character varying (100)	name character varying (100)
1	7	Кухонный гарнитур "Модерн"	АО "СтройМебель"
2	1	Кухонный гарнитур "Уют"	ООО "СтильДрев"
3	3	Материнская плата "Gamer XTREM..."	ООО "МикроТех"
4	4	Офисное кресло "Director"	ООО "СтильДрев"
5	8	Стиральная машина "EcoWash 3000"	ООО "ЭлектроникСистем"
6	2	Стиральная машина "SM-5000"	АО "БытМашПром"
7	5	Холодильник "Frost+ 300"	АО "БытМашПром"

Рисунок 13 – Результат выполнения запроса: выборка названий продуктов и их производителей

Запрос 2

Выборка названий производителей и категорий товаров, которые они производят.

```
SELECT
    pr.id,
    pr.name,
    ARRAY_AGG(DISTINCT pc.name) AS categories
FROM producer AS pr
JOIN product AS pt
    ON pt.id_producer = pr.id
JOIN product_category AS pc
    ON pt.id_product_category = pc.id
GROUP BY pr.id, pr.name
```

Рисунок 14 – Выборка названий производителей и категорий товаров, которые они производят

Выводится id производителя, название производителя и массив названий категорий товаров, которые он производит, при помощи соединения таблиц producer и product по полям связи id_producer и id, соединения таблиц product и product_category по полям связи id_prouct_category и id, группировка по id производителя и названию производителя.

Результат выполнения запроса:

	id [PK] integer	name character varying (100)	categories character varying[]
1	1	ООО "СтильДрев"	{Мебель}
2	2	АО "БытМашПром"	{Электроника}
3	3	ООО "МикроТех"	{Электроника}
4	19	АО "СтройМебель"	{Мебель}
5	20	ООО "ЭлектроникСистем"	{"Бытовая техника",Электроника}
6	21	ИП "Кулинария"	{Посуда}
7	24	ООО "МебельЛюкс"	{Мебель,Посуда,Электроника}
8	25	ИП "ТехМаркет"	{Электроника}
9	26	ООО "ХолодСервис"	{"Бытовая техника"}

Рисунок 15 – Результат выполнения запроса: выборка названий производителей и категорий товаров, которые они производят

Запрос 3

Выборка названий категорий продукции и количество производителей, которые их производят.

```
SELECT
    pc.id,
    pc.name,
    COUNT(pc.id) AS producers_quantity
FROM product_category AS pc
JOIN product AS pt
    ON pt.id_product_category = pc.id
JOIN producer AS pr
    ON pt.id_producer = pr.id
GROUP BY pc.id, pc.name
ORDER BY LENGTH(pc.name), pc.name ASC
```

Рисунок 16 – Выборка названий категорий продукции и количество производителей, которые их производят

Выводится id категории товара, название категории товара и количество производителей, производящих товары этой категории, при помощи соединения таблиц product_category и product по полям связи id и id_product_category, соединения таблиц product и producer по полям связи id_producer и id, группировка по id категории продукта и названию категории продукта, сортировка по длине названия категории продукта и по названию категории продукта.

Результат выполнения запроса:

	id [PK] integer	name character varying (50)	producers_quantity bigint
1	1	Мебель	18
2	8	Посуда	7
3	2	Электроника	21
4	3	Бытовая техника	9

Рисунок 17 – Результат выполнения запроса: выборка названий категорий продукции и количество производителей, которые их производят

Запрос 4

Выборка товаров, в названии которых есть слово через дефис.

```
SELECT
    id,
    name
FROM product
WHERE name LIKE '%-%'
```

Рисунок 18 – Выборка товаров, в названии которых есть слово через дефис

Выводится id товара и название товара, условием выборки является соответствие названия товара шаблону.

Результат выполнения запроса:

	id [PK] integer	name character varying (100)
1	2	Стиральная машина "SM-5000"

Рисунок 19 – Результат выполнения запроса: выборка товаров, в названии которых есть слово через дефис

Запрос 5

Выборка документов о поступлении и списании за 2025 год

```
SELECT
    d.id,
    d.date,
    dc.name
FROM document AS d
JOIN document_category AS dc
    ON d.id_document_category = dc.id
WHERE (date BETWEEN '2025-01-01' AND '2025-12-31')
    AND dc.name IN ('Поступление', 'Списание')
```

Рисунок 20 – Выборка документов о поступлении и списании за 2025 год

Выводится id документа, дата создания документа и название категории документа, при помощи соединения таблиц document_category и document по полям связи id и id_document_category, условием выборки является соответствие промежутку времени и соответствие названию категории документа.

Результат выполнения запроса:

	id integer	date date	name character varying (50)
1	4	2025-11-22	Поступление
2	5	2025-11-22	Поступление

Рисунок 21 – Результат выполнения запроса: выборка документов о поступлении и списании за 2025 год

Запрос 6

Выборка количества оформленных документов по сотрудникам.

```
SELECT
    ROW_NUMBER() OVER(PARTITION BY e.id),
    e.surname || ' ' || e.firstname || ' ' || e.patronymic AS name,
    dc.name AS document_category,
    COUNT(d.id) AS documents_quantity
FROM document AS d
JOIN employee AS e
    ON e.id = d.id_employee
JOIN document_category AS dc
    ON d.id_document_category = dc.id
GROUP BY e.id, dc.name
```

Рисунок 22 – Выборка количества оформленных документов по сотрудникам

Выводится номер строки, полное имя сотрудника, название категории документа и количество документов в группе, при помощи соединения таблиц document и employee по полям связи id_employee и id, соединения таблиц document и document_category по полям связи id_document_category и id, группировка по id сотрудника и названию категории документа.

Результат выполнения запроса:

	row_number bigint	name text	document_category character varying (50)	documents_quantity bigint
1	1	Волков Артем Дмитриев...	Переоценка	1
2	2	Волков Артем Дмитриев...	Поступление	2
3	1	Соколова Анна Сергеевна	Переоценка	1
4	2	Соколова Анна Сергеевна	Поступление	1
5	3	Соколова Анна Сергеевна	Списание	1

Рисунок 23 – Результат выполнения запроса: выборка количества оформленных документов по сотрудникам

Запрос 7

Выборка товаров, которые были хотя бы раз приняты в партиях.

```
SELECT
    pt.id,
    pt.name,
    pc.name AS product_category
FROM product AS pt
JOIN product_category AS pc
    ON pt.id_product_category = pc.id
WHERE pt.id = ANY(SELECT id_product FROM batch)
```

Рисунок 24 – Выборка товаров, которые были хотя бы раз приняты в партиях

Выводится id товара, название товара, название категории товара, при помощи соединения таблиц product и product_category по полям связи id_product_category и id, условием выборки является совпадение id товара с хотя бы одним из id_product из подзапроса, где выводятся все id товаров из таблицы batch.

Результат выполнения запроса:

	id integer	name character varying (100)	product_category character varying (50)
1	1	Кухонный гарнитур "Уют"	Мебель
2	2	Стиральная машина "SM-5000"	Электроника
3	4	Офисное кресло "Director"	Мебель
4	3	Материнская плата "Gamer XTREME"	Электроника

Рисунок 25 – Результат выполнения запроса: выборка товаров, которые были хотя бы раз приняты в партиях

Запрос 8

Выборка товаров, которые никогда не фигурировали ни в одной из партий на складе.

```

SELECT
    pt.id,
    pt.name
FROM product AS pt

EXCEPT

SELECT
    pt.id,
    pt.name
FROM batch AS b
JOIN product AS pt
    ON b.id_product = pt.id

```

Рисунок 26 – Выборка товаров, которые никогда не фигурировали ни в одной из партий на складе

Выводится id товара и название товара, путём разности всех товаров из тех, которые есть в таблице batch, путём соединения таблиц batch и product по полям связи id_product и id.

Результат выполнения запроса:

	id integer	name character varying (100)
1	5	Холодильник "Frost+ 300"
2	7	Кухонный гарнитур "Модерн"
3	8	Стиральная машина "EcoWash 3000"
4	9	Холодильник "CoolFridge X"
5	10	Материнская плата "Gamer Pro"

Рисунок 27 – Результат выполнения запроса: выборка товаров, которые никогда не фигурировали ни в одной из партий на складе

Запрос 9

Выборка сотрудников и их роли в системе.

```

WITH employees_names AS (
    SELECT
        id,
        surname,
        firstname,
        patronymic
    FROM employee
)
SELECT
    en.surname || ' ' || en.firstname || ' ' || en.patronymic AS name,
    COALESCE(su.id::TEXT, '-') AS id_user,
    COALESCE(r.sys_role, '-') AS role
FROM sys_user AS su
RIGHT JOIN employees_names AS en
    ON su.id_employee = en.id
LEFT JOIN role AS r
    ON su.id_role = r.id
ORDER BY r.sys_role

```

Рисунок 28 – Выборка сотрудников и их роли в системе

Выводится полное имя сотрудника, id пользователя и название роль, путём внешнего правого соединения вынесенного подзапроса employees_names (id сотрудника, его имя, фамилия и отчество) и таблицы sys_user по полям связи id и id_employee, внешнего левого соединения таблиц role и sys_user по полям связи id и id_role, сортировка по названию роли.

Результат выполнения запроса

	name text	id_user text	role character varying
1	Михайлов Роман Александрович	7	admin
2	Соколова Анна Сергеевна	2	manager
3	test test test	9	manager
4	Волков Артем Дмитриевич	1	moderator
5	Никитин Петр Алексеевич	5	moderator
6	Орлов Денис Романович	-	-
7	Павлов Иван Олегович	-	-

Рисунок 29 – Результат выполнения запроса: выборка сотрудников и их роли в системе

Запрос 10

Выборка производителей и количества их товаров, которые фигурировали хотя бы раз в партиях.

```

SELECT
    pr.name AS producer_name,
    COUNT(sub.id_product) AS products_in_batches
FROM producer AS pr
JOIN (
    SELECT DISTINCT pt.id AS id_product, pt.id_producer
    FROM product AS pt
    JOIN batch AS b
    ON pt.id = b.id_product
) AS sub
ON pr.id = sub.id_producer
GROUP BY pr.name
ORDER BY products_in_batches DESC, pr.name;

```

Рисунок 30 – Выборка производителей и количества их товаров, которые фигурировали хотя бы раз в партиях

Выводится название производителя и количество произведённых им товаров, находящихся на складе, путем соединения табличного подзапроса (id товара и id производителя этого товара) и таблицы producer по полям связи id_producer и id, группировка по названию производителя, сортировка по количеству товаров по убыванию, по названию производителя по возрастанию.

Результат выполнения запроса:

	producer_name character varying (100) 🔒	products_in_batches bigint 🔒
1	ООО "СтильДрев"	2
2	АО "БытМашПром"	1
3	ООО "МикроТех"	1

Рисунок 31 – Результат выполнения запроса: выборка производителей и количества их товаров, которые фигурировали хотя бы раз в партиях

Представления — это виртуальные таблицы. В отличие от таблиц, содержащих данные, представления содержат запросы, которые динамически выбирают данные, когда это необходимо. Представление может содержать данные из одной или нескольких таблиц или может быть создано на основе других представлений. При создании представлений использовался sql-скрипт, который представлен в листинге Б.2.

Представление report_batches

Вывод информации о партиях и название товара в партии

```
CREATE VIEW report_batches AS
SELECT b.id,
       p.name,
       b.cost,
       b.production_date,
       b.expiration_date
FROM batch b
JOIN product p ON b.id_product = p.id;
```

Рисунок 30 – Представление report_batches

Результат выборки для представления

	id integer	name character varying (100)	cost real	production_date date	expiration_date date
1	1	Кухонный гарнитур "Уют"	125500	2024-01-15	2024-01-15
2	8	Кухонный гарнитур "Уют"	2	2025-10-30	2026-01-30
3	4	Кухонный гарнитур "Уют"	24300	2024-01-20	2034-02-20
4	5	Стиральная машина "SM-5000"	89990	2025-03-05	2028-03-05
5	2	Стиральная машина "SM-5000"	48800	2025-02-01	2027-02-01
6	10	Офисное кресло "Director"	10	2025-11-29	2025-12-30
7	11	Материнская плата "Gamer XTREME"	134	2024-12-06	2024-12-05
8	9	Материнская плата "Gamer XTREME"	2	2025-01-30	2025-12-30
9	3	Материнская плата "Gamer XTREME"	18900	2025-02-20	2029-02-20

Рисунок 31 – Результат выборки для представления: report_batches

Представление содержит в себе информацию об id партии, названии товара, стоимости за 1 у.е., дате производства и сроке годности.

Представление report_documents_by_employee

Вывод информации о сотрудниках и количестве оформленных ими документов по категориям документов.

```
CREATE VIEW report_documents_by_employee AS
SELECT
    dense_rank() OVER (ORDER BY e.surname, e.firstname, e.patronymic) AS employee_number,
    (((e.surname::text || ' '::text) || e.firstname::text) || ' '::text) || e.patronymic::text AS employee,
    p.name AS "position",
    dc.name AS document_category,
    t.count AS documents
FROM employee e
JOIN (
    SELECT
        e_1.id AS employee_id,
        d.id_document_category,
        count(*) AS count
    FROM document d
    JOIN employee e_1
        ON d.id_employee = e_1.id
    GROUP BY e_1.id, d.id_document_category
) AS t
    ON e.id = t.employee_id
JOIN "position" p
    ON p.id = e.id_position
JOIN document_category dc
    ON t.id_document_category = dc.id
ORDER BY (((e.surname::text || ' '::text) || e.firstname::text) || ' '::text) || e.patronymic::text);
```

Рисунок 32 – Представление report_documents_by_employee

Результат выборки для представления

	employee_number bigint	employee text	position character varying (50)	document_category character varying (50)	documents bigint
1	1	Волков Артем Дмитриев...	Начальник склада	Поступление	2
2	1	Волков Артем Дмитриев...	Начальник склада	Переоценка	1
3	2	Соколова Анна Сергеевна	Кладовщик	Переоценка	1
4	2	Соколова Анна Сергеевна	Кладовщик	Поступление	1
5	2	Соколова Анна Сергеевна	Кладовщик	Списание	1

Рисунок 33 – Результат выборки для представления:

report_documents_by_employee

Представление содержит информацию о номере сотрудника в рамках окна, полное имя сотрудника, должность сотрудника, название категории документа и количество документов по этой категории.

Представление report_employees

Вывод информации о сотрудниках.

```
CREATE VIEW report_employees AS
SELECT
    row_number() OVER () AS number,
    t.surname,
    t.firstname,
    t.patronymic,
    t.name AS "position",
    t.phone_number,
    t.birth_date
FROM (
    SELECT e.surname,
           e.firstname,
           e.patronymic,
           e.phone_number,
           e.birth_date,
           p.name
    FROM employee e
    JOIN "position" p
      ON e.id_position = p.id
    ORDER BY e.surname, e.firstname, e.patronymic
) AS t;
```

Рисунок 32 – Представление report_employees

Результат выборки для представления

	number bigint	surname character varying (50)	firstname character varying (50)	patronymic character varying (50)	position character varying (50)	phone_number character varying (16)	birth_date date
1	1	test	test	test	Грузчик	+7 900 000-00-00	2001-06-16
2	2	Волков	Артем	Дмитриевич	Начальник склада	+7 911 123-45-67	1985-05-12
3	3	Михайлов	Роман	Александрович	Системный администратор	+7 921 693-19-54	2005-07-22
4	4	Никитин	Петр	Алексеевич	Грузчик	+7 911 765-43-21	1983-07-30
5	5	Орлов	Денис	Романович	Грузчик	+7 911 234-56-78	1995-02-17
6	6	Павлов	Иван	Олегович	Грузчик	+7 911 456-78-90	1988-11-08
7	7	Соколова	Анна	Сергеевна	Кладовщик	+7 911 987-65-43	1992-08-23

Рисунок 33 – Результат выборки для представления: report_employees

Представление содержит информацию о номере строки, фамилии, имени и отчестве сотрудника, должности сотрудника, номере телефона сотрудника и дате рождения сотрудника.

Представление report_expired_batches

Выводит информацию о просроченных партиях.

```
CREATE VIEW report_expired_batches AS
SELECT
  row_number() OVER () AS number,
  b.id AS batch_id,
  p.name AS product_name,
  b.expiration_date,
  COALESCE(sum(
    CASE
      WHEN d.id_document_category = 1 THEN dc.quantity
      WHEN d.id_document_category = 2 THEN - dc.quantity
      ELSE NULL::integer
    END), 0::bigint) AS remaining_quantity
FROM batch b
  JOIN product p ON b.id_product = p.id
  LEFT JOIN document_content dc ON dc.id_batch = b.id
  LEFT JOIN document d ON d.id = dc.id_document
WHERE b.expiration_date <= CURRENT_DATE
GROUP BY b.id, b.expiration_date, p.name
HAVING COALESCE(sum(
  CASE
    WHEN d.id_document_category = 1 THEN dc.quantity
    WHEN d.id_document_category = 2 THEN - dc.quantity
    ELSE NULL::integer
  END), 0::bigint) > 0
ORDER BY b.expiration_date;
```

Рисунок 34 – Представление report_expired_batches

Результат выборки для представление

	number bigint	batch_id integer	product_name character varying (100)	expiration_date date	remaining_quantity bigint
1	1	1	Кухонный гарнитур "Уют"	2024-01-15	1

Рисунок 35 – Результат выборки для представления: report_expired_batches

Представление содержит информацию о номере строки, id партии, названию товара, дата истечения срока годности, количество товаров, которые еще остались на складе.

Представление report_grants

Выводит информацию о привилегиях ролей.

```
CREATE VIEW report_grants AS
SELECT row_number() OVER () AS number,
       t.grantee,
       t.table_name,
       t.privileges
FROM ( SELECT role_table_grants.grantee,
              role_table_grants.table_name,
              string_agg(role_table_grants.privilege_type::text, ', '::text) AS privileges
        FROM information_schema.role_table_grants
        WHERE (role_table_grants.grantee::name =
               ANY (ARRAY['admin'::name, 'moderator'::name, 'manager'::name]))
          AND role_table_grants.table_name::name <> 'refresh_tokens'::name
        GROUP BY role_table_grants.grantee, role_table_grants.table_name
        ORDER BY role_table_grants.grantee, role_table_grants.table_name) t;
```

Рисунок 36 – Представление report_grants

Результат выборки для представления

	number bigint	grantee name	table_name name	privileges text
1	28	manager	address	SELECT
2	29	manager	audit_log	INSERT
3	30	manager	batch	UPDATE, SELECT, INSERT
4	31	manager	document	SELECT, UPDATE, INSERT
5	32	manager	document_content	UPDATE, SELECT, INSERT

Рисунок 37 – Результат выборки для представления: report_grants

Представление содержит информацию о номере строки, роли, названии, привилегиях роли.

Представление report_interface_grants

Выводится информацию о доступе к интерфейсу.

```
CREATE VIEW report_interface_grants AS
SELECT table_privileges.grantee AS role,
       table_privileges.table_name AS section,
       array_agg(lower(table_privileges.privilege_type::text) ORDER BY (
         CASE lower(table_privileges.privilege_type::text)
           WHEN 'select'::text THEN 1
           WHEN 'insert'::text THEN 2
           WHEN 'update'::text THEN 3
           WHEN 'delete'::text THEN 4
           ELSE 5
         END)) AS permissions
FROM information_schema.table_privileges
WHERE (table_privileges.privilege_type::text =
       ANY (ARRAY['SELECT'::character varying::text,
                  'INSERT'::character varying::text,
                  'UPDATE'::character varying::text,
                  'DELETE'::character varying::text]))
AND (table_privileges.grantee::name =
     ANY (ARRAY['admin'::name, 'moderator'::name, 'manager'::name]))
GROUP BY table_privileges.grantee, table_privileges.table_name
ORDER BY table_privileges.grantee, table_privileges.table_name;
```

Рисунок 38 – Представление report_grants

Результат выборки для представления

	role name 	section name 	permissions text[] 
1	manager	address	{select}
2	manager	audit_log	{insert}
3	manager	batch	{select,insert,update}
4	manager	document	{select,insert,update}
5	manager	document_content	{select,insert,update}

Рисунок 39 – Результат выборки для представления: report_grants

Представление содержит информацию о роли, названии раздела и доступах к нему.

Представление report_no_products

Вывод выводит информацию о продуктах, которые нет на складе.

```
CREATE OR REPLACE VIEW report_grants AS
SELECT
    table_name,

    COALESCE(STRING_AGG(privilege_type, ', ' ORDER BY privilege_type)
        FILTER (WHERE grantee = 'admin'), '-') AS admin,

    COALESCE(STRING_AGG(privilege_type, ', ' ORDER BY privilege_type)
        FILTER (WHERE grantee = 'manager'), '-') AS manager,

    COALESCE(STRING_AGG(privilege_type, ', ' ORDER BY privilege_type)
        FILTER (WHERE grantee = 'moderator'), '-') AS moderator

FROM information_schema.table_privileges
WHERE grantee IN ('admin', 'manager', 'moderator')
    AND privilege_type IN ('SELECT', 'INSERT', 'UPDATE', 'DELETE')
GROUP BY table_name
ORDER BY table_name;
```

Рисунок 40 – Представление report_grants

Результат выборки для представления

	table_name name	admin text	manager text	moderator text
1	gender	SELECT	SELECT	SELECT
2	position	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDA...
3	producer	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDA...
4	product	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDA...
5	product_category	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDA...
6	refresh_tokens	DELETE, INSERT, SELECT, UPDATE	DELETE, INSERT	DELETE, INSERT
7	report_batches	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT
8	report_documents_by_employee	DELETE, INSERT, SELECT, UPDATE	-	SELECT
9	report_employees	DELETE, INSERT, SELECT, UPDATE	-	SELECT
10	report_expired_batches	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT

Рисунок 41 – Результат выборки для представления: report_no_products

Представление содержит информацию о номере строки, id товара, названии товара, названии производителя.

Представление report_producer_subject_statistics

Выводит информацию о продукции о количестве производителей по регионам.

```
CREATE VIEW report_producer_subject_statistics AS
SELECT row_number() OVER () AS number,
       a.subject,
       count(*) AS producer_quantity,
       string_agg(pr.name::text, ', '::text) AS producer_name
FROM producer pr
JOIN address a ON a.id = pr.id_address
GROUP BY a.subject;
```

Рисунок 42 – Представление report_producer_subject_statistics

Результат выборки для представления

	number bigint	subject character varying (100)	producer_quantity bigint	producer_name text
1	1	Республика Татарстан	2	АО "БытМашПром", ООО "МебельЛюкс"
2	2	Новгородская область	6	ИП "Михайлов", ООО "ТехноДом", АО "СтройМебел...
3	3	Красноярский край	2	ООО "МикроТех", ИП "ТехМаркет"
4	4	Приморский край	3	ООО "СтильДрев", АО "АвтоВлад", АО "АвтоПром"
5	5	Москва	1	ООО "ХолодСервис"

Рисунок 43 – Результат выборки для представления:

report_producer_subject_statistics

Представление содержит информацию о номере строки, субъекте, количеству производителей, название производителей.

Представление report_products_left

Выводится информация о количестве оставшихся товаров на складе по каждому товару.

```
CREATE OR REPLACE VIEW report_products_left AS
SELECT row_number() OVER () AS number,
       t.id_product,
       t.product_name,
       t.producer_name,
       t.left_quantity
FROM ( SELECT pt.id AS id_product,
              pt.name AS product_name,
              pr.name AS producer_name,
              COALESCE(l.product_left, 0::bigint) AS left_quantity
        FROM ( SELECT b.id_product,
                      sum(t_1.quantity) AS product_left
                  FROM ( SELECT document_content.id_batch,
                                CASE
                                  WHEN (document_content.id_document IN ( SELECT document.id
                                                                              FROM document
                                                                              WHERE document.id_document_category = 2))
                                  THEN document_content.quantity * '-1'::integer
                                  ELSE document_content.quantity
                                END AS quantity
                              FROM document_content
                              WHERE NOT (document_content.id_document IN ( SELECT document.id
                                                                              FROM document
                                                                              WHERE document.id_document_category = 3))) t_1
                                JOIN batch b ON b.id = t_1.id_batch
                              GROUP BY b.id_product
                              ORDER BY (sum(t_1.quantity)) DESC) l
                  RIGHT JOIN product pt ON pt.id = l.id_product
                  JOIN producer pr ON pr.id = pt.id_producer
              ORDER BY (COALESCE(l.product_left, 0::bigint)) DESC, pr.name) t;
```

Рисунок 44 – Представление report_products_left

Результат выборки для представления

	number bigint	id_product integer	product_name character varying (100)	producer_name character varying (100)	left_quantity bigint
1	1	1	Кухонный гарнитур "Уют"	ООО "СтильДрев"	13
2	2	3	Материнская плата "Gamer XTREME"	ООО "МикроТех"	10
3	3	2	Стиральная машина "SM-5000"	АО "БытМашПром"	3
4	4	5	Холодильник "Frost+ 300"	АО "БытМашПром"	0
5	5	47	Шкаф для одежды "Classic Wardrobe"	АО "СтройМебель"	0

Рисунок 45 – Результат выборки для представления: report_products_left

Представление содержит информацию номере строки, id товара, название товара, название производителя, количество оставшихся продуктов.

Представление report_products_left_by_batch

Выводит информацию о количестве оставшихся продуктов на складе по партиям.

```
CREATE OR REPLACE VIEW report_products_left_by_batch AS
SELECT row_number() OVER () AS number,
       t.id_batch,
       t.id_product,
       t.product_name,
       t.left_quantity
FROM ( SELECT b.id AS id_batch,
              p.id AS id_product,
              p.name AS product_name,
              COALESCE(t_1.left_quantity, 0::bigint) AS left_quantity
        FROM batch b
        JOIN product p ON b.id_product = p.id
        LEFT JOIN ( SELECT dc.id_batch,
                          sum(
                            CASE
                              WHEN d.id_document_category = 2 THEN - dc.quantity
                              WHEN d.id_document_category = 1 THEN dc.quantity
                              ELSE NULL::integer
                            END) AS left_quantity
                      FROM document_content dc
                      JOIN document d ON d.id = dc.id_document
                      GROUP BY dc.id_batch) t_1 ON t_1.id_batch = b.id
        ORDER BY (COALESCE(t_1.left_quantity, 0::bigint)) DESC) t;
```

Рисунок 46 – Представление report_products_left_by_batch

Результат выборки для представления

	number bigint	id_batch integer	id_product integer	product_name character varying (100)	left_quantity bigint
1	1	4	1	Кухонный гарнитур "Уют"	12
2	2	3	3	Материнская плата "Gamer XTREME"	10
3	3	2	2	Стиральная машина "SM-5000"	3
4	4	1	1	Кухонный гарнитур "Уют"	1
5	5	5	2	Стиральная машина "SM-5000"	0

Рисунок 47 – Результат выборки для представления:

report_products_left_by_batch

Представление содержит номер строки, id партии, id товара, название товара, количество оставшихся продуктов в партии.

Представление report_system_users

Выводит информацию о пользователях системы,

```
CREATE OR REPLACE VIEW report_system_users AS
SELECT row_number() OVER () AS number,
       t.role,
       t.surname,
       t.firstname,
       t.patronymic,
       t."position"
FROM ( SELECT r.sys_role AS role,
              e.surname,
              e.firstname,
              e.patronymic,
              ps.name AS "position"
        FROM employee e
        JOIN sys_user su ON su.id_employee = e.id
        JOIN "position" ps ON e.id_position = ps.id
        JOIN role r ON su.id_role = r.id
        ORDER BY r.sys_role) t;
```

Рисунок 48 – Представление report_system_users

Результат выборки для представление

	number bigint	role character varying (50)	surname character varying (50)	firstname character varying (50)	patronymic character varying (50)	position character varying (50)
1	1	admin	Михайлов	Роман	Александрович	Системный администратор
2	2	manager	Соколова	Анна	Сергеевна	Кладовщик
3	3	manager	test	test	test	Грузчик
4	4	moderator	Никитин	Петр	Алексеевич	Грузчик
5	5	moderator	Волков	Артем	Дмитриевич	Начальник склада

Рисунок 49 – Результат выборки из представления: report_system_users

Представление содержит номер строки, роль, фамилия, имя и отчество сотрудника, должность сотрудника.

Представление report_tables_activity

Выводит информацию об активности в таблицах за последний месяц.

```
CREATE OR REPLACE VIEW report_tables_activity AS
SELECT row_number() OVER () AS number,
       t.table_name,
       t.action,
       t.actors,
       t.action_quantity
FROM ( SELECT audit_log.table_name,
              audit_log.action,
              string_agg(DISTINCT audit_log.changed_by, ', '::text
                          ORDER BY audit_log.changed_by) AS actors,
              count(*) AS action_quantity
        FROM audit_log
       WHERE audit_log.table_name <> 'refresh_tokens'::text
       AND audit_log.changed_at >= (CURRENT_DATE - '7 days'::interval)
       GROUP BY audit_log.table_name, audit_log.action
       ORDER BY audit_log.table_name) t;
```

Рисунок 50 – Представление report_tables_activity

Результат выборки из представления

	number bigint	table_name text	action text	actors text	action_quantity bigint
1	1	address	DELETE	admin	2
2	2	address	INSERT	admin, postgres	11
3	3	address	UPDATE	admin	2
4	4	batch	INSERT	postgres	1
5	5	batch	UPDATE	postgres	1

Рисунок 51 – Результат выборки для представления: report_tables_activity

Представление содержит номер строки, название таблицы, действие, название актора, количество действий в данной таблице с данным типом действия.

Представление report_tables_activity_per_hour

Выводит информацию об активности в таблицах по часам за последний месяц.

```
CREATE OR REPLACE VIEW report_tables_activity AS
SELECT EXTRACT(hour FROM audit_log.changed_at) AS hour,
       count(*) AS action_count,
       string_agg(DISTINCT audit_log.changed_by, ', '::text
                  ORDER BY audit_log.changed_by) AS actors
FROM audit_log
WHERE audit_log.table_name <> 'refresh_tokens'::text
AND audit_log.changed_at >= (now() - '1 mon'::interval)
GROUP BY (EXTRACT(hour FROM audit_log.changed_at))
ORDER BY (EXTRACT(hour FROM audit_log.changed_at));
```

Рисунок 52 – Представление report_tables_activity_per_hour

Результат выборки для представления

	hour numeric 	action_count bigint 	actors text 
1	9	4	manager, postgres
2	10	7	admin
3	11	22	admin
4	12	16	admin, postgres
5	13	19	admin, postgres

Рисунок 53 – Результат выборки для представления:
report_tables_activity_per_hour

Представление содержит час, количество действий, актор.

3.3 Хранимые функции

Хранимая функция является группой инструкций языка pl/pgsql, которая была один раз откомпилирована, и может выполняться много раз. При создании функций использовался sql-скрипт, который представлен в листинге Б.3.

Описание хранимых функций

```
CREATE FUNCTION get_interface_grants(p_role text) RETURNS TABLE(role text, section text, permissions text[])
LANGUAGE plpgsql STABLE
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM report_interface_grants rig
        WHERE rig.role = p_role
    ) THEN
        RAISE EXCEPTION 'ERROR: no grants for this role (%)', p_role;
    END IF;

    RETURN QUERY
    SELECT
        rig.role::text,
        rig.section::text,
        rig.permissions
    FROM report_interface_grants AS rig
    WHERE rig.role = p_role
    ORDER BY rig.section;
END;
$$;
```

Рисунок 54 – Хранимая функция get_interface_grants

Результат выбора функции

	role text	section text	permissions text[]
1	manager	address	{select}
2	manager	audit_log	{insert}
3	manager	batch	{select,insert,update}
4	manager	document	{select,insert,update}
5	manager	document_content	{select,insert,update}

Рисунок 55 – Результат вызова функции get_interface_grants

```

CREATE FUNCTION get_products_left_in_batch(v_id integer) RETURNS integer
    LANGUAGE plpgsql
    AS $$
DECLARE
    v_products_left INT DEFAULT 0;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM report_products_left_by_batch WHERE id_batch = v_id) THEN
        RAISE EXCEPTION 'ERROR: no batch with such id (%)', v_id;
    END IF;

    SELECT
        left_quantity INTO v_products_left
    FROM report_products_left_by_batch WHERE
    WHERE id_batch = v_id;

    RETURN v_products_left;
END;
$$;

```

Рисунок 56 – Хранимая функция get_products_left_in_batch

Результат выбора функции

	get_products_left_in_batch	
	integer	
1		1

Рисунок 57 – Результат вызова функции get_products_left_in_batch

3.4 Триггеры, события

Триггер – это хранимая процедура, которая выполняется действиями по модификации данных: добавлением INSERT, удалением DELETE строки в заданной таблице, или изменением UPDATE данных в определённом столбце заданной таблицы реляционной базы данных.

Триггерная функция audit_trigger. Используется для логирования всех DML действий над таблицами предметной области.

```
CREATE OR REPLACE FUNCTION audit_trigger() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log(table_name, action, old_data, changed_by)
        VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD), current_user);
        RETURN OLD;

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log(table_name, action, old_data, new_data, changed_by)
        VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD), row_to_json(NEW), current_user);
        RETURN NEW;

    ELSIF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log(table_name, action, new_data, changed_by)
        VALUES (TG_TABLE_NAME, TG_OP, row_to_json(NEW), current_user);
        RETURN NEW;

    END IF;
END;
$$ LANGUAGE plpgsql;
```

Рисунок 58 – Триггерная функция audit_trigger

Триггерная функция delete_old. Используется для автоматического удаления старых логов (старше одного месяца).

```
CREATE FUNCTION delete_old() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM audit_log WHERE changed_at < CURRENT_TIMESTAMP - INTERVAL '1 month';

    RETURN NEW;
END;
$$;
```

Рисунок 59 – Триггерная функция delete_old

4 Проектирование и создание приложения

Для разработки программного обеспечения был выбран редактор кода Microsoft Visual Studio Code. Данный редактор является свободно распространяемой средой разработки и поддерживает все технологии, используемые в проекте, а также предоставляет средства подсветки синтаксиса и расширения функциональности.

В качестве языка программирования серверной части использован язык Go — современный минималистичный язык, ориентированный на разработку серверной логики. После компиляции приложение представляет собой один исполняемый файл, что упрощает его перенос и развертывание без необходимости установки дополнительных зависимостей, характерных для таких языков, как Python, C# или Java.

Для создания API был выбран фреймворк Gin для языка Go. По сравнению со стандартной библиотекой `net/http`, данный фреймворк предоставляет готовые структуры и методы, необходимые для маршрутизации запросов и обработки HTTP-сообщений, что упрощает процесс разработки.

Языком реализации графического интерфейса выбран JavaScript с использованием библиотеки React. Данный набор технологий широко применяется в современной web-разработке и позволяет создавать интерактивные пользовательские интерфейсы.

Для управления инфраструктурой была использована платформа Docker, позволяющая контейнеризировать программные компоненты и обеспечить удобное управление процессом развертывания. В качестве средств автоматизации были применены Bash-скрипты, обеспечивающие запуск, остановку, резервное копирование и перенос информационной

системы на другую вычислительную среду. Листинги скриптов в Приложении Д.

Структура разрабатываемого приложения представляет собой единый Git-репозиторий, содержащий исходный код серверной и клиентской частей, а также конфигурационные файлы инфраструктуры и автоматизации.

Директория с серверной частью приложения содержит точку входа в API-сервис (`main.go`) и директорию `internal`, в которой размещён основной исходный код. Внутри данной директории каждая логическая подсистема (сервис) вынесена в отдельную папку со следующей структурой:

- `handler.go` – обработчики HTTP-запросов;
- `model.go` – структуры данных, соответствующие моделям предметной области (для каждой таблицы базы данных);
- `repository.go` – слой доступа к данным, реализующий взаимодействие с базой данных с использованием заранее определённых SQL-запросов;
- `routes.go` – описание маршрутов API и связывание их с соответствующими обработчиками.

Директория с клиентской частью приложения структурирована следующим образом: точка входа (`main.jsx`), файл `App.jsx`, содержащий основной компонент приложения и логику маршрутизации, а также директории, соответствующие отдельным страницам пользовательского интерфейса. Каждая директория страницы имеет одинаковую структуру:

- `Page.jsx` – компонент, отвечающий за проверку прав доступа к разделу с использованием глобального контекста приложения;
- `Component.jsx` – основной компонент страницы, реализующий отображение данных в виде таблиц или карточек, пагинацию,

взаимодействие с серверной частью и работу с модальными окнами для создания, редактирования и удаления записей.

В каждой директории компонентов приложения размещён собственный Dockerfile, предназначенный для сборки соответствующего контейнерного образа. Файлы разрабатываемого проекта представлены на рисунках Г.1 и Г.2.

4.1 Описание интерфейса

В итоговой версии приложения были реализованы все задуманные окна и формы. Интерфейс содержит 17 различных страниц для соответствующих разделов, представленных на рисунке Г.3:

- AddressesTable – страница отображения таблицы address в табличном виде, содержит столбцы id, субъект, район, город, улица, дом (рисунок Г.4);
- BatchesTable – страница отображения таблицы batch в табличном виде, содержит столбцы id, товар, стоимость за 1 у.е, дата производства, срок годности, дата добавления (рисунок Г.5);
- DocumentCategoriesTable – страницы отображения таблицы document_category в табличном виде, содержит столбцы id, название, описание (рисунок Г.6);
- DocumentsCards – страница для отображения таблицы document в виде карточек, содержит информацию об id, дате создания, сотруднике, типе документа (рисунок Г.7);
- DocumentsItemPage – страница для отображения таблицы document_content в табличном виде (рисунок Г.8);
- EmployeesCards – страница для отображения таблицы employee в виде карточек, выводит информацию о ФИО, должности и номере телефона (рисунок Г.9);

- PositionsTable – страница для отображения таблицы position в табличном виде, содержит id, название, описание (рисунок Г.10);
- ProducersCards – страницы для отображения таблицы producer в виде карточек, выводит информацию о названии, ФИО контактного лица, адрес (рисунок Г.11);
- ProductCategories – страница для отображения таблицы product_category в табличном виде, поля таблицы: id, название, (рисунок Г.12);
- Products – страница для отображения таблицы products в виде карточек товаров, выводит информацию о названии товара, категории товара, производителе и картинке товара (рисунок Г.13);
- Roles – страница для отображения таблицы role в табличном виде, содержит информацию о id, названии, описании и системной роли (рисунок Г.14);
- SysUsers – страница для отображения таблицы sys_user в табличном виде, содержит колонки id, логин, роль и ФИО сотрудника (рисунок Г.15);
- AuditLogs – страница для отображения системных логов в виде таблицы со следующими колонками: id, таблица, действие, старые данные, новые данные, изменил, время изменения (рисунок Г.16);
- Home – домашняя страница с отображением товаров, находящихся на складе, информацией о количестве товаров, поставщиков, категорий товаров и сотрудников (рисунок Г.17);
- Login – страница входа в систему содержит поля логин и пароль (рисунок Г.18);
- Profile – страница профиль, отображение фамилии, имени и отчества текущего пользователя, его должности и адресе (рисунок Г.19).

На каждой странице реализовано редактирование соответствующих записей при помощи всплывающего модального окна (рисунок Г.20).

4.2 Описание разработки интерфейса

Для реализации пользовательского интерфейса было выбрано web-программирование с использованием браузера в качестве среды выполнения. В процессе разработки применяются технологии HTML, CSS и JavaScript с использованием библиотеки React.js.

С точки зрения реализации интерфейс представляет собой одностраничное приложение (Single Page Application), в котором содержимое страницы изменяется динамически без полной перезагрузки документа. Приложение использует один HTML-документ, в котором отображаемые элементы обновляются в зависимости от действий пользователя, таких как нажатие кнопок или выбор пунктов меню.

Для организации навигации между разделами приложения используется маршрутизатор BrowserRouter, обеспечивающий сопоставление адресов в строке браузера с соответствующими страницами интерфейса.

Авторизация в приложении реализована с использованием контекста AuthContext, который хранит информацию о текущем пользователе, его роли, токенах доступа и обновления, а также перечень доступных секций системы. Доступ к разделам интерфейса формируется динамически на основе прав пользователя, которые задаются путём изменения разрешений на уровне базы данных.

Навигационное меню реализовано в виде компонента Offcanvas, отображаемого в виде выдвижного модального окна слева. Состав доступных пунктов меню обновляется динамически на основании данных, получаемых из контекста авторизации.

Программная оболочка информационной системы состоит из 12 форм, каждая представляет из себя модальное окно с полями, соответствующими для конкретного объекта (рисунки Г.20 – Г.39).

4.3 Администрирование системы

Ревизованная система поддерживает следующий функционал: ведение базы данных (просмотр, добавление, изменение и удаление записей), исполнение часто встречающихся запросов в готовом виде посредством функции поиска по базе данных (ввод в поле ввода шаблона названий или наименований или выбор пунктов выпадающего списка с категориями объектов), такие запросы являются параметрическими.

Рассмотрим цикл работы с базой данных на примере таблицы product. Для ведения этой таблицы необходимо перейти при помощи иерархического меню на соответствующую страницу: “Продукты”, данных из базы данных будут отображены в виде карточек товаров.

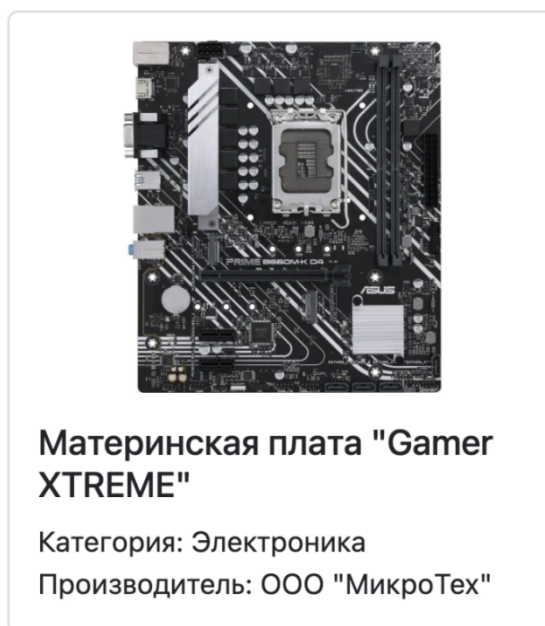
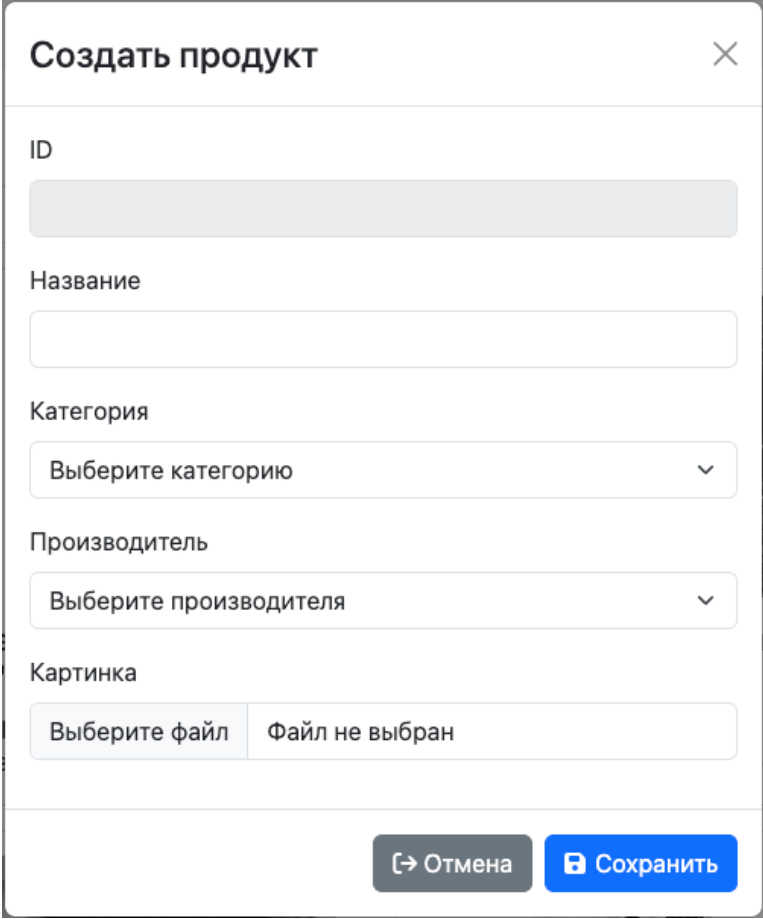


Рисунок 60 – Пример карточки товара

Для добавления новой записи в таблиц необходимо иметь соответствующие права. Для создания необходимо нажать на кнопку “Добавить продукт”, после чего откроется модальное окно с формой для создания, поля: название, категория, производитель и картинка. Все поля должны быть заполнены. Для сохранения заполненных полей необходимо нажать кнопку “Сохранить” внизу окна.



Создать продукт

ID

Название

Категория

Выберите категорию

Производитель

Выберите производителя

Картинка

Выберите файл

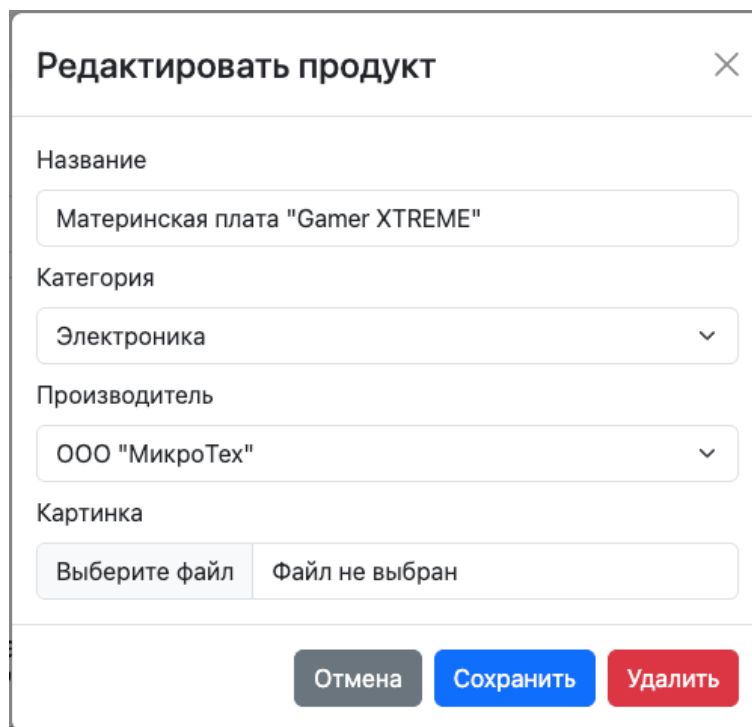
Файл не выбран

Отмена

Сохранить

Рисунок 61 – Модальное окно с формой для создания

Для редактирования и удаления созданных записей необходимо иметь соответствующие права. Чтобы изменить или удалить данные необходимо нажать на соответствующую карточку товара, после чего откроется модальное окно редактирования введённой ранее информации. Для сохранения изменённой информации необходимо нажать кнопку “Сохранить”.



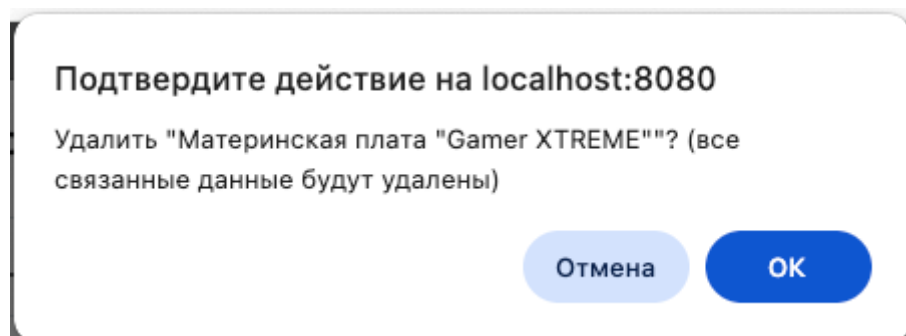
The screenshot shows a modal window titled "Редактировать продукт" (Edit product) with a close button (X) in the top right corner. The form contains the following fields:

- Название** (Name): A text input field containing "Материнская плата "Gamer XTREME"". Below the input is a light blue button labeled "Выберите файл" (Choose file).
- Категория** (Category): A dropdown menu showing "Электроника" (Electronics) with a downward arrow.
- Производитель** (Manufacturer): A dropdown menu showing "ООО "МикроТех"" (OOO "MicroTech") with a downward arrow.
- Картинка** (Image): A text input field containing "Файл не выбран" (File not selected).

At the bottom of the modal, there are three buttons: "Отмена" (Cancel) in gray, "Сохранить" (Save) in blue, and "Удалить" (Delete) in red.

Рисунок 62 – Модальное окно с формой для редактирования и удаления

Для удаления записи необходимо иметь соответствующие права. Необходимо нажать кнопку “Удалить”, после чего появится всплывающее окно со следующим сообщением “Удалить "название товара"? (все связанные данные будут удалены)”. После подтверждения действия будет удалена выбранная запись и данных, которые связаны с ней.



The screenshot shows a confirmation dialog box with the title "Подтвердите действие на localhost:8080". The main text reads: "Удалить "Материнская плата "Gamer XTREME""? (все связанные данные будут удалены)". At the bottom right, there are two buttons: "Отмена" (Cancel) in light blue and "ОК" (OK) in dark blue.

Рисунок 63 – Окно подтверждение удаления

На странице реализована возможность запрашивать данные при помощи запросов в готовом виде и параметрических: вывод продуктов с пагинацией, поиск по названию, поиск по категории.

Для выполнения запроса в готовом виде достаточно простой перейти на страницу “Продукты”, выведутся данные с пагинацией, реализовано при помощи ключевых слов LIMIT и OFFSET в запросе, далее возможен переход между “страницами”, изменения параметра сдвига.

Для выполнения параметрического запроса по поиску продукта по его названию необходимо ввести в поле ввода “Поиск по названию” любые символы, если они совпадут каким-либо образом с какими-либо товарами, то выведутся записи, удовлетворяющие заданному названию.

Для фильтрации по категории товара необходимо при помощи селектора категорий выбрать требуемую категорию, после чего будет выведены карточки, данные в которых удовлетворяют выбранной категории.

Вывод с пагинацией, поиск по названию и фильтрация по категории товара могут быть скомбинированы, получаем результат поиска по названию с учётом категории товара на соответствующей странице.

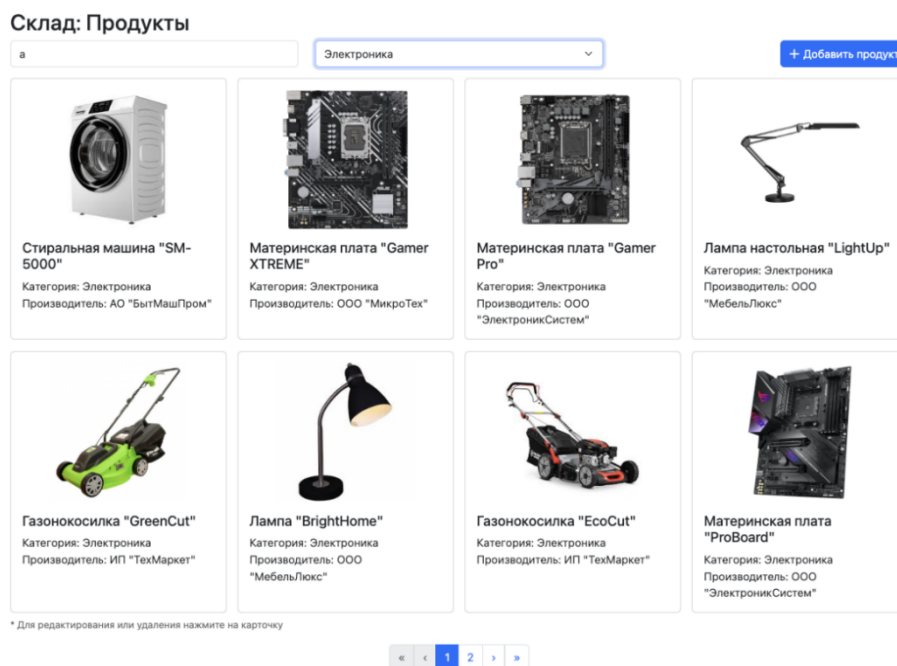


Рисунок 64 – Результат выполнения комбинированного запроса

4.4 Отчёты к базе данных

В ходе выполнения проекта была реализована возможность формирования отчёта (каждый отчёт является отдельным представлением в базе данных, поэтому данные в отчётах всегда актуальные) с последующим сохранением в файл формата PDF или печатью непосредственно на печатном устройстве. Для этого был использован встроенный инструмент браузера: `window.print()`.

Подготовленные отчёты:

- Принятые партии. Информация о принятых партиях;
- Непринятые партии. Информация о партиях, которые ещё не были задокументированны;
- Просроченные партии. Информация о просроченных партиях;
- Остатки по партиям. Остаток продукции по партиям;
- Отсутствующие товары. Товары, которых нет на складе;
- Остатки продукции. Остатки продукции на складе;
- Стоимость товаров. Стоимость товаров, находящихся на складе;
- Производители по регионам. Количество производителей по регионам;
- Сотрудники. Информация о сотрудниках склада;
- Документы по сотрудникам. Список сотрудников и количество документов по каждому сотруднику по каждой категории документов;
- Права ролей. Права ролей в системе;
- Доступные разделы. Доступные разделы и права пользователей;
- Пользователи системы. Список пользователей системы;
- Активность по таблицам. Активность за неделю;
- Активность по часам. Активность в системе по часам за последний месяц.

Примеры отчётов представлены на рисунках Г.29 – Г.43.

4.5 Тестирование работы приложения

4.5.1 Разработка тест-кейсов

Для качественного тестирования разработанного проекта необходимо тестировать как клиентскую часть, интерфейс, так и серверную часть, в качестве метода выбрано ручное тестирование с оформлением в виде таблицы тест-кейса.

В обоих проектах происходит валидация данных: перед отправкой через API, на стороне JS и после получения на сервере, на стороне Go; она обязательно должна быть протестирована, чтобы не происходило нарушения целостности в базе данных.

Так как приложение представляет из себя многопользовательскую информационную систему, то так же необходимо провести проверку корректности авторизации на стороне клиента при динамическом изменении границ доступа для ролей.

Исходя из того, что авторизация возможна только при корректной работе сервиса аутентификации, то и он точно так же должен быть подвергнут всестороннему тестированию.

План тестирования:

- сервис аутентификации: получение токенов доступа и обновления, обновление токена доступа, выход из системы;
- авторизация: вход в систему, используя роли различные права доступа для проверки работы системы авторизации, динамическое изменение границ доступа;
- валидация: ввод некорректных и корректных данных, просмотр логов в консоли разработчика в браузере и журнала сервера.

4.5.2 Тестирование

Тест-кейсы и полученные результаты тестирования в Приложении В.

4.6 Состав и содержание документации

4.6.1 Справочное руководство

Назначение программы:

Назначением приложения является использования информационной системы для автоматизации работы склада: документооборот, создание автоматических отчётов.

Системные и технические требования. Для правильной работы приложения необходимо иметь соответствующую конфигурацию технических средств и установленное на компьютере определённое программное обеспечение. Системные и аппаратные включают:

- операционную систему семейства Unix (Linux, macOS) или операционную систему Windows 10 и выше с установленной подсистемой Windows Subsystem for Linux (WSL 2)

- монитор Full HD 1920x1080;
- объём оперативной памяти 8 ГБ и более;
- файлы данных программы ИС для склада занимают дисковое пространство в зависимости от необходимости в ресурсах локальной задачи.

Учебный вариант данных содержит около 4,9 МБ;

- манипулятор «мышь» или другие устройства;
- ресурсы операционной системы;
- установленный Docker.

Условия выполнения:

- запуск программы осуществляется путём запуска скрипта `start.sh`, запускается `docker-compose`, по адресу `localhost:8080/` доступен интерфейс приложения, по адресу `localhost:5050/` доступна web-версия `pgAdmin4` для администрирования базы данных;

- для завершения работы предусмотрен скрипт для остановки функционирования информационной системы, запуск скрипта `stop.sh` автоматически останавливает сервер БД, интерфейс и API-сервер;

- для перехода с домашней страницы на любые другие предусмотрено всплывающее слева иерархическое меню, переход на страницу осуществляется нажатием на название соответствующего раздела;

- для вывода результата на экран используется режим таблицы, для сохранения полученных отчётов необходимо нажать кнопку “Сохранить” и во всплывшем меню печати выбрать пункт “Сохранить” или “Печать”.

В ходе работы с информационной системой вероятны ситуации появления на экране некоторых сообщений, отображающих состояние системы, например при ошибке. Возможные сообщения:

- “user with such username not found” – уведомление при авторизации, обозначает, что введённый логин или пароль некорректен, повторить попытку ввода корректных данных;

- “permission denies for table” – уведомление при выборке данных, обозначает, что доступ текущей роли к данной таблице запрещён;

- “create error” – уведомление при добавлении записи, обозначает, что при попытке создания записи в БД произошла ошибка, повторить ввод корректных данных для создания;

- “select error” – уведомление при выборке данных, обозначает, что при попытке выборки данных из БД произошла ошибка, повторить ввод корректных параметров для запроса;

- “update error” – уведомление при обновлении записи, обозначает, что при попытке обновления данных в БД произошла ошибка, повторить ввод корректных данных для обновления;

- “delete error” – уведомление при удалении данных, обозначает, что при попытке удаления данных из БД произошла ошибка, повторить ввод корректного параметра для удаления;

- “invalid data” – уведомление при создании или обновлении, обозначает, что введённые данные не удовлетворяют установленным в БД ограничениям, нарушение целостности данных;

4.6.2 Руководство пользователя

4.6.2.1 Администратор

Администратор информационной системы имеет полный доступ ко всем объектам базы данных.

Для запуска информационной системы необходимо запустить скрипт `start.sh`, в процессе запуска возможно скачивание необходимых зависимостей, после успешного запуска выводится таблица запущенных docker-образов.

В скрипте запуска предусмотрено начало легирования стандартного вывода образа из `docker-compose`, все логи сохраняются в директории `logs` в корневом каталоге проекта, каждый сервис логируется в отдельный файл.

Для остановки информационно системы необходимо запустить скрипт `stop.sh`, который останавливает `docker-compose`.

Предусмотрен скрипт для резервного копирования базы данных для этого необходимо предварительно запустить информационную систему, после чего запустить скрипт `make_dump.sh` с параметром в виде пути к файлу для сохранения sql-скрипта (файл создается автоматически). Для использования созданного дампа необходимо удалить `docker-volume` при помощи команды: `docker volume rm warehouse_db_data`, выполнение этой команды необходимо в директории, из которой изначально запускалась информационная система. После этого запустить скрипт `debug.sh` с параметром в виде пути к созданному ранее дампу базы данных.

Так же предусмотрено полное резервное копирование информационной системы, для этого необходимо предварительно запустить информационную систему, после чего запустить скрипт `backup.sh`, выполниться автоматическое создание архива с названием: `warehouse_(дата создания резервной копии)`. Для восстановления системы из созданной копии необходимо разархивировать полученный ранее архив и запустить скрипт `restore.sh`, который автоматически восстановит и запустит систему.

Для изменение пароля от базы данных (поль `postgres`) необходимо в файле `docker-compose.yml` изменить переменную окружения `POSTGRES_PASSWORD`

для сервиса `warehouse_db` на желаемый пароль.

Администрирование базы данных осуществляется посредством `pgAdmin4`, который запускается как отдельный web-сервис, доступный по адресу `localhost:5050/`, почту и пароль можно менять, изменяя переменные окружения `PGADMIN_DEFAULT_EMAIL` и `PGADMIN_DEFAULT_PASSWORD` для сервиса `pgadmin` на желаемые данные для входа от имени администратора.

Администратор имеет полный доступ к элементам интерфейса, а также дополнительные разделы: пользователи, роли и логи. Все разделы доступны из меню.

4.6.2.2 Менеджер

Менеджер имеет ограниченный доступ к объектам базы данных, границей его ответственности является предметная область: партии, документы, продукты, производители и категории товаров. Доступ к этим разделам ограничивается только отсутствием права на удаление. Так же менеджер имеет доступ на просмотр данных в справочниках.

Полный цикл работы менеджера полностью отражается в типовой задаче: приёмка партии нового товара новой категории от нового производителя, находящегося по известному адресу, с последующей переоценкой и списанием части товара.

Ход работы:

1. Авторизация в системе. Предварительно открыть домашнюю страницу (рисунок Г.17), после чего перейти на страницу входа (рисунок Г.18). Ввести данные для входа в соответствующие поля и нажать кнопку “Войти”. После чего произойдёт переход обратно на домашнюю страницу.

2. Добавление нового производителя по известному адресу. Открыть меню (рисунок Г.3), перейти на страницу “Производители” (рисунок Г.11), нажать кнопку “Добавить производителя”, после чего откроется модальное окно с формой создания производителя (рисунок Г.26). Ввести корректные

данные (адрес выбрать из выпадающего списка), после чего нажать кнопку “Сохранить”, после успешного сохранения форма автоматически закроется.

3. Добавление категории продуктов. Открыть меню, перейти на страницу “Категории продуктов” (рисунок Г.12), нажать кнопку “Добавить категорию”, ввести в форму создания категории (рисунок Г.22) корректные данные, нажать кнопку “Сохранить”, модальное окно автоматически закроется.

4. Добавление продукта. Открыть меню, перейти на страницу “Продукты” (рисунок Г.13), нажать кнопку “Добавить продукт”, ввести название товара, выбрать заранее созданных производителя и категорию товара, выбрать картинку для отображения в карточке товара (рисунок Г.28), нажать кнопку “Сохранить”, модальное окно автоматически закроется.

5. Добавление партии. Открыть меню, перейти на страницу “Партии” (рисунок Г.5), нажать на кнопку “Добавить партию”, ввести данные о партии, выбрать товар из выпадающего списка (рисунок Г.28), нажать кнопку “Сохранить”, модальное окно автоматически закроется.

6. Проверка добавленной партии (работа с отчётами). Открыть меню, перейти на страницу “Все отчёты” (рисунок Г.44), перейти на страницу отчёта “Непринятые партии”, убедиться, что в отчёте указано только что добавленная партия (рисунок Г.30).

7. Создание документа о приёмке партии товара. Открыть меню, перейти на страницу “Документы” (рисунок Г.7), нажать кнопку “Добавить документ”, откроется страница содержания документа (рисунок Г.8), ввести сегодняшнюю дату и выбрать из выпадающего списка “Поступление”, нажать кнопку “Сохранить”, произойдёт создание документа и появятся поля для добавления партий.

8. Добавление партии нового товара в документ. Выбрать из выпадающего списка созданную партию и ввести количество единиц принимаемого товара, нажать кнопку “Добавить строку”.

9. Фиксация успешной приёмки партии товара (работы с отчётами). Открыть отчёт “Принятые партии” (рисунок Г.29), убедиться, что партия есть в списке, нажать кнопку “Сохранить”, откроется окно сохранения текущего отчёта (Г.45), нажать кнопку “Сохранить” и выбрать путь для скачивания, файл с отчётом будет успешно сохранён.

10. Переоценка товара. Открыть меню, перейти на страницу “Документы”, нажать кнопку “Добавить документ”, выбрать из выпадающего списка “Переоценка”, нажать “Сохранить”, выбрать партию для переоценки из выпадающего списка, ввести в поле “Новая цена” новую стоимость за 1 у.е. товара, нажать “Добавить строку”.

11. Проверка успешности переоценки. Открыть отчёт “Суммарная стоимость товаров”, убедиться, что новая стоимость на 1 у.е. зафиксирована.

12. Списание товара. Открыть меню, перейти на страницу “Документы”, создать документ о списании, выбрать партию для списания, ввести корректное количество единиц товара для списания (свериться с количеством в выпадающем списке), нажать кнопку “Добавить строку”.

13. Фиксация изменения товара из партии, открыть отчёт “Остатки по партиям”, удостовериться в том, что количество товаров, оставшихся на складе, изменилось.

14. Просмотр информации о своём пользователе. Нажать на своё ФИО в шапке страницы, нажать кнопку “Профиль”, откроется страницы “Профиль” (рисунок Г.19), просмотреть информацию о пользователе. Выйти на домашнюю страницу, нажав на кнопку “Склад” в шапке страницы.

15. Выход из системы. Нажать на своё ФИО в шапке страницы, нажать кнопку “Выйти”, выполнится выход из системы, откроется домашняя страница.

5. Методы защиты базы данных

Возможные угрозы безопасности:

- угрозы конфиденциальности (неправомерный доступ к информации), заключается в том, что информация становится доступной тем, для кого она не предназначена;
- угрозы авторизации (неправомерный доступ к объекту базы данных), заключается в том, что различные объекты имеют различные уровни доступности для разных ролей в системе;
- угрозы целостности (неправомерное изменение данных), заключается в том, что некоторые изменения могут нарушить логическую структуру данных в базе данных.

Методы защиты баз данных, используемые в текущем проекте:

- защита паролем;
- разграничение прав доступа к объектам базы данных;
- поддержание целостности данных в СУБД;
- аутентификация.

5.1 Защита паролем

Данный вид защиты является одним из основных методов защиты любой информационной системы от несанкционированного доступа к ней. Для создания ситуации, когда подбор кодового слова почти невозможен, в данной системе используется метод создания пароля следующим образом: пароль генерируется в терминале при использовании нескольких методов повышения энтропии кодовой строки (конвейер из стандартных модулей операционной системы).

Хеш-функция — функция, осуществляющая преобразование массива входных данных произвольной длины в выходную битовую строку установленной длины, выполняемое определённым алгоритмом.

Для служебных паролей используется следующий алгоритм: `date +%s | sha256sum | base64 | head -c 32; echo`. В результате строки используются в качестве переменных окружения `docker`-образа сервера для подключения к БД через соответствующую роли, и как пароль для роли в СУБД.

Этот метод генерации паролей используется исключительно для генерации служебных паролей ролей базы данных и не применяется для хранения пользовательских паролей. Для пользовательской аутентификации применяется отдельный механизм, исключающий хранение паролей в открытом виде.

Для безопасного хранения пользовательский пароль используется `GenerateFromPassword` из модуля `bcrypt`, с автоматическим добавлением соли, параметр `cost` равен 10. Логика хеширования и проверки пароля реализована на сервере.

5.2 Управление доступом

Управление доступом реализуется посредством нативного механизма привилегий в СУБД. Для назначения привилегий используются ключевые слова `GRANT` (выдача прав на конкретный объект) и `REVOKE` (отзыв прав на конкретный объект).

В системе реализовано три уровня доступа к данным, совпадающих с ролями в БД, и владелец системы:

- администратор, имеет неограниченный доступ ко всей схеме базы данных, может как предоставлять, так и отзывать права на объекты, имеет эксклюзивный доступ к логам, пользователям и ролям;
- менеджер, имеет ограниченный доступ к объектам базы данных, строго ограничен предметной областью: складом. Может добавлять и изменять данные, но не может удалять;

– модератор, выполняет роль редактора данных, как и менеджер имеет доступ только в границах предметной области. Может только просматривать и изменять данные;

– владелец, имеет полный доступ к базе данных и информационной системе в целом, может менять пароли ролей, управлять ролями, добавлять, редактировать и удалять объекты базы данных, регулировать работу приложения, изменять вид и поведение элементов интерфейса.

Привилегии ролей указаны в таблице 15.

Таблица 15 – Привилегии ролей базы данных

Название таблицы	Администратор	Менеджер	Модератор
address	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE
audit_log	DELETE, INSERT, SELECT, UPDATE	INSERT	INSERT
batch	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
document	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
document_category	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE
document_content	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
employee	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE
gender	SELECT	SELECT	SELECT
position	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE
producer	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
product	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
product_category	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
role	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT
sys_user	DELETE, INSERT, SELECT, UPDATE	NULL	SELECT

5.3 Поддержание целостности данных

Ключевым моментом при работе с реляционными базами данных является контроль связей между таблицами. При любом некорректном вводе значений целостность данных может быть нарушена, автоматический учёт партий, документов и товаров станет невозможным.

Для предотвращения подобных ситуаций в системе предусмотрена валидация данных перед внесением из в БД, такая валидация есть и на стороне интерфейса, и на стороне сервера, для этого используются служебные представления-отчёты, на основе которые, например, контролируется количество списываемых товаров.

На уровне самой базы данных поддержание целостности основано на ограничениях, добавляемых как во время создания объекта, при помощи ключевого слова CHECK, так и уже во время существования объекта, при помощи ключевого слова CONSTRAINT, являющегося частью команды на изменение структуры объекта `ALTER TABLE table_name ADD CONSTRAINT`.

Таким образом, контроль целостности данных реализуется на трёх уровнях: интерфейса, серверной логики и СУБД.

5.4 Аутентификация

Для подтверждения того, что клиент, который обращается к серверу, является правомерным пользователем, используется система аутентификации на основе JWT (Json Web Token) – ключа аутентификации пользователя, используется при обращении к защищённым методам API.

Токен представляет из себя строку, состоящую из трёх частей: заголовок (используемый алгоритм криптографической подписи и тип

токена), полезная нагрузка (имя пользователя, роль и срок годности) и криптографическая подпись.

Каждый токен имеет срок годности: один час, после истечения часа необходима повторная авторизация, которая может быть выполнена в двух форматах: повторный ввод логина и пароли или использования токена обновления, который автоматически возвращает новый токен доступа, и жизненный цикл повторяется.

Заключение

В результате курсового проектирования были изучены основы проектирования баз данных, работа со специализированными программами, улучшены знания в области работы базой данных. Были также, улучшены навыки программирования. Проектирование прошло успешно, все функции реализованы, и задача выполнена.

На этапе формирования требований к системе были сформулированы цели и задачи курсового проекта.

В данной работе были решены все поставленные во введении задачи, начиная со знакомства с предметной областью и заканчивая созданием автоматизированной системы для обработки информации – учёт товаров на складе.

В ходе работы на этапе проектирования была разработана реляционная модель базы данных средствами программы DBDesigner. Также создана структура базы данных средствами языка программирования реляционных базы данных SQL в программе PostgreSQL. Результатом выполнения запросов для создания таблиц базы данных на языке SQL была создана схема данных базы данных Склад.

На этапе реализации был осуществлён ввод данных в базу данных и организован поиск необходимой информации по критерию и отображение результирующего списка данных. Также были разработаны результирующие документы – отчёты, web-страницы на основании созданной базы данных.

Для дальнейшей работы с приложением требуется пройти ещё три этапа создания автоматизированной информационной системы – это этап тестирования, ввода в действия и этап эксплуатации и сопровождения системы.

В приложении к пояснительной записке курсового проекта помещены разработанные диаграммы: диаграмма IDEF0 (рисунок А.3), декомпозиция диаграммы IDEF0 (рисунок А.4), диаграмма IDEF3 (рисунок А.5), ER-

диаграмма (рисунок А.7), схема реляционной модели базы данных (рисунок А.7), схема данных в СУБД (рисунок А.9) и диаграмма Ганта (рисунок А.6).

Таким образом, можно сделать вывод, что задание на курсовое проектирование выполнено в полном объёме. Исходный код разработанной системы доступен по ссылке: <https://github.com/ccrayp/warehouse>.

Список литературы

1. ГОСТ 2.701—2008 Единая система конструкторской документации. Схемы. Виды и типы. Общие требования к выполнению. – М.: Стандартиформ, 2009. – 15 с.
2. ГОСТ 2.743—91 Единая система конструкторской документации. Обозначения условные графические в схемах. Элементы цифровой техники. – М.: Стандартиформ, 2009. – 62 с.
3. React. Быстрый старт./ Стефанов С., СПб.: Питер, 2023 г.
4. RESTful Web API. Паттерны и практики./ Амундсен М., Астана: Спринт Бук, 2025 г.
5. Базы данных. Учебный курс./Глушаков С.В., Ломотько Д.В., Харьков-2000.
6. Базы данных: Разработка и управление./ Гэри Хансен, М-1999 г.
7. Базы данных: Учебник для высших учебных заведений/ Под ред. проф. А.Д. Хомоненко СПб.: Корона принт, 2000.
8. Практикум по пакетам прикладных программ/ под редакцией С.В. Назарова, М-1999 г.
9. Проектирование информационных систем: курс лекций. Учебное пособие./В.И. Грекул, Г.Н. Денищенко, Н.Л. Коровкина. М.: Интернет-Ун-т Информ. технологий, 2005г.
10. Язык программирования Go./ Донован Алан А. А., Керниган Брайян У., М.: ООО “И.Д. Вильямс”, 2016 г.

Приложение А

(обязательное)

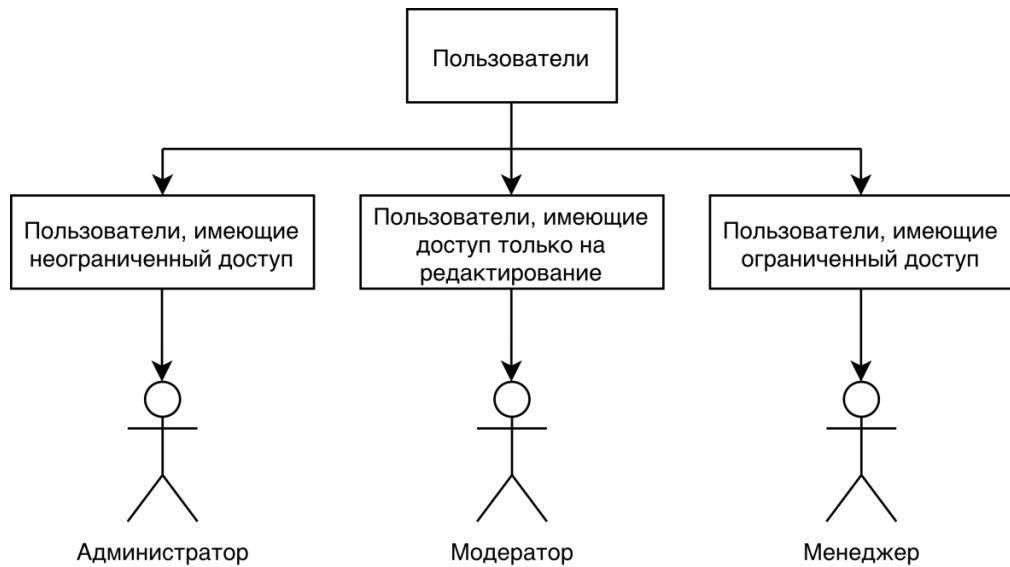


Рисунок А.1 – Группы пользователей системы

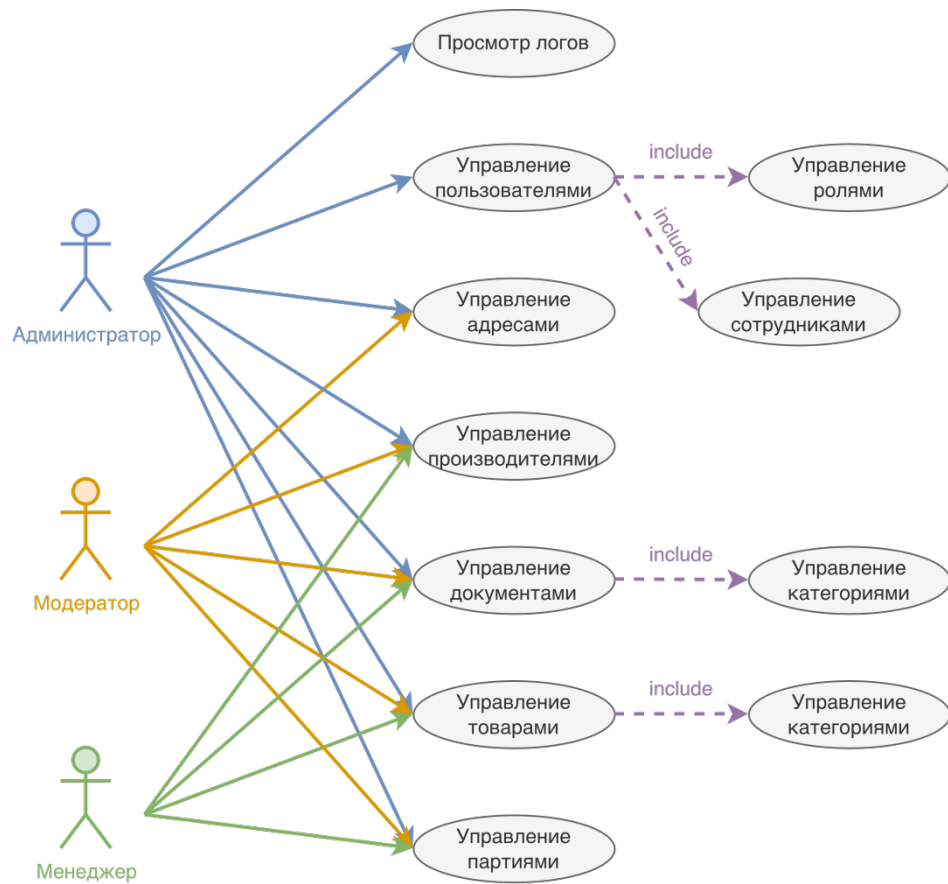


Рисунок А.2 – Диаграмма прецедентов

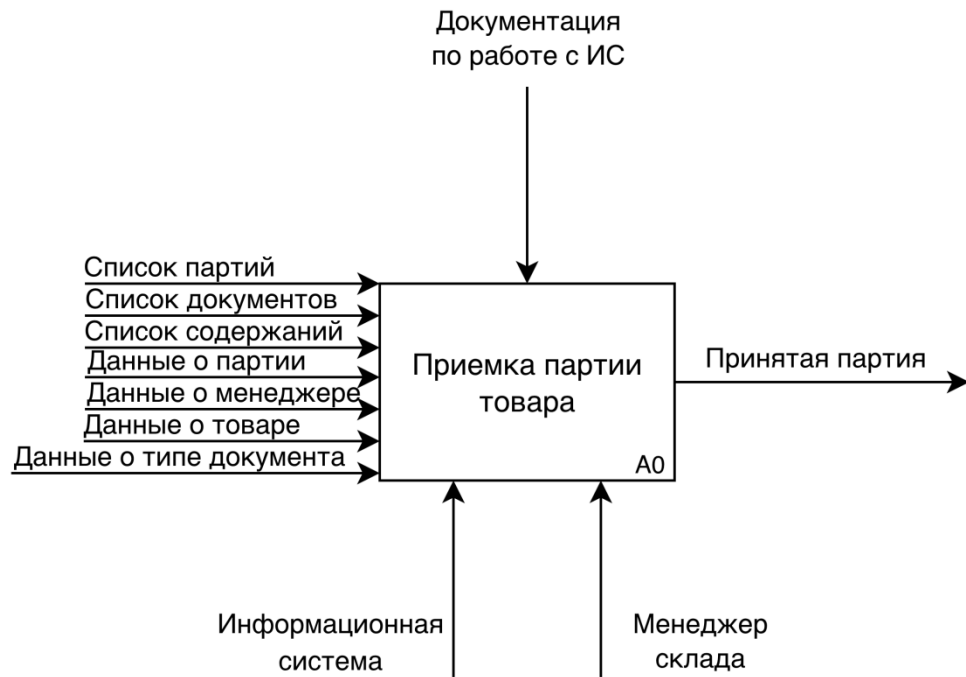


Рисунок А.3 – IDEF0 основного процесса

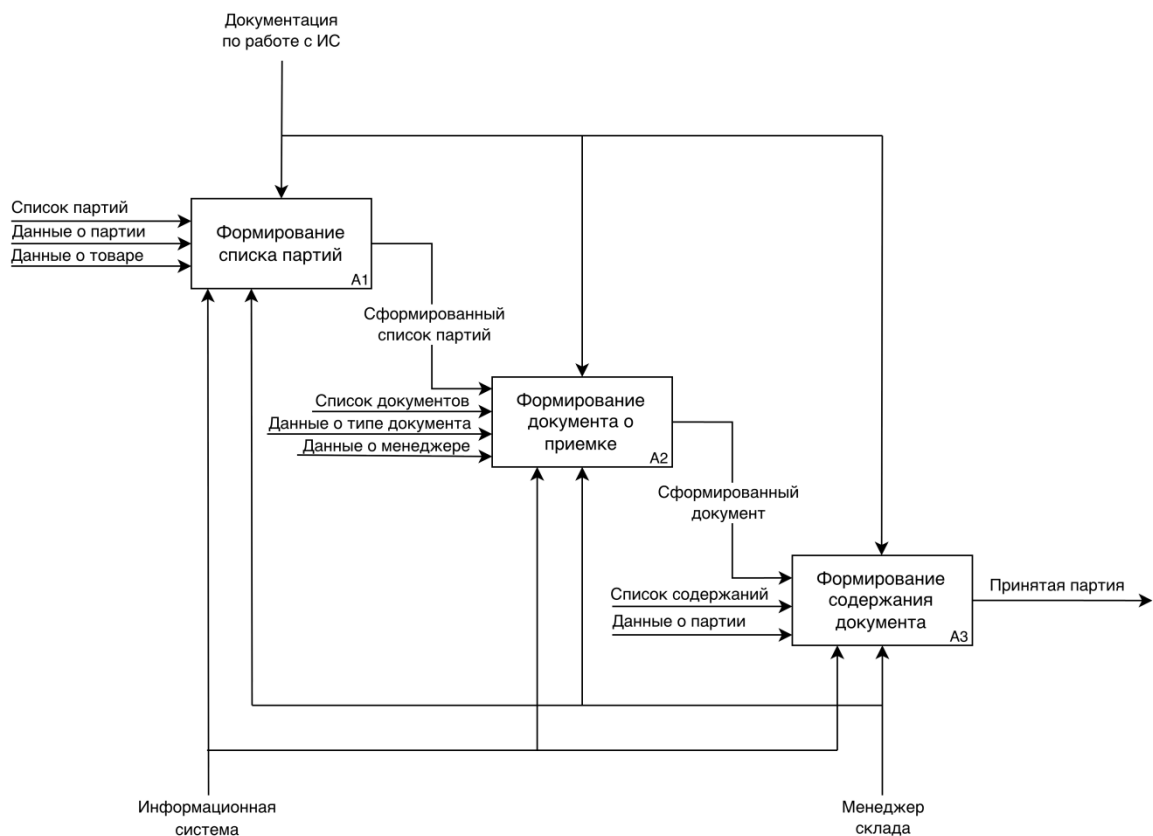


Рисунок А.4 – Декомпозиция IDEF0 основного процесса

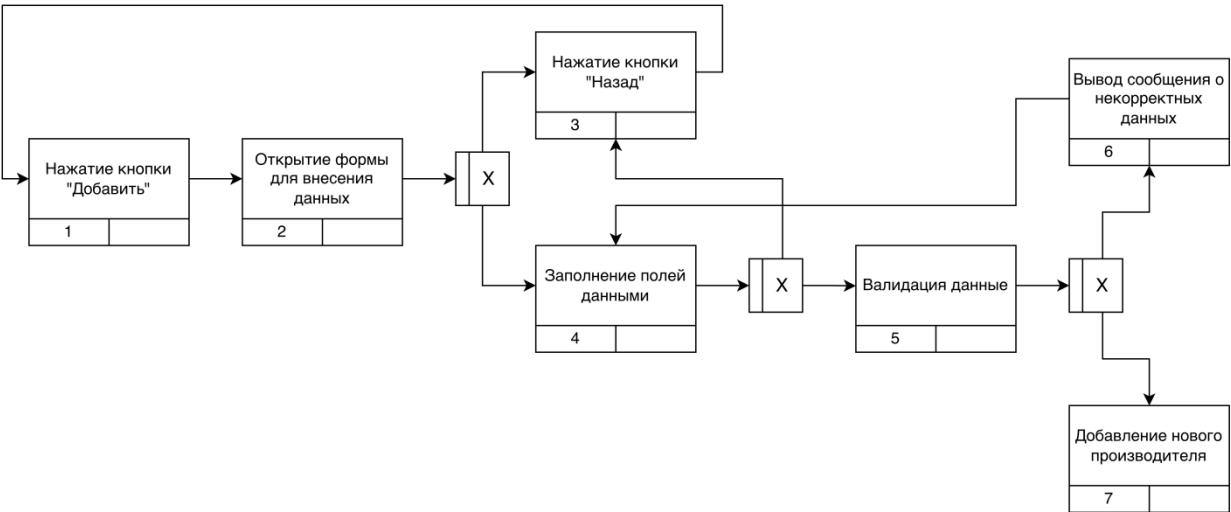


Рисунок А.5 – IDEF3 процесса добавления производителя

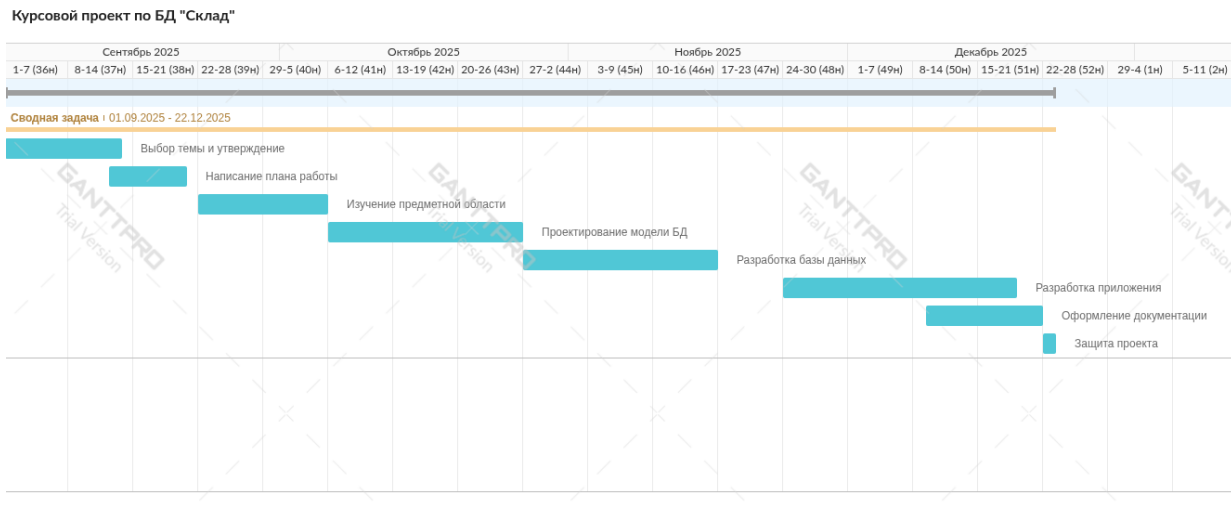


Рисунок А.6 – Диаграмма Ганта

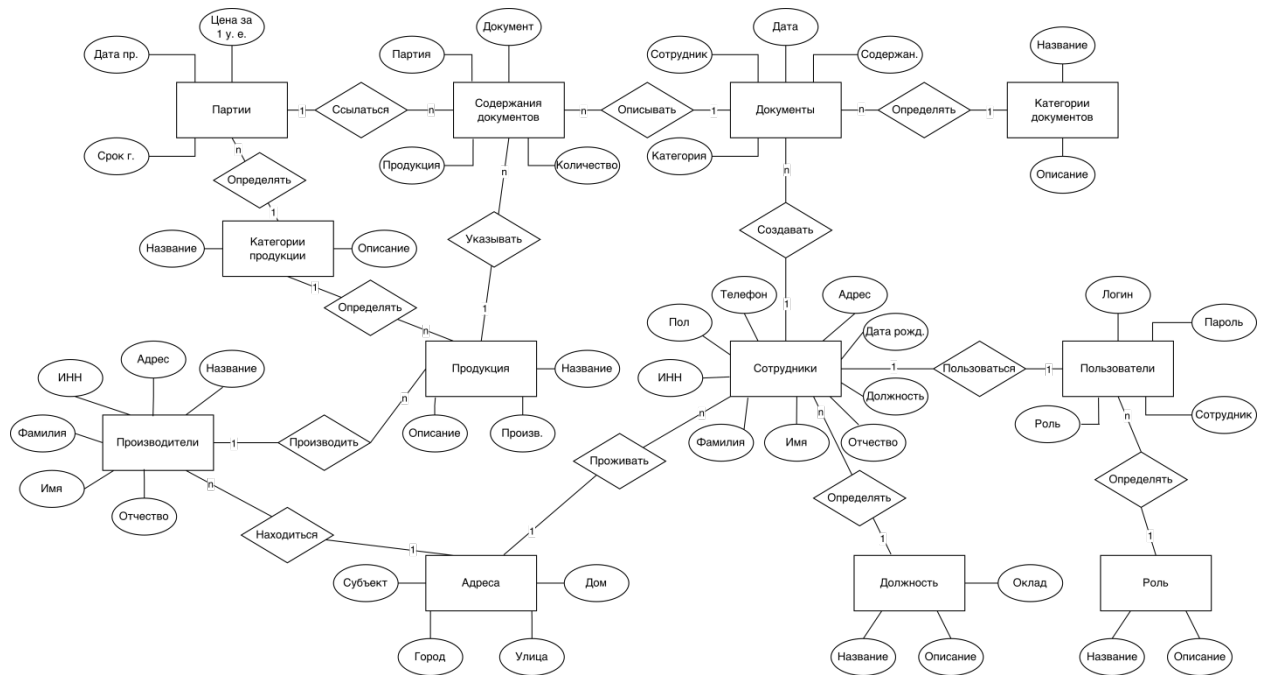


Рисунок А.7 – ER-модель

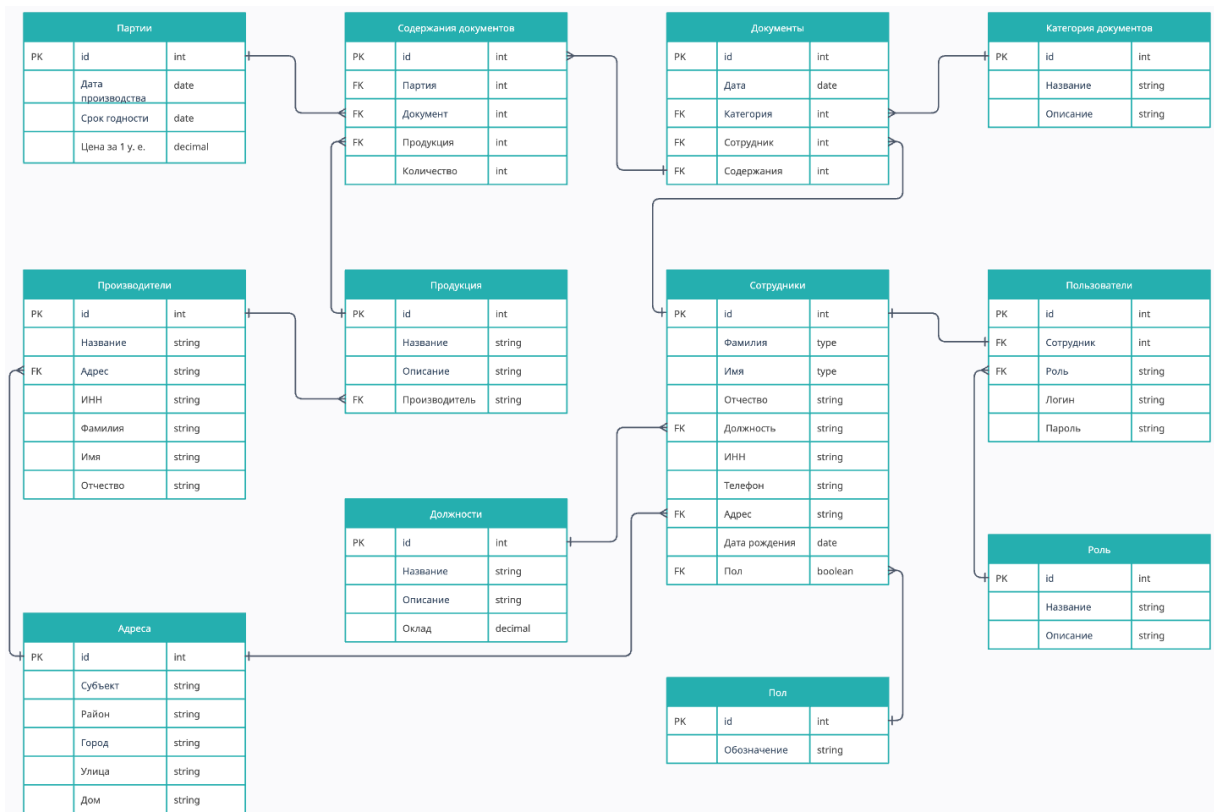


Рисунок А.8 – Схема реляционной модели базы данных

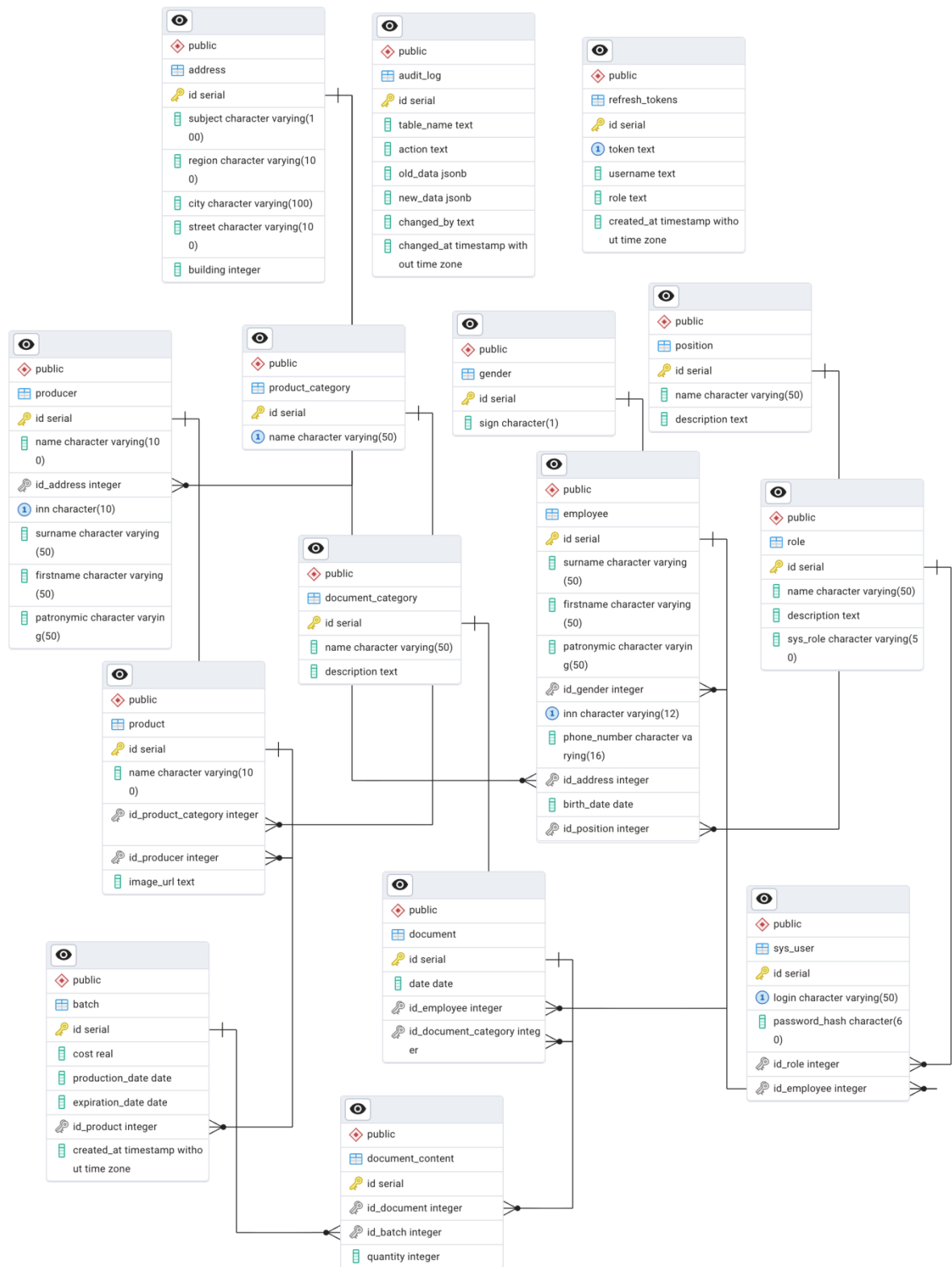


Рисунок А.9 – Схема данных в СУБД

Приложение Б

(обязательное)

Б.1 – Скрипт SQL для создания таблиц базы данных

```
CREATE TABLE public.address (  
  id integer NOT NULL,  
  subject character varying(100) NOT NULL,  
  region character varying(100) NOT NULL,  
  city character varying(100) NOT NULL,  
  street character varying(100) NOT NULL,  
  building integer NOT NULL  
);
```

```
CREATE SEQUENCE public.address_id_seq  
AS integer  
START WITH 1  
INCREMENT BY 1  
NO MINVALUE  
NO MAXVALUE  
CACHE 1;
```

```
ALTER SEQUENCE public.address_id_seq OWNED BY public.address.id;
```

```
CREATE TABLE public.audit_log (  
  id integer NOT NULL,  
  table_name text NOT NULL,  
  action text NOT NULL,  
  old_data jsonb,  
  new_data jsonb,  
  changed_by text NOT NULL,  
  changed_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL  
);
```

```
CREATE SEQUENCE public.audit_log_id_seq  
AS integer  
START WITH 1  
INCREMENT BY 1  
NO MINVALUE  
NO MAXVALUE  
CACHE 1;
```

```
ALTER SEQUENCE public.audit_log_id_seq OWNED BY public.audit_log.id;
```

```
CREATE TABLE public.batch (  
  id integer NOT NULL,  
  cost real NOT NULL,  
  production_date date NOT NULL,
```

```

expiration_date date,
id_product integer NOT NULL,
created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL
);

```

```

CREATE SEQUENCE public.batch_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;

```

```

ALTER SEQUENCE public.batch_id_seq OWNED BY public.batch.id;

```

```

CREATE TABLE public.product (
id integer NOT NULL,
name character varying(100) NOT NULL,
id_product_category integer NOT NULL,
id_producer integer NOT NULL,
image_url text DEFAULT '/static/products/placeholder.png'::text NOT NULL
);

```

```

CREATE TABLE public.document (
id integer NOT NULL,
date date NOT NULL,
id_employee integer NOT NULL,
id_document_category integer NOT NULL
);

```

```

CREATE TABLE public.document_category (
id integer NOT NULL,
name character varying(50) NOT NULL,
description text NOT NULL
);

```

```

CREATE SEQUENCE public.document_category_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;

```

```

ALTER SEQUENCE public.document_category_id_seq OWNED BY
public.document_category.id;

```

```

CREATE TABLE public.document_content (
id integer NOT NULL,
id_document integer NOT NULL,

```

```
id_batch integer NOT NULL,
quantity integer NOT NULL
);
```

```
CREATE SEQUENCE public.document_content_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.document_content_id_seq OWNED BY
public.document_content.id;
```

```
CREATE SEQUENCE public.document_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.document_id_seq OWNED BY public.document.id;
```

```
CREATE TABLE public.employee (
id integer NOT NULL,
surname character varying(50) NOT NULL,
firstname character varying(50) NOT NULL,
patronymic character varying(50),
id_gender integer NOT NULL,
inn character varying(12) NOT NULL,
phone_number character varying(16) NOT NULL,
id_address integer NOT NULL,
birth_date date NOT NULL,
id_position integer NOT NULL
);
```

```
CREATE SEQUENCE public.employee_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.employee_id_seq OWNED BY public.employee.id;
```

```
CREATE TABLE public.gender (
id integer NOT NULL,
```

```
sign character(1) NOT NULL
);
```

```
CREATE SEQUENCE public.gender_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.gender_id_seq OWNED BY public.gender.id;
```

```
CREATE TABLE public."position" (
id integer NOT NULL,
name character varying(50) NOT NULL,
description text NOT NULL
);
```

```
CREATE SEQUENCE public.position_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.position_id_seq OWNED BY public."position".id;
```

```
CREATE TABLE public.producer (
id integer NOT NULL,
name character varying(100) NOT NULL,
id_address integer NOT NULL,
inn character(10) NOT NULL,
surname character varying(50) NOT NULL,
firstname character varying(50) NOT NULL,
patronymic character varying(50)
);
```

```
CREATE SEQUENCE public.producer_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.producer_id_seq OWNED BY public.producer.id;
```

```
CREATE TABLE public.product_category (
```



```

id integer NOT NULL,
name character varying(50) NOT NULL
);

```

```

CREATE SEQUENCE public.product_category_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;

```

```

ALTER SEQUENCE public.product_category_id_seq OWNED BY
public.product_category.id;

```

```

CREATE SEQUENCE public.product_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;

```

```

ALTER SEQUENCE public.product_id_seq OWNED BY public.product.id;

```

```

CREATE TABLE public.refresh_tokens (
id integer NOT NULL,
token text NOT NULL,
username text NOT NULL,
role text NOT NULL,
created_at timestamp without time zone DEFAULT now() NOT NULL
);

```

```

CREATE SEQUENCE public.refresh_tokens_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;

```

```

ALTER SEQUENCE public.refresh_tokens_id_seq OWNED BY public.refresh_tokens.id;

```

```

CREATE TABLE public.role (
id integer NOT NULL,
name character varying(50) NOT NULL,
description text NOT NULL,
sys_role character varying(50) NOT NULL
);

```

```
CREATE TABLE public.sys_user (
  id integer NOT NULL,
  login character varying(50) NOT NULL,
  password_hash character(60) NOT NULL,
  id_role integer NOT NULL,
  id_employee integer NOT NULL
);
```

```
CREATE SEQUENCE public.role_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.role_id_seq OWNED BY public.role.id;
```

```
CREATE SEQUENCE public.sys_user_id_seq
AS integer
START WITH 1
INCREMENT BY 1
NO MINVALUE
NO MAXVALUE
CACHE 1;
```

```
ALTER SEQUENCE public.sys_user_id_seq OWNED BY public.sys_user.id;
```

```
ALTER TABLE ONLY public.address ALTER COLUMN id SET DEFAULT
nextval('public.address_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.audit_log ALTER COLUMN id SET DEFAULT
nextval('public.audit_log_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.batch ALTER COLUMN id SET DEFAULT
nextval('public.batch_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.document ALTER COLUMN id SET DEFAULT
nextval('public.document_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.document_category ALTER COLUMN id SET DEFAULT
nextval('public.document_category_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.document_content ALTER COLUMN id SET DEFAULT
nextval('public.document_content_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.employee ALTER COLUMN id SET DEFAULT
nextval('public.employee_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.gender ALTER COLUMN id SET DEFAULT
nextval('public.gender_id_seq'::regclass);
```

```
ALTER TABLE ONLY public."position" ALTER COLUMN id SET DEFAULT  
nextval('public.position_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.producer ALTER COLUMN id SET DEFAULT  
nextval('public.producer_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.product ALTER COLUMN id SET DEFAULT  
nextval('public.product_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.product_category ALTER COLUMN id SET DEFAULT  
nextval('public.product_category_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.refresh_tokens ALTER COLUMN id SET DEFAULT  
nextval('public.refresh_tokens_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.role ALTER COLUMN id SET DEFAULT  
nextval('public.role_id_seq'::regclass);
```

```
ALTER TABLE ONLY public.sys_user ALTER COLUMN id SET DEFAULT  
nextval('public.sys_user_id_seq'::regclass);
```

Б.2 – Скрипты SQL для создание представлений базы данных

```

CREATE VIEW public.report_batches AS
SELECT b.id, p.name,
b.cost,
b.production_date,
b.expiration_date
FROM (public.batch b
JOIN public.product p ON ((b.id_product = p.id)));

CREATE VIEW public.report_documents_by_employee AS
SELECT dense_rank() OVER (ORDER BY e.surname, e.firstname, e.patronymic) AS
employee_number,
((((e.surname)::text || ' '::text) || (e.firstname)::text) || ' '::text) ||
(e.patronymic)::text) AS employee,
p.name AS "position",
dc.name AS document_category,
t.count AS documents
FROM (((public.employee e
JOIN ( SELECT e_1.id AS employee_id,
d.id_document_category,
count(*) AS count
FROM (public.document d
JOIN public.employee e_1 ON ((d.id_employee = e_1.id)))
GROUP BY e_1.id, d.id_document_category) t ON ((e.id = t.employee_id)))
JOIN public."position" p ON ((p.id = e.id_position)))
JOIN public.document_category dc ON ((t.id_document_category = dc.id)))
ORDER BY (((((e.surname)::text || ' '::text) || (e.firstname)::text) || '
':text) || (e.patronymic)::text);

CREATE VIEW public.report_employees AS
SELECT row_number() OVER () AS number,
t.surname,
t.firstname,
t.patronymic,
t.name AS "position",
t.phone_number,
t.birth_date
FROM ( SELECT e.surname,
e.firstname,
e.patronymic,
e.phone_number,
e.birth_date,
p.name
FROM (public.employee e
JOIN public."position" p ON ((e.id_position = p.id)))
ORDER BY e.surname, e.firstname, e.patronymic) t;

CREATE VIEW public.report_expired_batches AS
SELECT row_number() OVER () AS number,

```

```

b.id AS batch_id,
p.name AS product_name,
b.expiration_date,
COALESCE(sum(
CASE
WHEN (d.id_document_category = 1) THEN dc.quantity
WHEN (d.id_document_category = 2) THEN (- dc.quantity)
ELSE NULL::integer
END), (0)::bigint) AS remaining_quantity
FROM ((public.batch b
JOIN public.product p ON ((b.id_product = p.id)))
LEFT JOIN public.document_content dc ON ((dc.id_batch = b.id)))
LEFT JOIN public.document d ON ((d.id = dc.id_document)))
WHERE (b.expiration_date <= CURRENT_DATE)
GROUP BY b.id, b.expiration_date, p.name
HAVING (COALESCE(sum(
CASE
WHEN (d.id_document_category = 1) THEN dc.quantity
WHEN (d.id_document_category = 2) THEN (- dc.quantity)
ELSE NULL::integer
END), (0)::bigint) > 0)
ORDER BY b.expiration_date;

CREATE VIEW public.report_grants AS
SELECT row_number() OVER () AS number,
t.grantee,
t.table_name,
t.privileges
FROM ( SELECT role_table_grants.grantee,
role_table_grants.table_name,
string_agg((role_table_grants.privilege_type)::text, ', '::text) AS
privileges
FROM information_schema.role_table_grants
WHERE (((role_table_grants.grantee)::name = ANY (ARRAY['admin'::name,
'moderator'::name, 'manager'::name])) AND
((role_table_grants.table_name)::name <> 'refresh_tokens'::name))
GROUP BY role_table_grants.grantee, role_table_grants.table_name
ORDER BY role_table_grants.grantee, role_table_grants.table_name) t;

CREATE VIEW public.report_interface_grants AS
SELECT table_privileges.grantee AS role,
table_privileges.table_name AS section,
array_agg(lower((table_privileges.privilege_type)::text) ORDER BY
CASE lower((table_privileges.privilege_type)::text)
WHEN 'select'::text THEN 1
WHEN 'insert'::text THEN 2
WHEN 'update'::text THEN 3
WHEN 'delete'::text THEN 4
ELSE 5
END) AS permissions

```

```

FROM information_schema.table_privileges
WHERE (((table_privileges.privilege_type)::text = ANY
 (ARRAY[('SELECT'::character varying)::text, ('INSERT'::character
 varying)::text, ('UPDATE'::character varying)::text, ('DELETE'::character
 varying)::text])) AND ((table_privileges.grantee)::name = ANY
 (ARRAY['admin'::name, 'moderator'::name, 'manager'::name])))
GROUP BY table_privileges.grantee, table_privileges.table_name
ORDER BY table_privileges.grantee, table_privileges.table_name;

```

```

CREATE VIEW public.report_no_products AS
SELECT row_number() OVER () AS number,
t.id,
t.product_name,
t.producer_name
FROM ( SELECT pt.id,
pt.name AS product_name,
pr.name AS producer_name
FROM ((( SELECT product.id
FROM public.product
EXCEPT
SELECT b.id_product AS id
FROM (( SELECT document_content.id_batch,
CASE
WHEN (document_content.id_document IN ( SELECT document.id
FROM public.document
WHERE (document.id_document_category = 2))) THEN (document_content.quantity *
'-1'::integer)
ELSE document_content.quantity
END AS quantity
FROM public.document_content
WHERE (NOT (document_content.id_document IN ( SELECT document.id
FROM public.document
WHERE (document.id_document_category = 3))))) t_1
JOIN public.batch b ON ((b.id = t_1.id_batch)))
GROUP BY b.id_product
HAVING (sum(t_1.quantity) > 0)) l
JOIN public.product pt ON ((pt.id = l.id))
JOIN public.producer pr ON ((pr.id = pt.id_producer)))
ORDER BY pr.name, pt.name) t;

```

```

CREATE VIEW public.report_producer_subject_statistics AS
SELECT row_number() OVER () AS number,
a.subject,
count(*) AS producer_quantity,
string_agg((pr.name)::text, ', '::text) AS producer_name
FROM (public.producer pr
JOIN public.address a ON ((a.id = pr.id_address)))
GROUP BY a.subject;

```

```

CREATE VIEW public.report_products_left AS
SELECT row_number() OVER () AS number,

```

```

t.id_product,
t.product_name,
t.producer_name,
t.left_quantity
FROM ( SELECT pt.id AS id_product,
pt.name AS product_name,
pr.name AS producer_name,
COALESCE(l.product_left, (0)::bigint) AS left_quantity
FROM ((( SELECT b.id_product,
sum(t_1.quantity) AS product_left
FROM (( SELECT document_content.id_batch,
CASE
WHEN (document_content.id_document IN ( SELECT document.id
FROM public.document
WHERE (document.id_document_category = 2))) THEN (document_content.quantity *
'-1'::integer)
ELSE document_content.quantity
END AS quantity
FROM public.document_content
WHERE (NOT (document_content.id_document IN ( SELECT document.id
FROM public.document
WHERE (document.id_document_category = 3)))))) t_1
JOIN public.batch b ON ((b.id = t_1.id_batch)))
GROUP BY b.id_product
ORDER BY (sum(t_1.quantity)) DESC) l
RIGHT JOIN public.product pt ON ((pt.id = l.id_product)))
JOIN public.producer pr ON ((pr.id = pt.id_producer)))
ORDER BY COALESCE(l.product_left, (0)::bigint) DESC, pr.name) t;

CREATE VIEW public.report_products_left_by_batch AS
SELECT row_number() OVER () AS number,
t.id_batch,
t.id_product,
t.product_name,
t.left_quantity
FROM ( SELECT b.id AS id_batch,
p.id AS id_product,
p.name AS product_name,
COALESCE(t_1.left_quantity, (0)::bigint) AS left_quantity
FROM ((public.batch b
JOIN public.product p ON ((b.id_product = p.id)))
LEFT JOIN ( SELECT dc.id_batch,
sum(
CASE
WHEN (d.id_document_category = 2) THEN (- dc.quantity)
WHEN (d.id_document_category = 1) THEN dc.quantity
ELSE NULL::integer
END) AS left_quantity
FROM (public.document_content dc
JOIN public.document d ON ((d.id = dc.id_document)))

```

```
GROUP BY dc.id_batch) t_1 ON ((t_1.id_batch = b.id)))
ORDER BY COALESCE(t_1.left_quantity, (0)::bigint) DESC) t;
```

```
CREATE VIEW public.report_system_users AS
SELECT row_number() OVER () AS number,
t.role,
t.surname,
t.firstname,
t.patronymic,
t."position"
FROM ( SELECT r.sys_role AS role,
e.surname,
e.firstname,
e.patronymic,
ps.name AS "position"
FROM ((public.employee e
JOIN public.sys_user su ON ((su.id_employee = e.id)))
JOIN public."position" ps ON ((e.id_position = ps.id)))
JOIN public.role r ON ((su.id_role = r.id)))
ORDER BY r.sys_role) t;
```

```
CREATE VIEW public.report_tables_activity AS
SELECT row_number() OVER () AS number,
t.table_name,
t.action,
t.actors,
t.action_quantity
FROM ( SELECT audit_log.table_name,
audit_log.action,
string_agg(DISTINCT audit_log.changed_by, ', '::text ORDER BY
audit_log.changed_by) AS actors,
count(*) AS action_quantity
FROM public.audit_log
WHERE ((audit_log.table_name <> 'refresh_tokens'::text) AND
(audit_log.changed_at >= (CURRENT_DATE - '7 days'::interval)))
GROUP BY audit_log.table_name, audit_log.action
ORDER BY audit_log.table_name) t;
```

```
CREATE VIEW public.report_tables_activity_per_hour AS
SELECT EXTRACT(hour FROM audit_log.changed_at) AS hour,
count(*) AS action_count,
string_agg(DISTINCT audit_log.changed_by, ', '::text ORDER BY
audit_log.changed_by) AS actors
FROM public.audit_log
WHERE ((audit_log.table_name <> 'refresh_tokens'::text) AND
(audit_log.changed_at >= (now() - '1 mon'::interval)))
GROUP BY (EXTRACT(hour FROM audit_log.changed_at))
ORDER BY (EXTRACT(hour FROM audit_log.changed_at));
```

Б.3 – Скрипты SQL для создания хранимого кода базы данных

```
CREATE FUNCTION public.audit_trigger() RETURNS trigger
```



```

LANGUAGE plpgsql
AS $$
BEGIN
IF TG_OP = 'DELETE' THEN
INSERT INTO audit_log(table_name, action, old_data, changed_by)
VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD), current_user);
RETURN OLD;

ELSIF TG_OP = 'UPDATE' THEN
INSERT INTO audit_log(table_name, action, old_data, new_data, changed_by)
VALUES (TG_TABLE_NAME, TG_OP, row_to_json(OLD), row_to_json(NEW),
current_user);
RETURN NEW;

ELSIF TG_OP = 'INSERT' THEN
INSERT INTO audit_log(table_name, action, new_data, changed_by)
VALUES (TG_TABLE_NAME, TG_OP, row_to_json(NEW), current_user);
RETURN NEW;

END IF;
END;
$$;

CREATE FUNCTION public.delete_old() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
DELETE FROM audit_log WHERE changed_at < CURRENT_TIMESTAMP - INTERVAL '1
month';

RETURN NEW;
END;
$$;

CREATE FUNCTION public.get_interface_grants(p_role text) RETURNS TABLE(role
text, section text, permissions text[])
LANGUAGE plpgsql STABLE
AS $$
BEGIN
IF NOT EXISTS (
SELECT 1
FROM report_interface_grants rig
WHERE rig.role = p_role
) THEN
RAISE EXCEPTION 'ERROR: no grants for this role (%)', p_role;
END IF;

RETURN QUERY
SELECT
rig.role::text,

```

```

rig.section::text,
rig.permissions
FROM report_interface_grants AS rig
WHERE rig.role = p_role
ORDER BY rig.section;
END;
$$;

```

```

CREATE FUNCTION public.get_products_left_in_batch(v_id integer) RETURNS
integer
LANGUAGE plpgsql
AS $$
DECLARE
v_products_left INT DEFAULT 0;
BEGIN
IF NOT EXISTS (SELECT 1 FROM report_products_left_by_batch WHERE id_batch =
v_id) THEN
RAISE EXCEPTION 'ERROR: no batch with such id (%)', v_id;
END IF;
SELECT
left_quantity INTO v_products_left
FROM report_products_left_by_batch WHWRE
WHERE id_batch = v_id;

RETURN v_products_left;
END;
$$;

```

Приложение В

(обязательное)

Таблица В.1 – Тест-кейс, проверка аутентификации с валидными данными

ID тест-кейса/Приоритет	TU01	1
Идея: Проверка корректности работы сервиса аутентификации с валидными данными		
Входные параметры:		
– логин: artem_volkov; – пароль: volkov; – адрес для авторизации: localhost:8080/api/auth/login.		
История редактирования		
Создан 18.12.2025	Новый тест-кейс	
Предусловие		
Открыто ПО для тестирования API: Postmant		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1) Ввести в поле ввода адреса адрес для авиризации 2) Выбрать метод запроса “POST” 3) Ввести в поле ввода Body логин и пароль 4) Отправить запрос	Аутентификация прошла успешно код ответа – 200, оба токена: доступа и обновления, получены	Pass

Таблица В.2 – Тест-кейс, проверка аутентификации с невалидными данными

ID тест-кейса/Приоритет	TU02	2
Идея: Проверка корректности работы сервиса аутентификации с невалидными данными		
Входные параметры:		
– логин: artem_volkov; – пароль: volkow; – адрес для авторизации: localhost:8080/api/auth/login.		
История редактирования		
Создан 18.12.2025	Новый тест-кейс	
Предусловие		
Открыто ПО для тестирования API: Postmant		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1) Ввести в поле ввода адреса адрес для авторизации	Произошла ошибка код ответа – 401, сообщение “wrong password”	Pass
2) Выбрать метод запроса “POST”		
3) Ввести в поле ввода Body логин и пароль		
4) Отправить запрос		

Таблица В.3 – Тест-кейс, проверка корректности обновления токена доступа

ID тест-кейса/Приоритет	TU03	3
Идея: Проверить корректность получения нового токена доступа по токenu валидному обновления		
Входные параметры:		
– валидный токен обновления		
– адрес для обновления: localhost:8080/api/auth/refresh		
История редактирования		
Создан 18.12.2025	Новый тест-кейс	
Предусловие		
Открыто ПО для тестировани API: Postmant		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1) Ввести в поле ввода адреса адрес для обновления	Обновление прошло успешно, код ответа – 200, получены новые токены: доступа и обновления	Pass
2) Выбрать метод запроса “POST”		
3) Ввести в поле ввода Body валидный токен обновления		
4) Отправить запрос		

Таблица В.4 – Тест-кейс, вход в систему из-под разных ролей

ID тест-кейса/Приоритет	TU04	4
Идея: Проверить работу авторизации		
Входные параметры:		
– логин и пароль для пользователя с ролью “администратор”; – логин и пароль для пользователя с ролью “менеджер”; – адрес интерфейса приложения: localhost:8080/		
История редактирования		
Создан 18.12.2025	Новый тест-кейс	
Предусловие		
Открыт браузер, поддерживающий JavaScript		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1) Перейти по адресу интерфейса приложения 2) Перейти на страницу входа и авторизоваться с данными администратора 3) Открыть меню и зафиксировать доступные разделы 4) Выйти из системы 5) Перейти на страницу входа и авторизоваться с данными менеджера 6) Открыть меню и зафиксировать доступные раздела 7) Выйти из системы 8) Сравнить полученные результаты	Меню для разных ролей отличается	Pass

Таблица В.5 – Тест-кейс, проверка добавление товара с некорректными данными

ID тест-кейса/Приоритет	TU05	5
Идея: Проверить работу валидации данных на сервере		
Входные параметры:		
– данные товара с названием товара из более чем 100 символов.		
История редактирования		
Создан 18.12.2025	Новый тест-кейс	
Предусловие		
Совершён вход в систему от имени менеджера и открыта форма добавления товара, открыта консоль разработчика в браузере		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1) Ввести некорректные данные в форму 2) Нажать кнопку “Сохранить” 3) Проверить консоль разработчика 4) Просмотреть ответ сервера	Произошла ошибка, код ответа – 400, сообщение “name too long (N) out of 100”	Pass

Таблица В.6 – Тест-кейс, проверка добавления товара с корректными данными

ID тест-кейса/Приоритет	TU06	6
Идея: Удостовериться в корректности создания записей в БД		
Входные параметры:		
История редактирования		
Создан 18.12.2025	Новый тест-кейс	
Предусловие		
Совершён вход в систему от имени менеджера и открыта форма добавления товара, открыта консоль разработчика в браузере		
Выполняемая часть		
Список команд	Ожидаемый результат	Результат теста
1) Ввести корректные данные в форму 2) Нажать кнопку “Сохранить” 3) Форма автоматически закрылась 4) Проверить консоль разработчика 5) Просмотреть ответ сервера	Запись успешно добавлена, код ответа – 201, сообщение “product created”	Pass

Приложение Г

(обязательное)

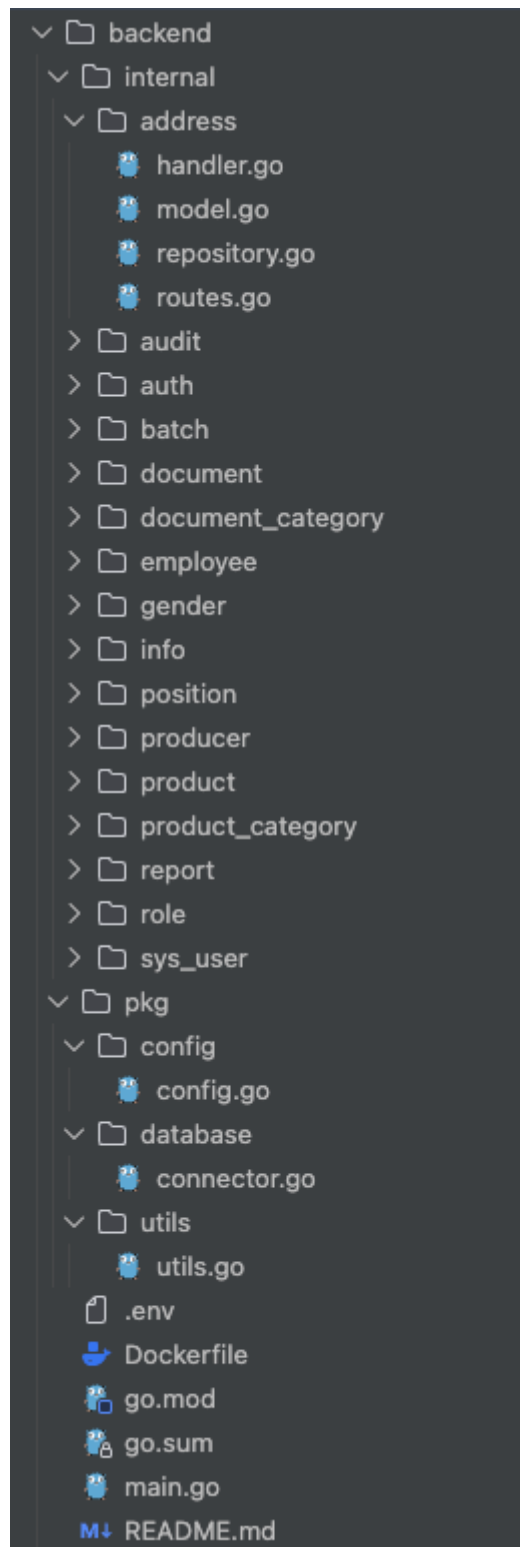


Рисунок Г.1 – Структура проекта серверной части

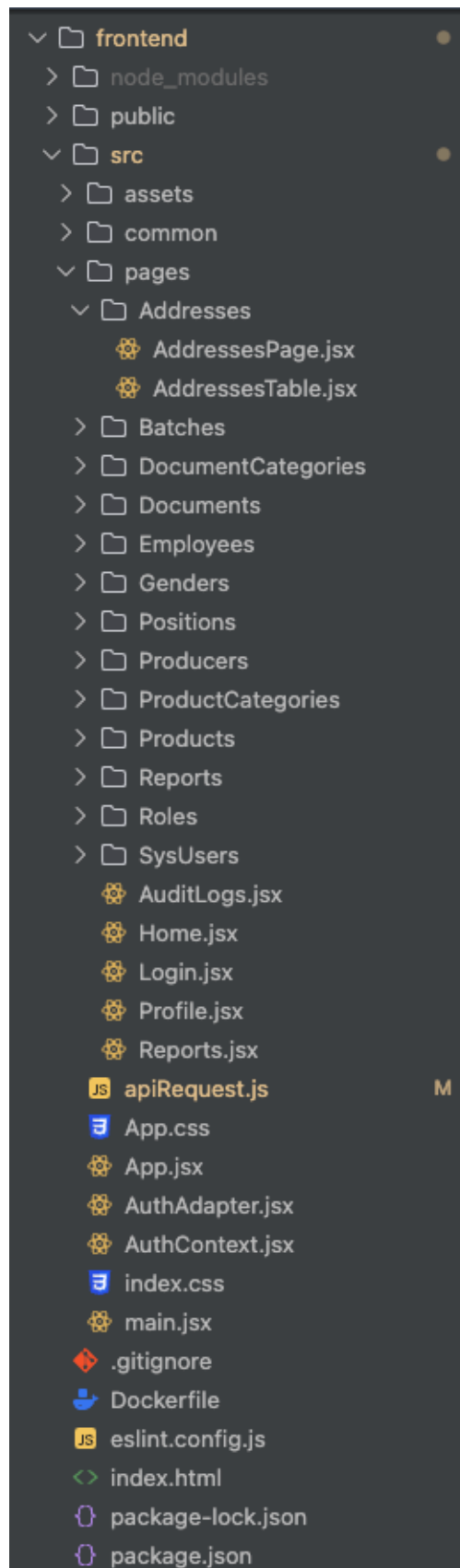


Рисунок Г.2 – Структура проекта клиентской части

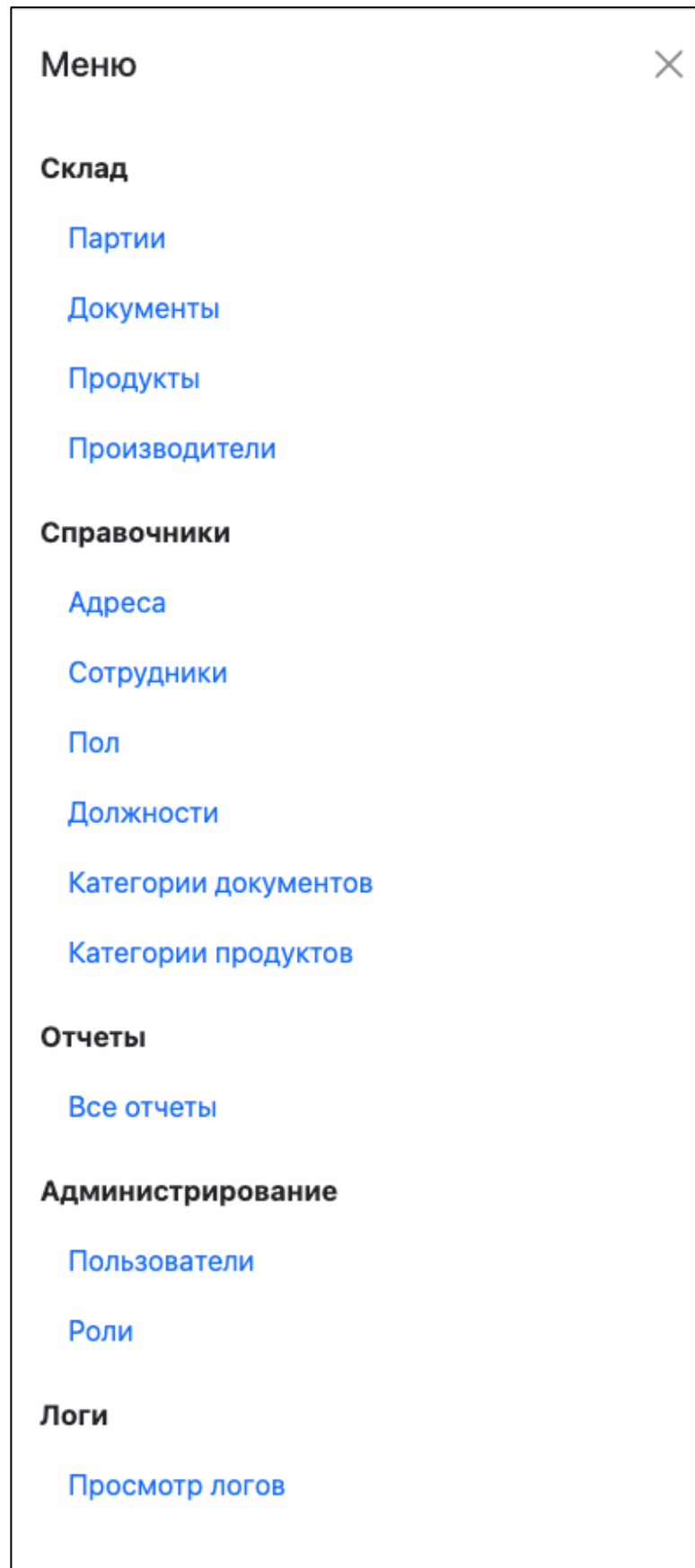


Рисунок Г.3 – Меню

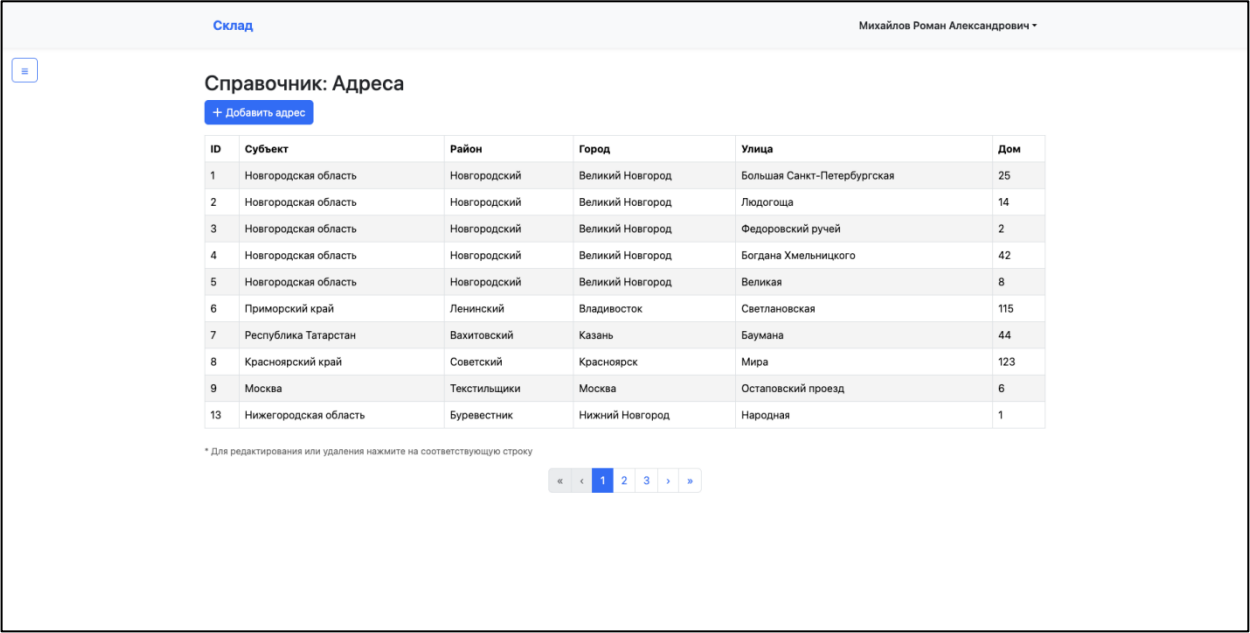


Рисунок Г.4 – Страница Адреса

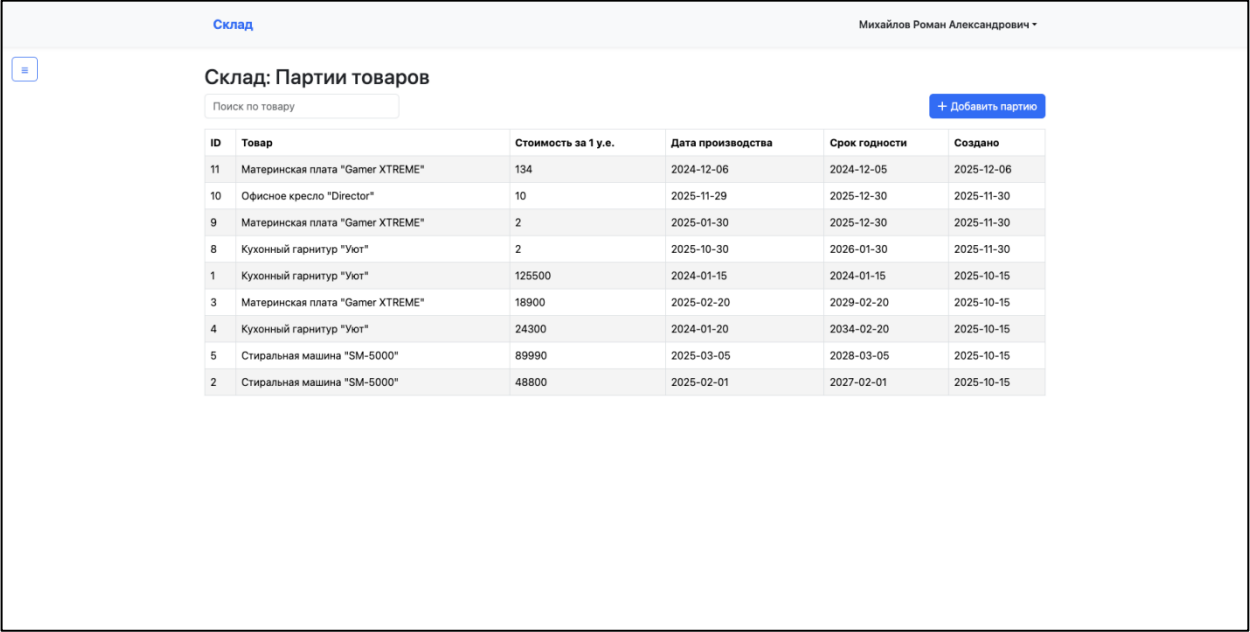


Рисунок Г.5 – Страница Партии

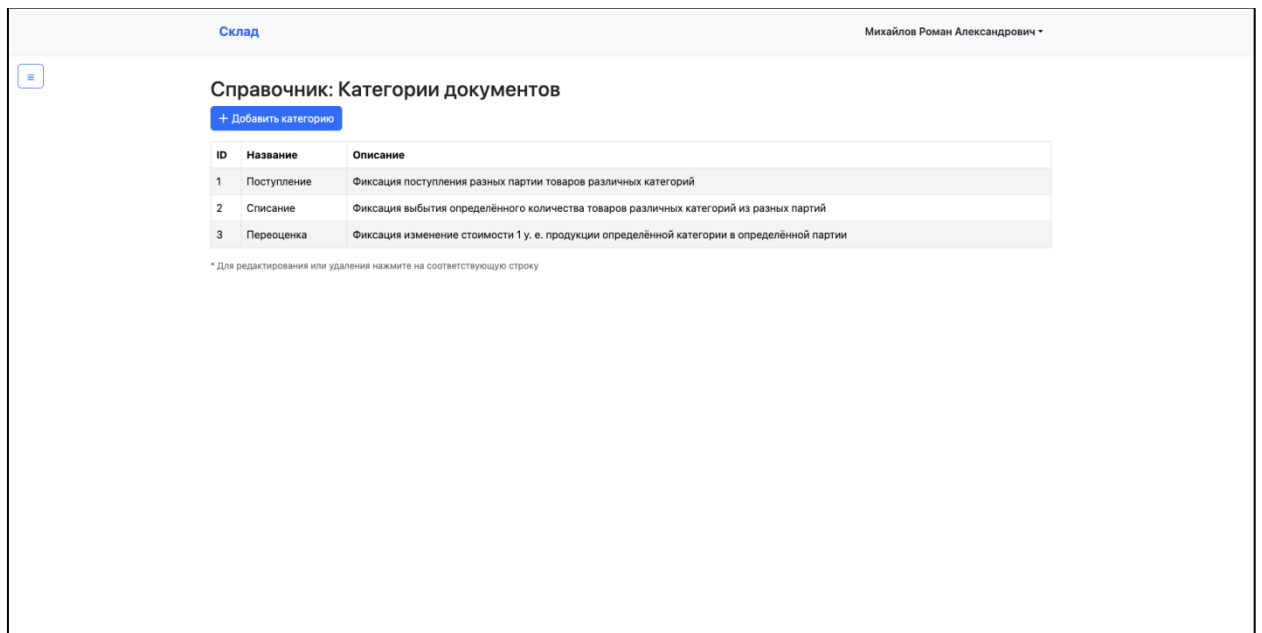


Рисунок Г.6 – Страница Категории документов

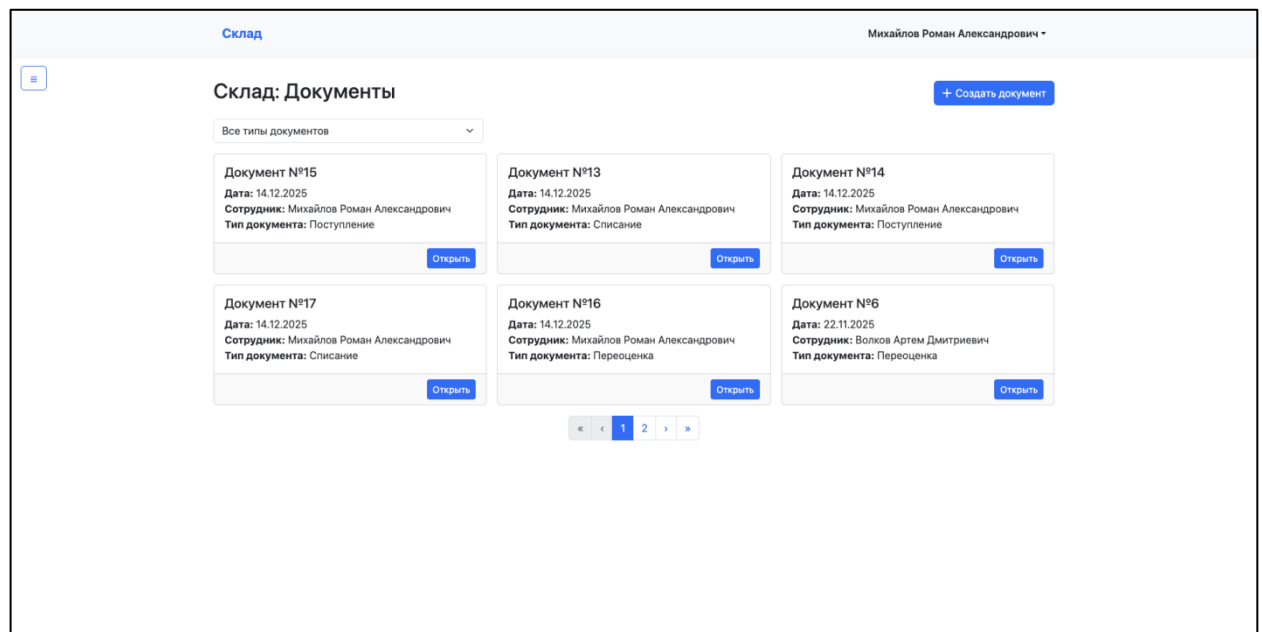


Рисунок Г.7 – Страница Документы

Склад

Михайлов Роман Александрович

Склад: Документ №15

Дата
14.12.2025

Тип документа
Поступление

Сохранить

Удалить

Содержимое

Партия	Количество
№10 — Офисное кресло "Director" 10 P	10
№11 — Материнская плата "Gamer XTREME" 134 P	5
Партия	

+ Добавить строку

Рисунок Г.8 – Страница Содержание документа

Склад

Михайлов Роман Александрович

Справочник: Сотрудники

Поиск по ФИО

Все должности

+ Добавить сотрудника

Волков Артем Дмитриевич
Начальник склада
Телефон: +7 911 123-45-67

Михайлов Роман Александрович
Системный администратор
Телефон: +7 921 694-19-53

Никитин Петр Алексеевич
Грузчик
Телефон: +7 911 765-43-21

Орлов Денис Романович
Грузчик
Телефон: +7 911 234-56-78

Павлов Иван Олегович
Грузчик
Телефон: +7 911 456-78-90

Соколова Анна Сергеевна
Кладовщик
Телефон: +7 911 987-65-43

* Для редактирования или удаления нажмите на карточку

Рисунок Г.9 – Страница Сотрудники

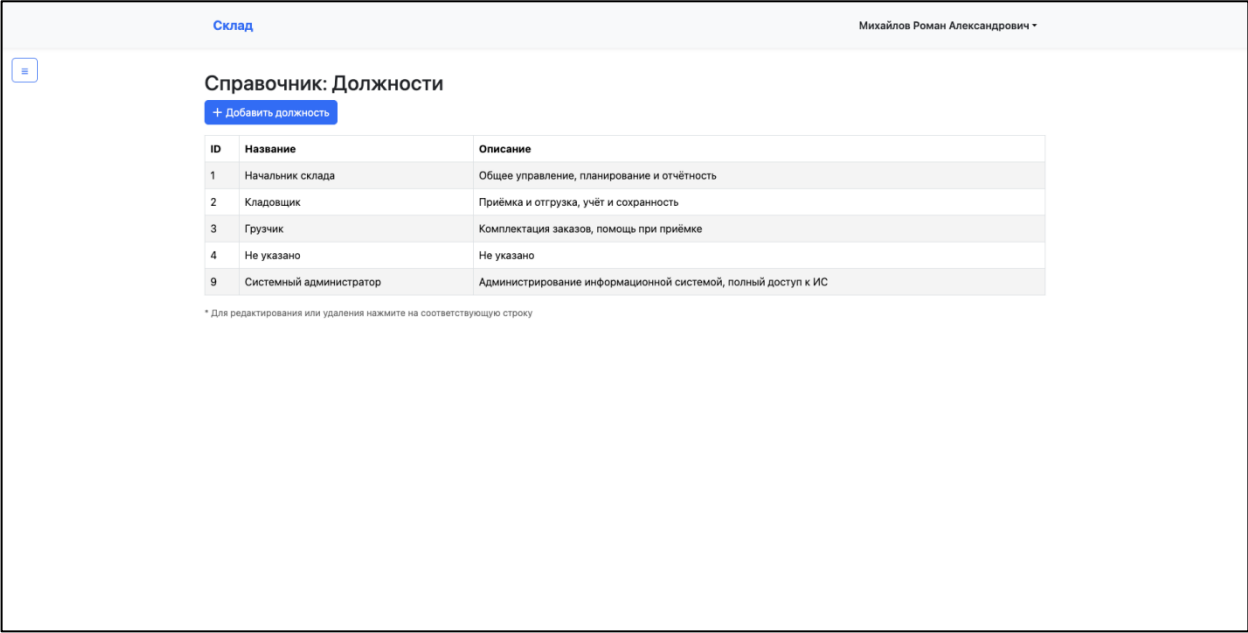


Рисунок Г.10 – Страница Должности

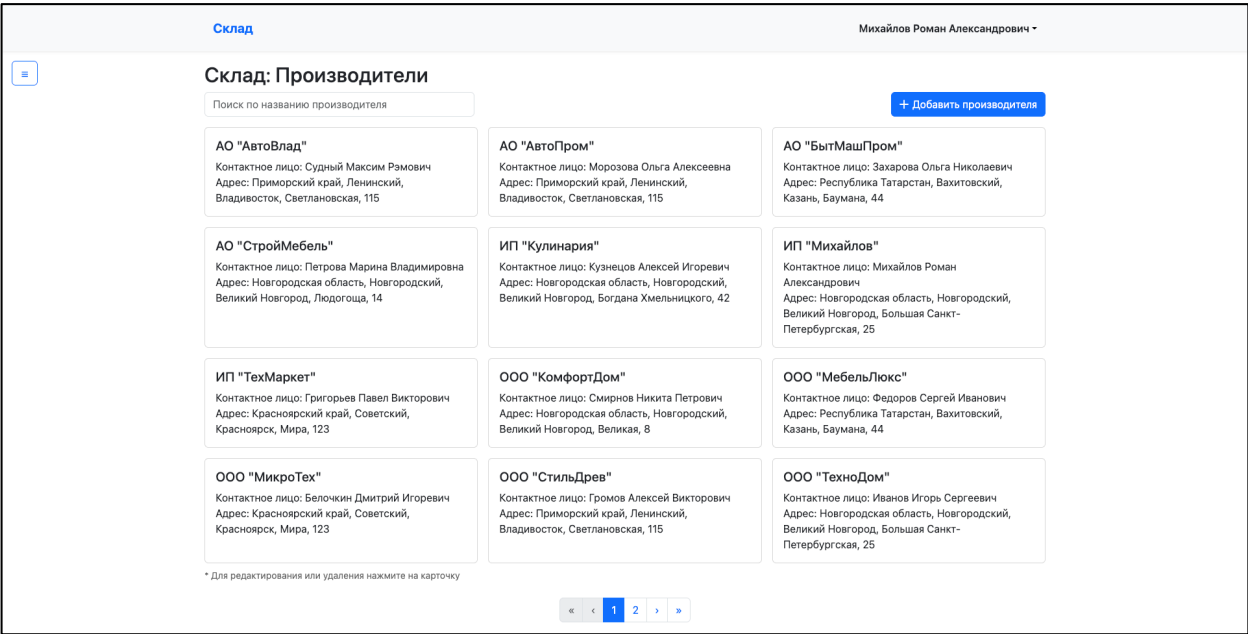


Рисунок Г.11 – Страница Адреса

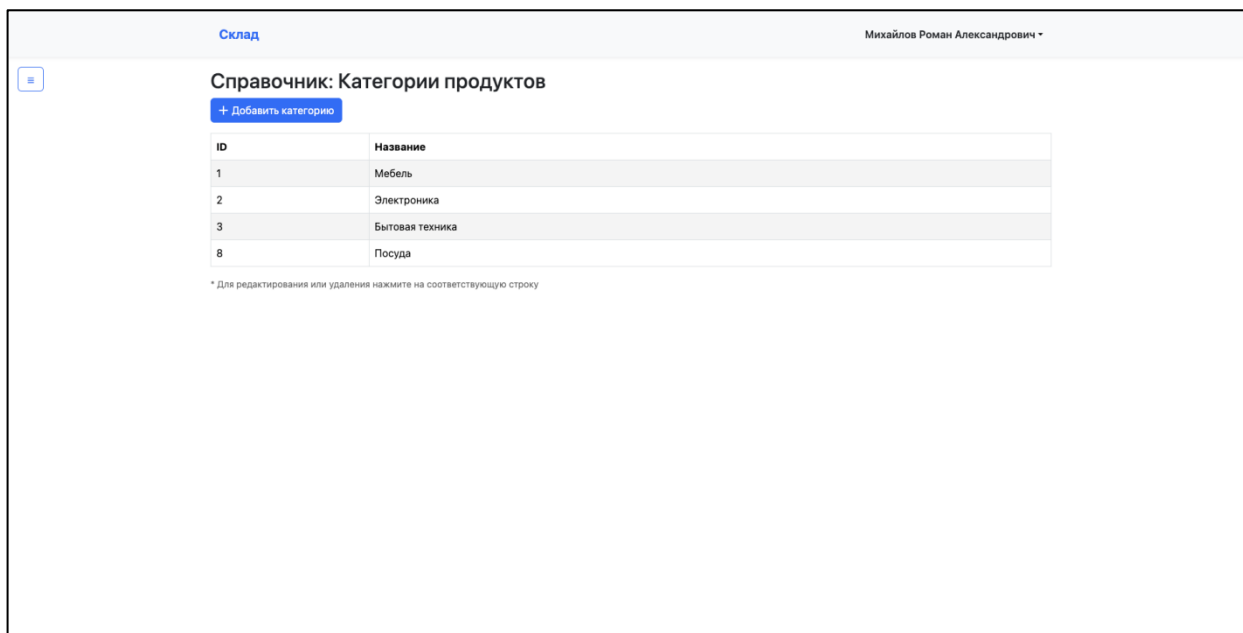


Рисунок Г.12 – Страница Категории продуктов

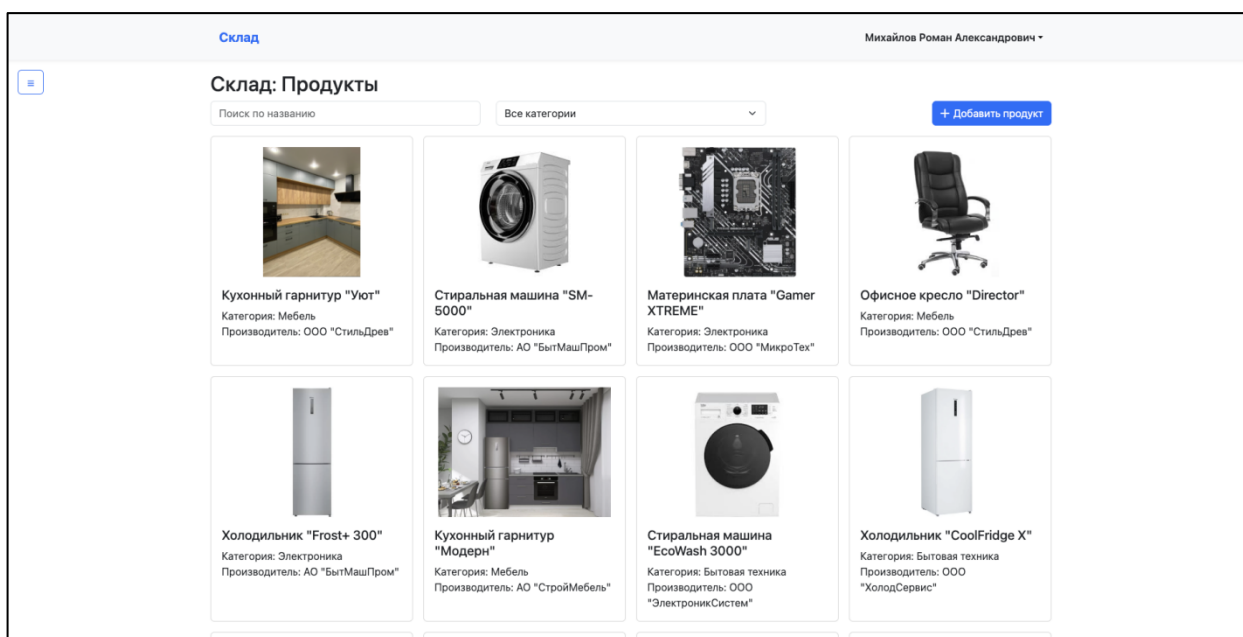


Рисунок Г.13 – Страница Продукты

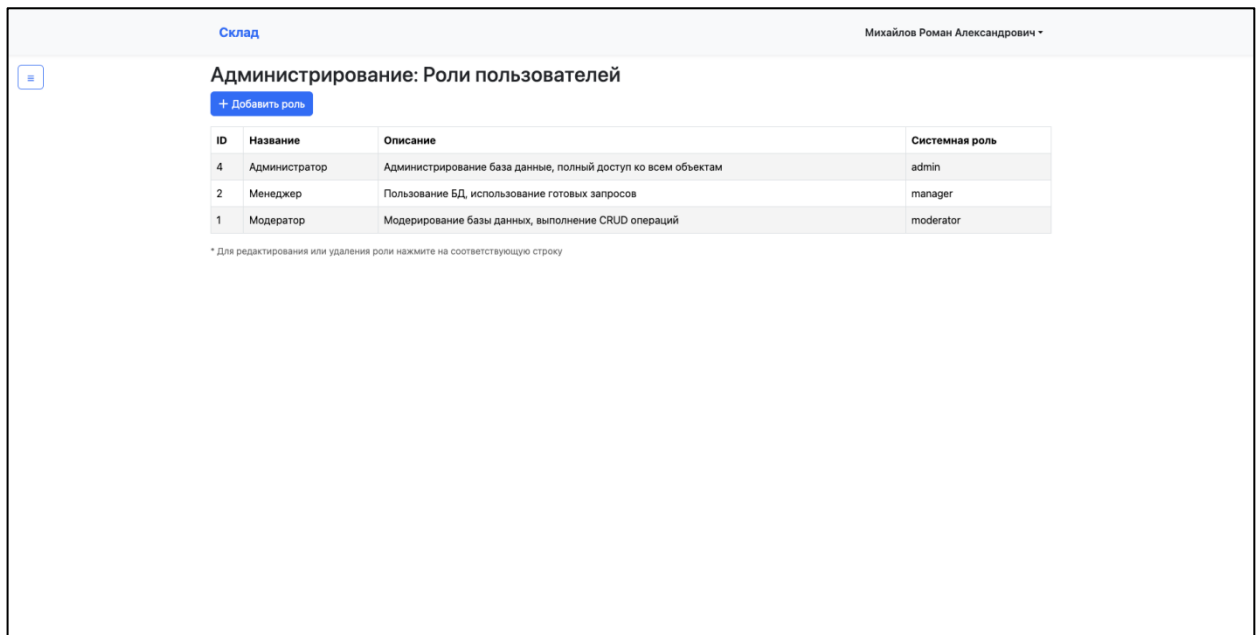


Рисунок Г.14 – Страница Роли

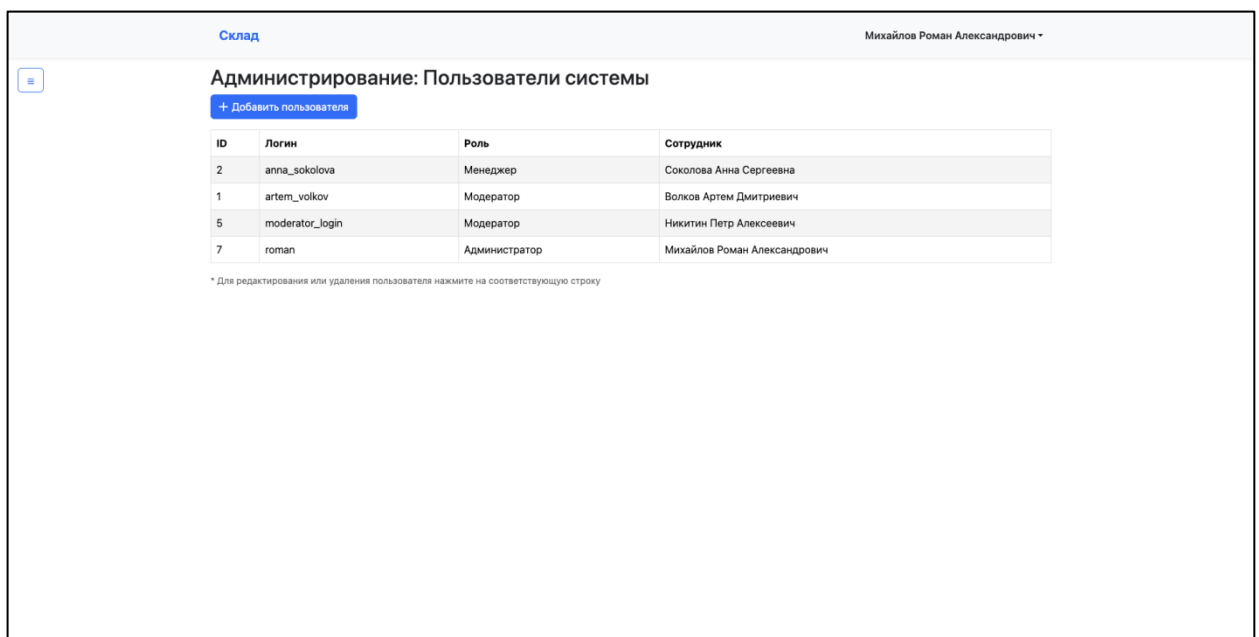


Рисунок Г.15 – Страница Пользователи системы

Склад

Михайлов Роман Александрович

Администрирование: Логи изменений

Фильтр по роли

Фильтр по действию (INSERT, UPDATE, DELETE)

Фильтр по таблице

«

1

2

3

4

5

6

7

8

»

ID	Таблица	Действие	Старые данные	Новые данные	Изменил	Время изменения
1013	refresh_tokens	INSERT	-	{ "created_at": "2025-12-16T13:27:22.335854", "id": 277, "role": "admin", "token": "01a39145828a77ae117d5914bd8897779cc657f25538f591c262da693df96e2a", "username": "roman" }	postgres	16.12.2025, 13:27:22
1012	refresh_tokens	DELETE	{ "created_at": "2025-12-16T13:25:17.188335", "id": 276, "role": "manager", "token": "bd6615564cae3eb619f4ff807c8d6f519512da12b8fcla20a0df3983ddb54776", "username": "anna_sokolova" }	-	postgres	16.12.2025, 13:27:18
1011	refresh_tokens	INSERT	-	{ "created_at": "2025-12-16T13:25:17.188335", "id": 276, "role": "manager", "token": "bd6615564cae3eb619f4ff807c8d6f519512da12b8fcla20a0df3983ddb54776", "username": "anna_sokolova" }	postgres	16.12.2025, 13:25:17

Рисунок Г.16 – Страница Логи

Склад

Михайлов Роман Александрович



Товаров

55

Поставщиков

14

Категорий

4

Сотрудников

6

Товары на складе



Кухонный гарнитур "Уют"



Стиральная машина "SM-5000"



Материнская плата "Gamer XTREME"



Офисное кресло "Director"

Рисунок Г.17 – Домашняя страница

Склад

Вход

Вход в систему

Логин

Введите логин

Пароль

Введите пароль

Войти

Рисунок Г.18 – Страница Вход

Склад

Михайлов Роман Александрович

☰

Профиль пользователя

Фамилия: Михайлов

Имя: Роман

Отчество: Александрович

Должность: Системный администратор

Адрес: Новгородская область, Новгородский, Великий Новгород, Большая Санкт-Петербургская, 25

Рисунок Г.19 – Страница Профиль

Редактировать роль

✕

Название

Менеджер

Описание

Пользование БД, использование готовых запросов

Системная роль

manager

⬅ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.20 – Форма Роль

Редактировать пользователя

✕

Логин

anna_sokolova

Пароль (оставьте пустым чтобы не менять)

Роль

Менеджер

Сотрудник

Соколова Анна Сергеевна

⬅ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.21 – Форма Пользователь

Редактировать категорию

ID

8

Название

Посуда

↩ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.22 – Форма Категория товара

Редактировать категорию

ID

1

Название

Поступление

Описание

Фиксация поступления разных партии товаров различных категорий

↩ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.23 – Форма Категория документа

Редактировать должность

Название

Системный администратор

Описание

Администрирование информационной системой, полный

↩ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.24 – Форма Должность

Редактировать адрес

Субъект

Новгородская область

Район

Новгородский

Город

Великий Новгород

Улица

Богдана Хмельницкого

Дом

42

↩ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.25 – Форма Адрес

Редактировать производителя

×

Название

ИП "Кулинария"

ИНН

4567890123

Фамилия

Кузнецов

Имя

Алексей

Отчество

Игоревич

Адрес

Новгородская область, Новгородский, Великий Новго| ▾

⬅ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.26 – Форма Производитель

Редактировать сотрудника

×

ID

4

Фамилия

Орлов

Имя

Денис

Отчество

Романович

Пол

М

▼

ИНН

525203456789

Телефон

+7 911 234-56-78

Адрес

Новгородская область, Новгородский, Великий Новго

▼

Дата рождения

17.02.1995

📅

Должность

Грузчик

▼

↩ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.27 – Форма Сотрудник

Редактировать продукт

ID

1

Название

Кухонный гарнитур "Уют"

Категория

Мебель

Производитель

ООО "СтильДрев"

Картинка

Выберите файл

Файл не выбран

↗ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.28 – Форма Продукт

Редактировать партию

Товар

Стиральная машина "SM-5000"

Стоимость

89990

Дата производства

05.03.2025

Срок годности

05.03.2028

↗ Отмена

💾 Сохранить

🗑 Удалить

Рисунок Г.28 – Форма Партия

Партии товаров

Сгенерирован автоматически из базы данных

ID	Название товара	Стоимость	Дата производства	Срок годности
1	Кухонный гарнитур "Уют"	125500	15.01.2024	15.01.2024
8	Кухонный гарнитур "Уют"	2	30.10.2025	30.01.2026
4	Кухонный гарнитур "Уют"	24300	20.01.2024	20.02.2034
5	Стиральная машина "SM-5000"	89990	05.03.2025	05.03.2028
2	Стиральная машина "SM-5000"	48800	01.02.2025	01.02.2027
11	Материнская плата "Gamer XTREME"	134	06.12.2024	05.12.2024
9	Материнская плата "Gamer XTREME"	2	30.01.2025	30.12.2025
3	Материнская плата "Gamer XTREME"	18900	20.02.2025	20.02.2029
10	Офисное кресло "Director"	10	29.11.2025	30.12.2025

Сотрудник: Михайлов Роман Александрович

Должность: Системный администратор

Дата генерации: 17.12.2025 • Формат: PDF • Источник: PostgreSQL (pgx)

Рисунок Г.29 – Отчёт Партии товаров

Непринятые партии

Сгенерирован автоматически из базы данных

№	ID партии	ID товара	Название товара
1	8	1	Кухонный гарнитур "Уют"
2	9	3	Материнская плата "Gamer XTREME"

Сотрудник: Михайлов Роман Александрович

Должность: Системный администратор

Дата генерации: 17.12.2025 • Формат: PDF • Источник: PostgreSQL (pgx)

Рисунок Г.30 – Отчёт Непринятые партии

Просроченные партии

Сгенерирован автоматически из базы данных

№	ID партии	Название товара	Срок годности	Остаток
1	1	Кухонный гарнитур "Уют"	15.01.2024	2
2	11	Материнская плата "Gamer XTREME"	05.12.2024	5

Сотрудник: Михайлов Роман Александрович

Должность: Системный администратор

Дата генерации: 17.12.2025 • Формат: PDF • Источник: PostgreSQL (pgx)

Рисунок Г.31 – Отчёт Просроченные партии

Остатки продуктов по партиям

Сгенерирован автоматически из базы данных

№	ID партии	ID продукта	Наименование продукта	Остаток
1	3	3	Материнская плата "Gamer XTREME"	10
2	11	3	Материнская плата "Gamer XTREME"	5
3	2	2	Стиральная машина "SM-5000"	3
4	1	1	Кухонный гарнитур "Уют"	2
5	8	1	Кухонный гарнитур "Уют"	0
6	10	4	Офисное кресло "Director"	0
7	4	1	Кухонный гарнитур "Уют"	0
8	5	2	Стиральная машина "SM-5000"	0
9	9	3	Материнская плата "Gamer XTREME"	0

Сотрудник: Михайлов Роман Александрович

Должность: Системный администратор

Дата генерации: 17.12.2025 • Формат: PDF • Источник: PostgreSQL (pgx)

Рисунок Г.32 – Отчёт Остатки по партиям

Отсутствующие товары

Сгенерирован автоматически из базы данных

№	ID	Название товара	Производитель
1	5	Холодильник "Frost+ 300"	АО "БытМашПром"
2	7	Кухонный гарнитур "Модерн"	АО "СтройМебель"
3	39	Кухонный гарнитур "Элегант"	АО "СтройМебель"
4	28	Кухонный стол "Classic"	АО "СтройМебель"
5	54	Кухонный стол "Modern"	АО "СтройМебель"
6	18	Тумба под ТВ "Classic"	АО "СтройМебель"
7	47	Шкаф для одежды "Classic Wardrobe"	АО "СтройМебель"
8	33	Кастрюля "CookMaster"	ИП "Кулинария"
9	46	Кастрюля "ProCook"	ИП "Кулинария"
10	12	Сковорода "Chef 28"	ИП "Кулинария"
11	55	Сковорода "Chef Classic"	ИП "Кулинария"
12	37	Сковорода "Chef Pro 30"	ИП "Кулинария"
13	22	Сковорода "PanExpert"	ИП "Кулинария"

Рисунок Г.33 – Отчёт отсутствующие товары

Остатки товаров

Сгенерирован автоматически из базы данных

№	Название товара	Производитель	Остаток на складе
1	Материнская плата "Gamer XTREME"	ООО "МикроТех"	15
2	Стиральная машина "SM-5000"	АО "БытМашПром"	3
3	Кухонный гарнитур "Уют"	ООО "СтильДрев"	2
4	Холодильник "Frost+ 300"	АО "БытМашПром"	0
5	Шкаф для одежды "Classic Wardrobe"	АО "СтройМебель"	0
6	Кухонный гарнитур "Элегант"	АО "СтройМебель"	0
7	Кухонный гарнитур "Модерн"	АО "СтройМебель"	0
8	Кухонный стол "Classic"	АО "СтройМебель"	0
9	Кухонный стол "Modern"	АО "СтройМебель"	0
10	Тумба под ТВ "Classic"	АО "СтройМебель"	0
11	Кастрюля "ProCook"	ИП "Кулинария"	0
12	Сковорода "Chef 28"	ИП "Кулинария"	0
13	Сковорода "PanExpert"	ИП "Кулинария"	0

Рисунок Г.34 – Отчёт Остатки продукции

Суммарная стоимость товаров

Сгенерирован автоматически из базы данных

№	ID партии	Название товара	Стоимость за 1 У.е.	Остаток на складе	Суммарная стоимость
1	1	Кухонный гарнитур "Уют"	125500	2	251000
2	3	Материнская плата "Gamer XTREME"	18900	10	189000
3	11	Материнская плата "Gamer XTREME"	134	5	670
4	2	Стиральная машина "SM-5000"	48800	3	146400
5	BCE	BCE	193334	20	587070

Сотрудник: Михайлов Роман Александрович

Должность: Системный администратор

Дата генерации: 17.12.2025 • Формат: PDF • Источник: PostgreSQL (pgx)

Рисунок Г. 35— Отчёт Стоимость товаров

Производители по регионам

Сгенерирован автоматически из базы данных

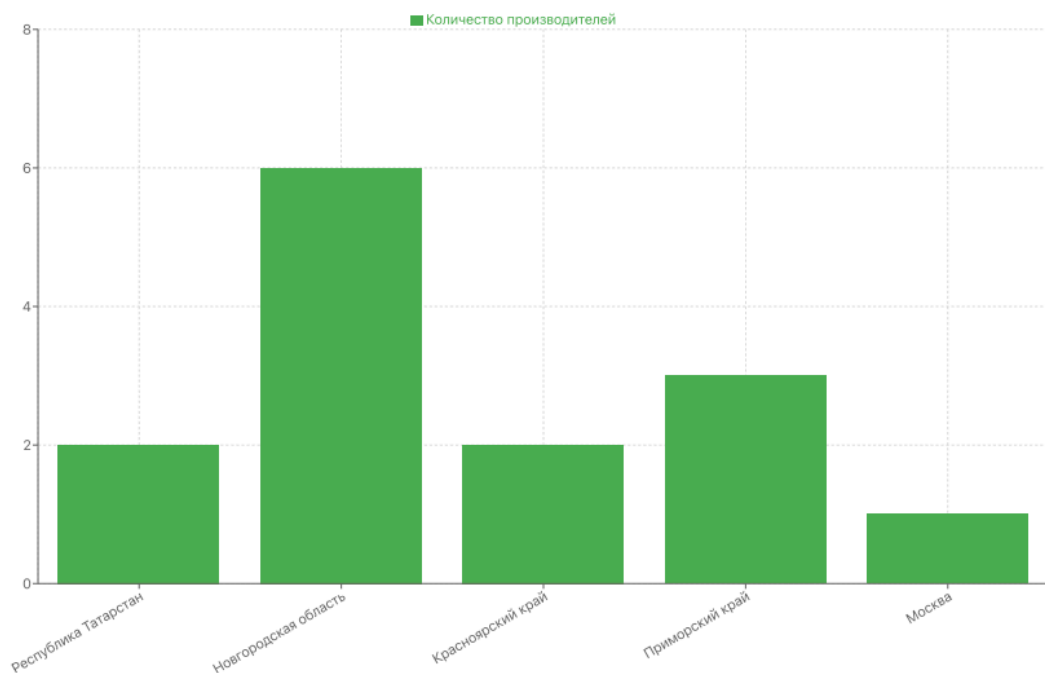


Рисунок Г.36 – Отчёт Производители по регионам

Сотрудники

Сгенерирован автоматически из базы данных

№	Фамилия	Имя	Отчество	Должность	Телефон	Дата рождения
1	Волков	Артем	Дмитриевич	Начальник склада	+7 911 123-45-67	12.05.1985
2	Михайлов	Роман	Александрович	Системный администратор	+7 921 694-19-53	22.07.2005
3	Никитин	Петр	Алексеевич	Грузчик	+7 911 765-43-21	30.07.1983
4	Орлов	Денис	Романович	Грузчик	+7 911 234-56-78	17.02.1995
5	Павлов	Иван	Олегович	Грузчик	+7 911 456-78-90	08.11.1988
6	Соколова	Анна	Сергеевна	Кладовщик	+7 911 987-65-43	23.08.1992

Сотрудник: Михайлов Роман Александрович

Должность: Системный администратор

Дата генерации: 17.12.2025 • Формат: PDF • Источник: PostgreSQL (pgx)

Рисунок Г.37 – Отчёт Сотрудники

Документы по сотрудникам

Сгенерирован автоматически из базы данных

Сотрудник	ФИО	Должность	Категория документа	Количество документов
1	Волков Артем Дмитриевич	Начальник склада	Переоценка	1
1	Волков Артем Дмитриевич	Начальник склада	Поступление	2
2	Михайлов Роман Александрович	Системный администратор	Поступление	2
2	Михайлов Роман Александрович	Системный администратор	Переоценка	1
2	Михайлов Роман Александрович	Системный администратор	Списание	2
3	Соколова Анна Сергеевна	Кладовщик	Списание	1
3	Соколова Анна Сергеевна	Кладовщик	Поступление	1
3	Соколова Анна Сергеевна	Кладовщик	Переоценка	1

Сотрудник: Михайлов Роман Александрович

Должность: Системный администратор

Дата генерации: 17.12.2025 • Формат: PDF • Источник: PostgreSQL (pgx)

Рисунок Г.38 – Отчёт Документы по сотрудникам

Привилегии пользователей

Сгенерирован автоматически из базы данных

Таблица	Администратор	Менеджер	Модератор
address	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE
audit_log	DELETE, INSERT, SELECT, UPDATE	DELETE, INSERT, SELECT, UPDATE	DELETE, INSERT, SELECT, UPDATE
batch	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
document	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
document_category	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE
document_content	DELETE, INSERT, SELECT, UPDATE	INSERT, SELECT, UPDATE	SELECT, UPDATE
employee	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE
gender	SELECT	SELECT	SELECT
position	DELETE, INSERT, SELECT, UPDATE	SELECT	SELECT, UPDATE

Рисунок Г.39 – Отчёт Права ролей

Права доступа по ролям

Сгенерирован автоматически из базы данных

Роль	Раздел	Права
admin	address	select, insert, update, delete
admin	audit_log	select, insert, update, delete
admin	batch	select, insert, update, delete
admin	document	select, insert, update, delete
admin	document_category	select, insert, update, delete
admin	document_content	select, insert, update, delete
admin	employee	select, insert, update, delete
admin	gender	select
admin	position	select, insert, update, delete
admin	producer	select, insert, update, delete
admin	product	select, insert, update, delete
admin	product_category	select, insert, update, delete
admin	refresh_tokens	select, insert, update, delete

Рисунок Г.40 – Отчёт Доступные разделы

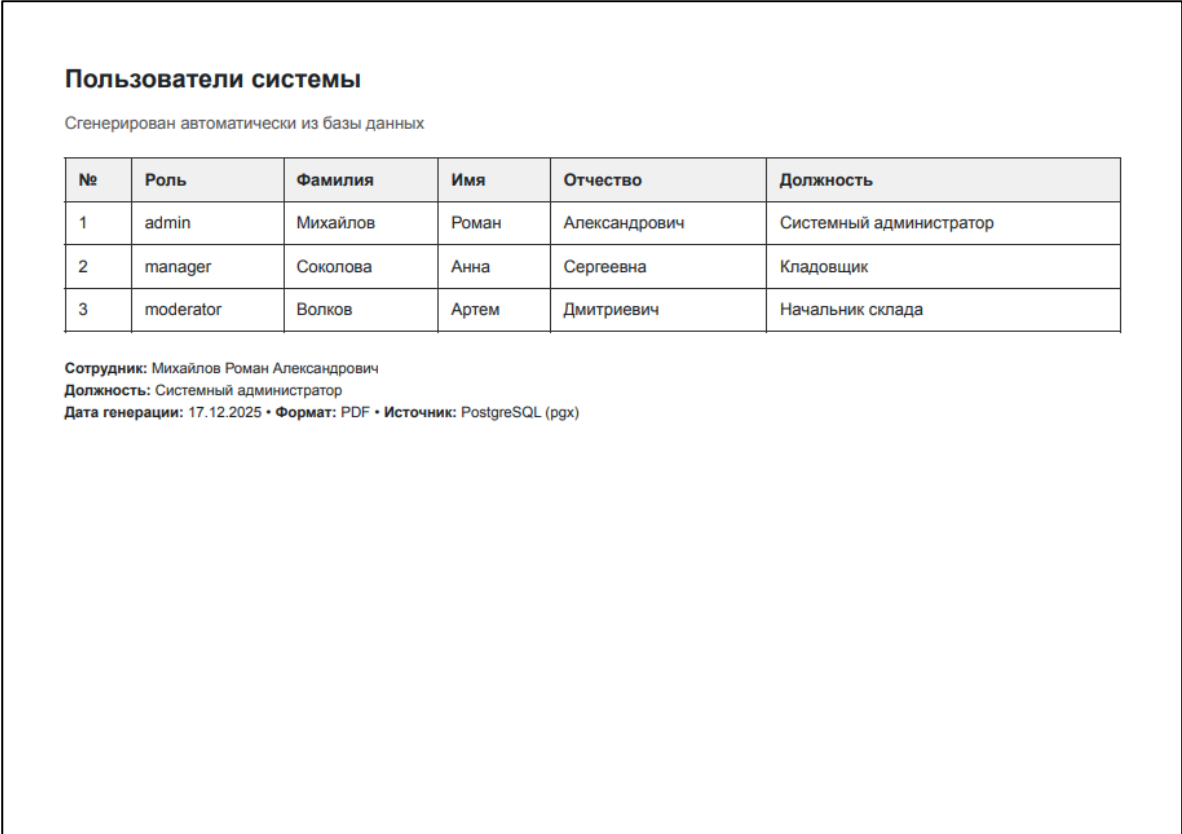


Рисунок Г.41 – Отчёт Пользователи системы

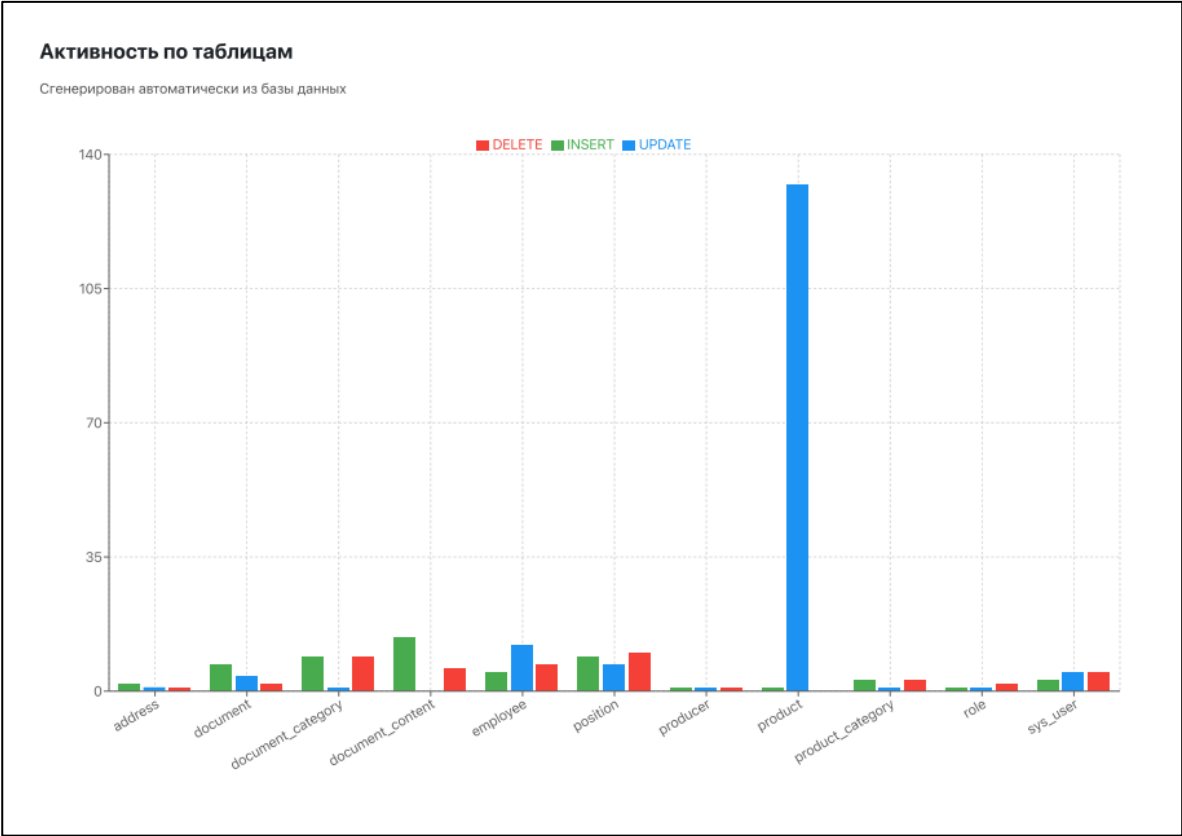


Рисунок Г.42 – Отчёт Активность по таблицам

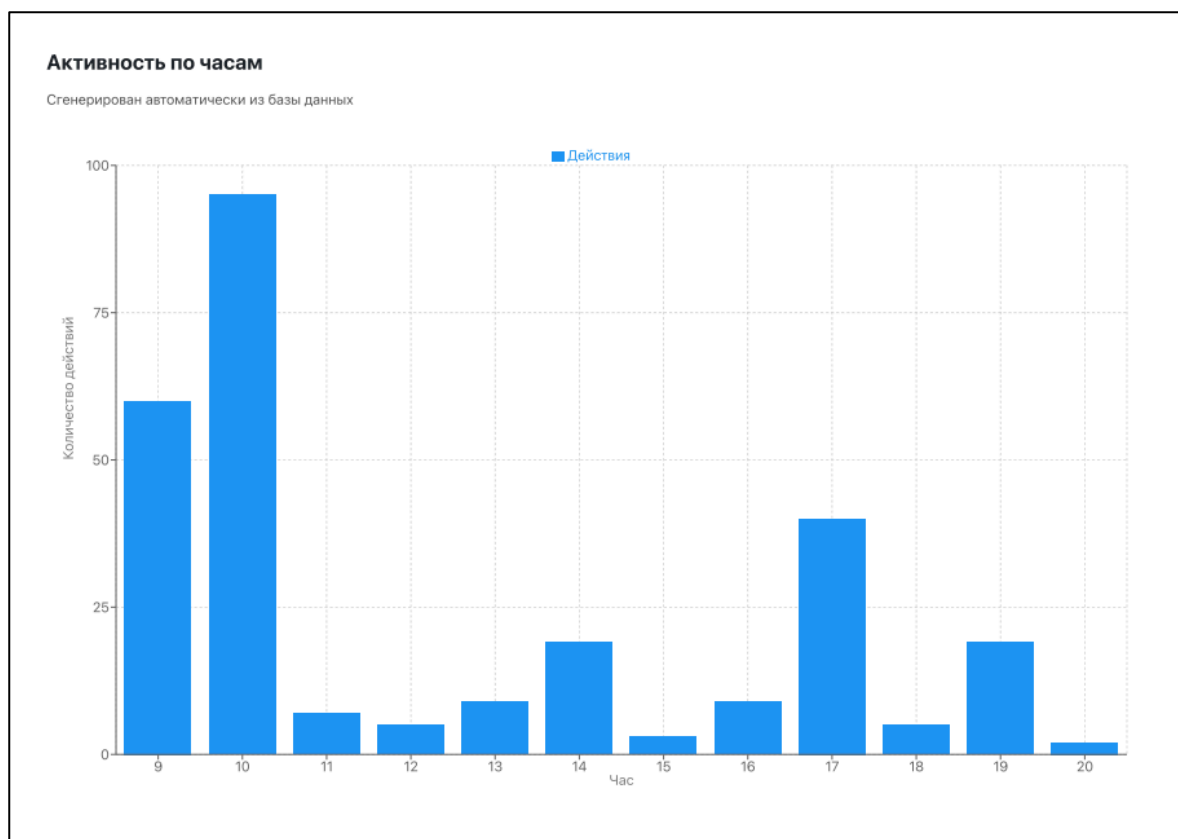


Рисунок Г.43 – Отчёт Активность по часам

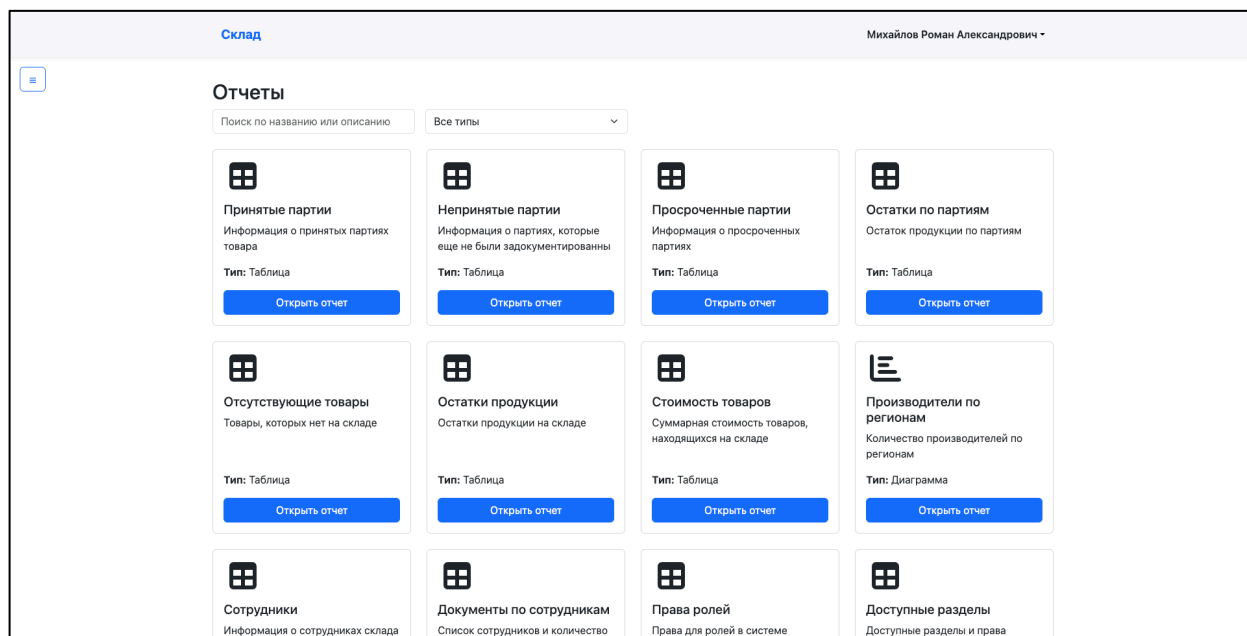


Рисунок Г.44 – Страница Все отчёты

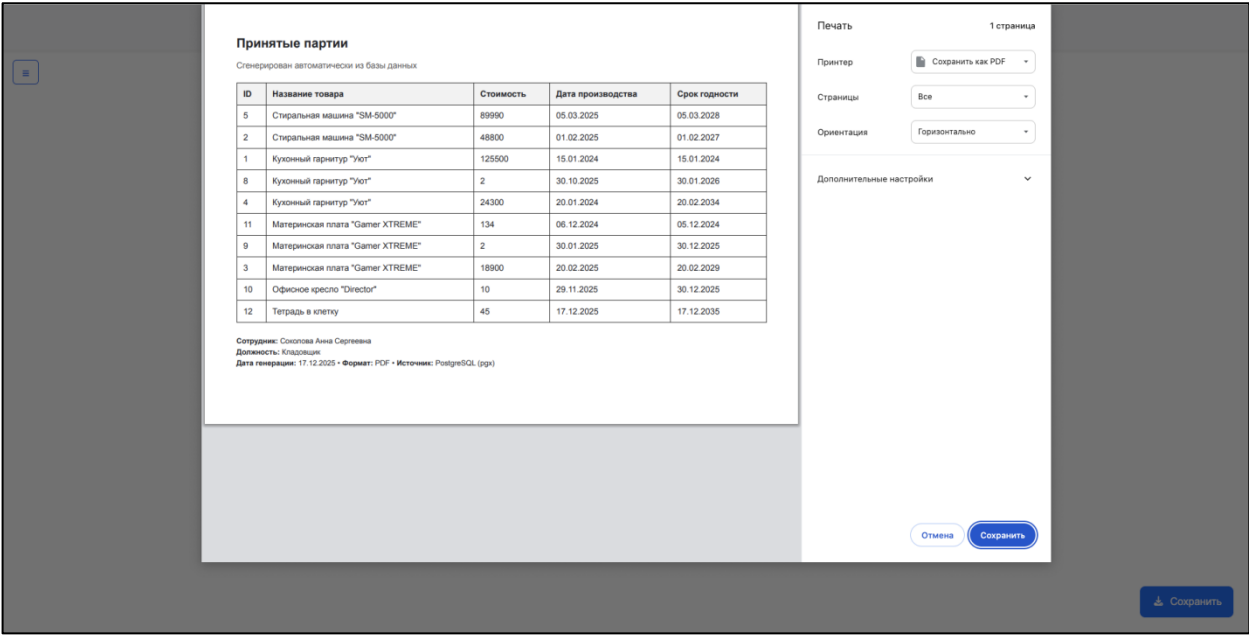


Рисунок Г.45 – Сохранение отчета

Приложение Д

(обязательное)

Д.1 – Листинг bash-скрипта start.sh для запуска приложения

```
#!/bin/bash

docker-compose up -d

docker ps

docker-compose logs -f nginx > logs/nginx.log 2>&1 &
docker-compose logs -f pgadmin > logs/pgadmin4.log 2>&1 &
docker-compose logs -f warehouse_db > logs/postgresql.log 2>&1 &
docker-compose logs -f server > logs/backend.log 2>&1 &
docker-compose logs -f frontend > logs/frontend.log 2>&1 &
```

Д.2 – Листинг bash-скрипта stop.sh для остановки приложения

```
#!/bin/bash

docker-compose down
```

Д.3 – Листинг bash-скрипта debug.sh для отладки приложения

```
#!/bin/bash

echo "Запуск Docker Compose..."
docker-compose up --build -d

echo "Контейнеры запущены!"
docker ps

if [ -n "$1" ]; then
DUMP_FILE="$1"

CONTAINER="storage_db-warehouse_db-1"

echo "Ожидание запуска PostgreSQL..."
until docker exec "$CONTAINER" pg_isready -U postgres > /dev/null 2>&1; do
sleep 1
done

echo "PostgreSQL готов!"
echo "Инициализация базы данных из файла $DUMP_FILE ..."

docker exec -i "$CONTAINER" psql -U postgres -d warehouse < "$DUMP_FILE"
```



```
echo "Импорт завершён!"
fi
```

```
docker-compose logs -f nginx > logs/nginx.log 2>&1 &
docker-compose logs -f pgadmin > logs/pgadmin4.log 2>&1 &
docker-compose logs -f warehouse_db > logs/postgresql.log 2>&1 &
docker-compose logs -f server > logs/backend.log 2>&1 &
docker-compose logs -f frontend > logs/frontend.log 2>&1 &
```

Д.4 – Листинг bash-скрипта make_dump.sh для создания дампа базы данных

```
#!/bin/bash
REMOVE_VOLUME=false

if [ -n "$1" ]; then
DUMP_FILE="$1"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DUMP_FILE_WITH_TIME="${DUMP_FILE}_${TIMESTAMP}.sql"

# Создаём дамп ролей и пишем в файл
echo "Создаём дамп ролей в файл $DUMP_FILE_WITH_TIME ..."
docker exec -i storage_db-warehouse_db-1 pg_dumpall -U postgres --globals-only > "$DUMP_FILE_WITH_TIME"

if [ $? -eq 0 ]; then
echo "Дамп ролей успешно создан."
else
echo "Ошибка при создании дампа ролей. Volume не будет удалён."
REMOVE_VOLUME=false
echo "Остановка Docker Compose..."
docker-compose down
exit 1
fi

# Создаём дамп базы и добавляем в тот же файл
echo "Создаём дамп базы и добавляем в файл $DUMP_FILE_WITH_TIME ..."
docker exec -i storage_db-warehouse_db-1 pg_dump -F p -U postgres -d warehouse >> "$DUMP_FILE_WITH_TIME"

if [ $? -eq 0 ]; then
echo "Дамп базы успешно добавлен в файл: $DUMP_FILE_WITH_TIME"
REMOVE_VOLUME=true
else
echo "Ошибка при создании дампа базы. Volume не будет удалён."
REMOVE_VOLUME=false
fi
else
echo "Дамп базы не создаётся, путь к файлу не указан."
fi
```

Д.5 – Листинг bash-скрипта backup.sh для переноса информационной системы

```
#!/bin/bash
set -e

PROJECT_NAME="warehouse"
DB_CONTAINER="warehouse_db"
DB_NAME="warehouse"
DB_USER="postgres"

IMAGES_VOLUME="product_images"

WORKDIR="warehouse_backup"
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
ARCHIVE="warehouse_backup_${TIMESTAMP}.tar.gz"

SCRIPTS=(
start.sh
stop.sh
make_dump.sh
debug.sh
)

rm -rf "$WORKDIR"
mkdir -p "$WORKDIR"

docker compose exec -T "$DB_CONTAINER" \
pg_dumpall -U "$DB_USER" --globals-only \
> "$WORKDIR/roles.sql"

docker compose exec -T "$DB_CONTAINER" \
pg_dump -U "$DB_USER" "$DB_NAME" \
> "$WORKDIR/db_dump.sql"

docker run --rm \
-v ${PROJECT_NAME}_${IMAGES_VOLUME}:/data \
-v "$(pwd)/$WORKDIR":/backup \
alpine \
tar czf /backup/product_images.tar.gz -C /data .

cp docker-compose.yml "$WORKDIR/"

rsync -a \
--exclude node_modules \
--exclude dist \
--exclude build \
--exclude .git \
backend frontend nginx \
```

```

"$WORKDIR/"

for script in "${SCRIPTS[@]}; do
[ -f "$script" ] && cp "$script" "$WORKDIR/"
done

cat << 'EOF' > "$WORKDIR/restore.sh"
#!/bin/bash
set -e

docker compose up -d warehouse_db
sleep 10

if [ -f roles.sql ]; then
docker compose exec -T warehouse_db \
psql -U postgres < roles.sql
fi

docker compose exec -T warehouse_db \
psql -U postgres -d warehouse < db_dump.sql

docker run --rm \
-v warehouse_product_images:/data \
-v $(pwd):/backup \
alpine \
tar xzf /backup/product_images.tar.gz -C /data

docker compose up -d

EOF

chmod +x "$WORKDIR/restore.sh"

tar czf "$ARCHIVE" "$WORKDIR"
rm -rf "$WORKDIR"

```

Д.6 – Листинг docker-compose.yml

```

services:

  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "5050:80"
    volumes:
      - pgadmin_data:/var/lib/pgadmin
    networks:
      - backend

  warehouse_db:
    image: postgres:15
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: warehouse
    ports:
      - "8888:5432"
    volumes:
      - warehouse_db_data:/var/lib/postgresql/data
    networks:
      - backend

  server:
    build: ./backend
    depends_on:
      - warehouse_db
    env_file:
      - ./backend/.env
    volumes:
      - product_images:/app/uploads/products
    networks:
      - backend

  nginx:
    build: ./nginx
    ports:
      - "8080:80"
    depends_on:
      - frontend
      - server
    networks:
      - backend

```

```
frontend:
build: ./frontend
command: npm run dev -- --host 0.0.0.0 --port 5173
depends_on:
- server
networks:
- backend

volumes:
warehouse_db_data:
pgadmin_data:
product_images:

networks:
backend:
driver: bridge
```